

Mount Kenya University

Unlocking Infinite Possibilities

Name: CHANDLER MATERE SIMIYU

Admission Number: BSCCS/2024/58571

Unit Title: ADVANCED WEB DESIGN & DEVELOPMENT

Unit Code: BIT3208

Unit Name: Advanced Web Design and Development

Lecturer: MR. NYORO MICHAEL

Year III, Semester II

Project Title: NoteKeeper - Full Stack Web Application

Technologies Used: [Node.js](#) + Express + MySQL

GitHub Link: <https://github.com/Simiyuu/notes-app>

Week 1: Local Environment Setup.....	8
Fig 1: XAMPP Control Panel showing the local development environment installed on the machine.....	9
Fig 2: XAMPP Control Panel showing Apache and MySQL running on their default ports...	10
Fig 3: Browser showing the XAMPP localhost welcome page.....	10
Fig 4: Hello World Test.....	11
Fig 5: Database Connection Test.....	11
Week 2: Wireframes and GUI Design.....	12
Fig 1: Figma wireframe detailing the planned layout and user interface for the NoteKeeper login page.....	12
Fig 2: Browser showing the Login page with username and password fields used to authenticate registered users.....	13
Fig 3: Mobile View Mockup.....	14
Fig 4: Color Theme and Navigation Structure.....	15
Week 3: JavaScript and PHP Basics.....	16
Fig1: Javascript Form Validation.....	16
Fig 2: Password Strength Checker.....	17
Fig 3: Shows the Express.js route syntax used in server.js to handle HTTP requests.....	18
Fig 4 shows the database connection configuration and initDB function in server.js.....	19
Fig 5: Dynamic User Input Handling.....	19
Week 4 Task Schedule.....	20
Task 1 - Understanding Server-Side Programming.....	20
Fig 1: Simple Processing Page.....	20
Fig 2: Dynamic Welcome Page.....	20
Fig3: Form Submission System.....	21
Task 2 - HTML Forms and PHP Integration.....	22
Fig 1: Registration Form.....	22
Fig 2: Login Form.....	22
Task 3 - Simple Authentication.....	23
Fig 1: Basic Login Page.....	23
Fig 2: Username/passwords validation.....	24
Fig 3: Session-based welcome page.....	25
Task 4 - Backend Folder Organization.....	25
Fig 1: Project Structure.....	25
Task 5 - Introduction to Modern Backend Alternatives.....	26
Fig : PHP Modern Alternative (Node.js).....	26
Week 5 - Advanced Web Design and Development.....	27
Task 1: Database Creation.....	27
Fig 1: notes-app database.....	27
Task 2: Creating Tables.....	27
Fig 1: notes-app tables.....	27

Task 3: CRUD Operations.....	28
Fig 1: Add Records (INSERT).....	28
Fig 2: View Records (SELECT).....	28
Fig 3: Delete Records (DELETE).....	28
Task 4: Connecting PHP to Database.....	29
Fig 1: Database Connection.....	29
Task 5: Fetching Database Records.....	29
Fig 1: Database Records.....	29
Creativity: Note Counter.....	30
Fig 1: shows the dynamic note count badge in the sidebar header displaying the total number of notes.....	30
Week 6: Database Integration and CRUD Operations.....	31
Practical Task 1.....	31
Fig 1: Add a Note (INSERT).....	31
Fig 2: View Notes (SELECT).....	31
Fig 3: Edit a Note (UPDATE).....	32
Fig 4: Delete a Note (DELETE).....	32
Fig 5: Database Export.....	32
Practical Task 2.....	33
Fig 1: Note with Category Added.....	33
Fig 2: Records Stored with Category.....	33
Fig 3: View Records.....	33
Fig 4: Edit a Record.....	34
Fig 5: Delete a Record.....	34
Practical Task 3 (Challenge Task).....	35
Fig 1: User Registration.....	35
Fig 2: User Login.....	35
Fig 3: View Records.....	36
Fig 4: Search Functionality.....	36
Fig 5: Edit Record.....	37
Fig 6: Delete Record.....	37
Fig 7: Responsive Design.....	37
Fig 8: Form Validation.....	38
Week 7: User Authentication and Session Management.....	39
Practical Task 1: Student Portal Login System.....	39
Fig 1: Registration.....	39
Fig 2: Login.....	40
Fig 3: Dashboard.....	41
Fig 4: Logout.....	41
Fig 5: Database Script.....	42
Practical Task 2: Employee Portal	

Fig 1: Registration.....	43
Fig 2: Authentication.....	44
Fig 3: Protected Page.....	44
Fig 4: Session Management.....	45
Fig 5: Session Deleted on Logout.....	45
Practical Task 3: Multi-User Authentication System.....	45
Fig 1: Role Selection on Register.....	45
Fig 2: Register Three Users.....	47
Fig 3: Role Badge on Dashboard.....	47
Fig 4: Session Management.....	47
Fig 5: Access Restriction.....	47
Bonus Features.....	48
1. Remember Me: localStorage Keeps Session.....	48
2. Email Verification.....	48
3. Password Reset.....	50
4. 2FA: Google Authenticator.....	53
5. Profile Management.....	55
Week 8: Responsive Web Design and Mobile-First Development.....	57
Task 1: Create a Responsive Personal Profile Page.....	57
Fig 1: Desktop view of the personal profile page showing the hero section with responsive profile image, name, bio and navigation built using HTML, CSS Flexbox and a blue gradient theme.....	57
Fig 2: Tablet view of the profile page showing the responsive layout adjusting to a narrower viewport with the profile image stacking above the text content using CSS media queries at 768px breakpoint.....	57
Fig 3: Mobile view of the profile page showing the single column layout with the hamburger navigation menu and centered content, demonstrating the mobile first responsive design at the 480px breakpoint.....	59
Task 2: Build a Responsive Product Showcase.....	60
Fig 1: Desktop view of the product showcase displaying four fashion products.....	60
Fig 2: Tablet view of the product showcase at the 600px breakpoint showing two products per row.....	61
Fig 3: Mobile view of the product showcase showing one product per row approach as the base layout.....	62
Fig 4: Desktop hero section of the DRIP fashion showcase showing the responsive typography.....	63
Extra Practical Assignment.....	64
Fig. 1: Mobile Viewport Rendering of the Hero Section and Collapsed Navigation Menu...	64
Fig. 2: Desktop Viewport Demonstrating the 3-Column CSS Grid Implementation for Services.....	65
Fig. 3: Tablet/Desktop Viewport Demonstrating the 2-Column Grid Layout for the Contact Section.....	66

Week 9: Java Web Services: Servlet Lifecycle, Session Management, and Cookies..... 67

Practical Activity: Student Login System..... 67

Fig 1: Browser showing the NoteKeeper login form with username and password fields demonstrating the login page equivalent to login.jsp..... 67

Fig 2: Browser showing the NoteKeeper dashboard after successful login displaying the welcome message with the logged in username confirming session creation.....68

Fig 3: VS Code showing the requireAuth middleware in server.js demonstrating how unauthenticated requests are blocked and sessions are validated on every protected route..... 68

Fig 4: phpMyAdmin Browse view of the sessions table showing the active session token linked to the user_id confirming server side session creation after login..... 69

Fig 5: Browser showing the login page after clicking logout confirming the session was destroyed and the user was redirected back to the login page.....69

Class Exercise 1: Student Login Web Application..... 69

Fig 1: Browser showing the NoteKeeper register and login tabs demonstrating the login page with username and password input fields..... 69

Fig 2: Browser showing JavaScript validation error when submitting empty username field confirming client side validation is implemented..... 70

Fig 3: Browser showing the dashboard after successful login with the username displayed in the header confirming the welcome page and session username display.. 70

Fig 4: phpMyAdmin Browse view of the sessions table showing a newly created session token and its creation timestamp after login..... 70

Fig 5: Browser showing the login page after clicking the logout button confirming the session was invalidated and the user was redirected..... 71

Class Exercise 2: Session Tracking and Cookies Enhancement..... 71

Fig 1: Browser DevTools Application tab showing the token stored in localStorage confirming the logged in username is tracked client side without cookies.....71

Fig 2: Browser showing the NoteKeeper dark theme applied across all pages demonstrating the theme preference stored via CSS variables in styles.css..... 71

Fig 3: Browser DevTools Application tab showing the token and username values in localStorage confirming session ID equivalent and login persistence..... 72

Fig 4: VS Code showing the login route in server.js with the is_verified check and session token INSERT query demonstrating how login time is recorded via the created_at timestamp in the sessions table.....72

Assignment: Remember Me Feature..... 72

Fig 1: Browser showing the NoteKeeper login page.....72

Fig 2: Browser DevTools Application tab showing token and username persisted in localStorage after login confirming the Remember Me functionality.....73

Fig 3: Browser showing the app still logged in after closing and reopening confirming the Remember Me feature works across browser sessions..... 73

Fig 4: Browser showing the login page after logout confirming the token was cleared from both localStorage and the MySQL sessions table..... 73

Creativity Section..... 74

Fig 1: Gmail inbox showing the NoteKeeper email verification message sent via Nodemailer demonstrating an additional security layer not present in basic servlet

implementations.....	74
Fig 2: phpMyAdmin Browse view of the users table showing is_verified=1 confirming only verified users can create sessions.....	74
Student Summary.....	74
Week Reflection.....	75
WEEK 10: Dynamic HTML Generation and JDBC Basics.....	76
Practical Activity: Dynamic HTML and JDBC Basics.....	76
Fig 1: phpMyAdmin showing the notes_app database with all three tables listed demonstrating the database creation equivalent to creating studentdb.....	76
Fig 2: VS Code showing the DB_CONFIG object and initDB() function in server.js demonstrating the MySQL connection equivalent to JDBC connection setup.....	76
Fig 3: VS Code showing the CREATE TABLE statements in server.js demonstrating how the users and notes tables are created programmatically on server startup.....	77
Fig 4: Browser showing the NoteKeeper note creation form with title, category and content fields demonstrating the web form used to insert records into the database.....	77
Fig 5: Browser showing the NoteKeeper dashboard sidebar dynamically rendering all notes fetched from MySQL demonstrating dynamic HTML generation equivalent to JSP page display.....	78
Fig 6: phpMyAdmin Browse view of the notes table showing all inserted records with their id, user_id, title, content, category and created_at columns confirming successful database insertion.....	78
Creativity Section.....	79
Fig 1: Browser showing the Administrator dashboard with the dynamically generated users table pulled from MySQL confirming dynamic HTML generation based on database records.....	79
Fig 2: Browser showing the Lecturer dashboard with the dynamically generated student notes table confirming role-based dynamic content rendering from the database.....	79
Student Summary.....	79
Week Reflection.....	80
WEEK 11: Building Dynamic Pages with Full Database Integration (CRUD).....	80
Practical Activity — Full CRUD Implementation.....	80
Fig 1: Browser showing the NoteKeeper note creation form demonstrating the Add Student equivalent with title, category and content fields.....	80
Fig 2: Browser showing the sidebar listing all notes dynamically fetched from MySQL demonstrating the View All Students equivalent.....	81
Fig 3: Browser showing the edit form pre-filled with existing note data and the Update Note button demonstrating the Update Student Details equivalent.....	82
Fig 4: Browser showing the delete confirmation dialog appearing before a note is removed demonstrating the Delete with Confirmation requirement.....	82
Fig 5: Browser showing the search bar with a query typed and filtered notes appearing in real time demonstrating the Search by Name equivalent.....	83
Fig 6: Browser showing the green success message after saving a note and the red error message after failed validation demonstrating success and error message display... 83	
Creativity Section.....	84

Fig 1: Browser showing the category dropdown with General, Work, Personal and Study options demonstrating course-based organization equivalent to searching students by course.....	84
Fig 2: phpMyAdmin Browse view of the notes table showing the category column confirming category data is stored and retrievable from the database.....	85
Student Reflection.....	85
Week Summary.....	85
Week 12: Web Programming.....	86
Task 1: ASP.NET Student Management Module.....	86
Fig 1: Browser showing the NoteKeeper login page demonstrating student login equivalent to ASP.NET login form connected to SQL Server.....	86
Fig 2: Browser showing the NoteKeeper registration form with username, email, role dropdown and password fields demonstrating student registration with course selection equivalent to ASP.NET DropDownList.....	87
Fig 3: Browser showing the NoteKeeper profile panel displaying username, email, role, verification status and member since date demonstrating the student dashboard with personal information.....	88
Fig 4: Browser showing the Administrator dashboard with the dynamically generated users table demonstrating the GridView equivalent showing all registered students.....	88
Fig 5: phpMyAdmin Browse view of the sessions table showing active session tokens demonstrating server side session management equivalent to ASP.NET Session object...	89
Student Reflection.....	89
Week Summary.....	89
Creativity Section.....	90
Fig 1: VS Code showing the requireAuth middleware and session token validation in server.js demonstrating how Node.js session management compares to ASP.NET, Java Servlets and PHP session handling.....	90
Fig 2: phpMyAdmin showing the sessions table structure with token, user_id and created_at columns demonstrating database-backed session storage as an alternative to server memory storage used in ASP.NET and Java.....	91
Comparison Table.....	91
WEEK 13: Special Topics in Web Programming: COM, CORBA, RMI, LDAP, and Search Engines.....	92
Assignment: Enterprise Level Student Management.....	92
Fig 1: Browser showing the NoteKeeper login form.....	92
Fig 2: Browser showing the registration form with the role dropdown.....	93
Fig 3: Browser showing the Administrator dashboard with the users table.....	94
Fig 4: Browser showing the Lecturer dashboard with the read-only student notes panel....	94
Fig 5: Browser showing the Student dashboard with only their own notes visible.....	95
AUTHENTICATION COMPARISON TABLE.....	95
Creativity Section: NoteKeeper as an LDAP-Equivalent Authentication System.....	96
Fig 1: Browser showing the email verification flow registration success message and	

Gmail inbox with verification email.....	96
Fig 2: VS Code showing the login route in server.js with the is_verified check.....	97
Student Summary.....	97
Week Reflection.....	98

Week 1: Local Environment Setup

Fig 1: XAMPP Control Panel showing the local development environment installed on the machine

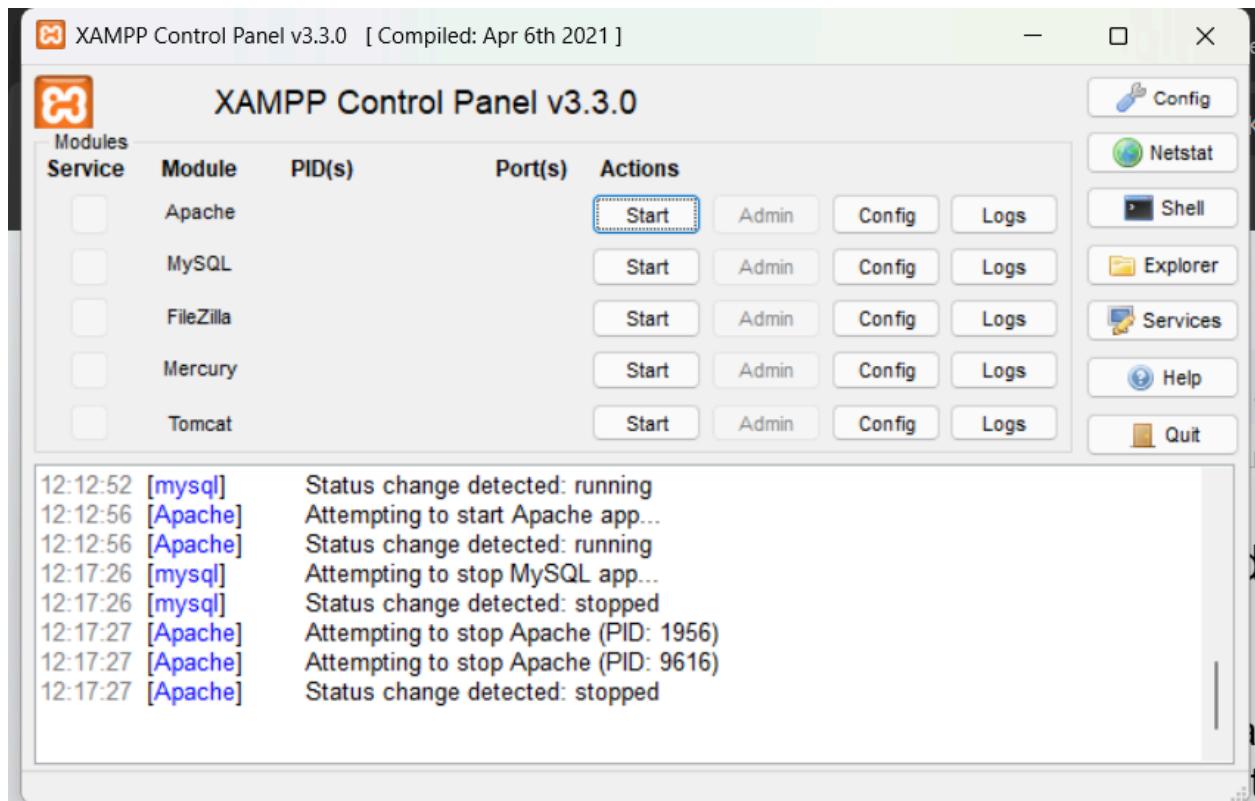


Fig 2: XAMPP Control Panel showing Apache and MySQL running on their default ports

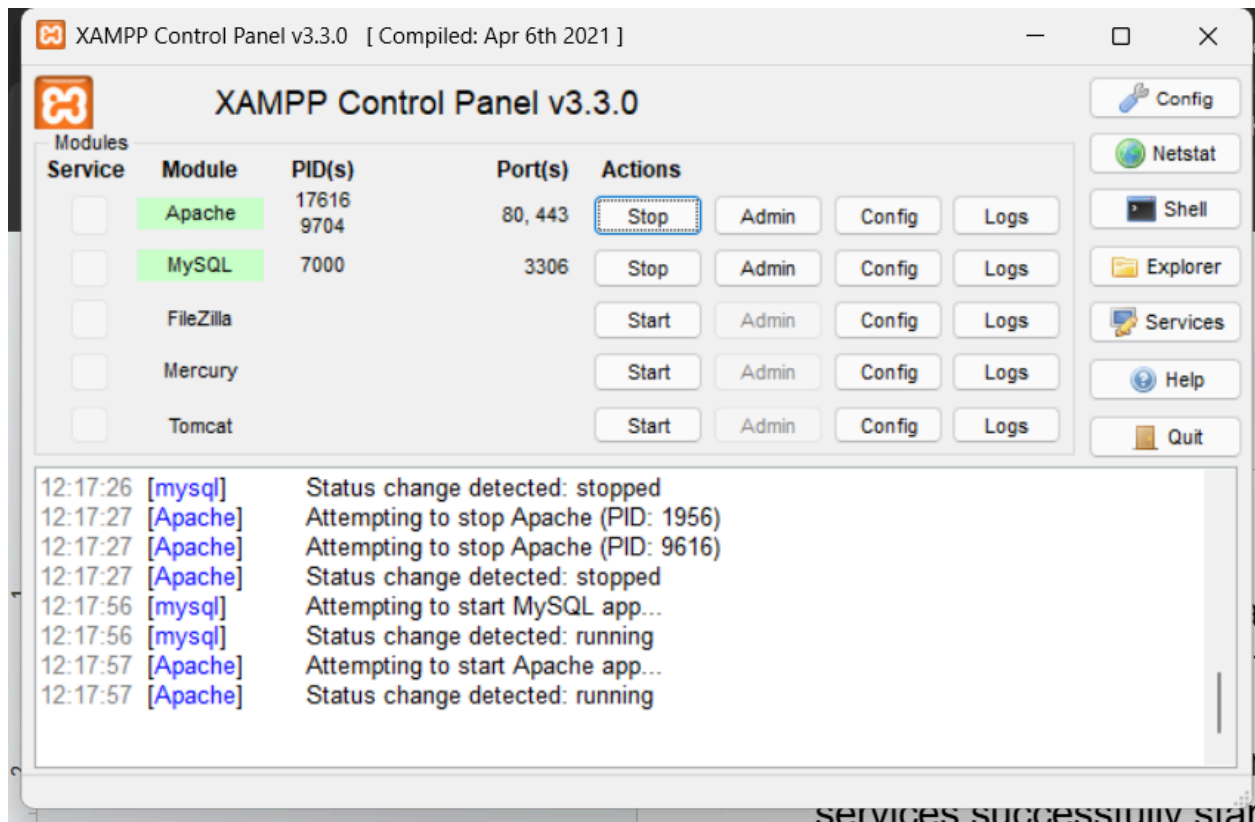


Fig 3: Browser showing the XAMPP localhost welcome page

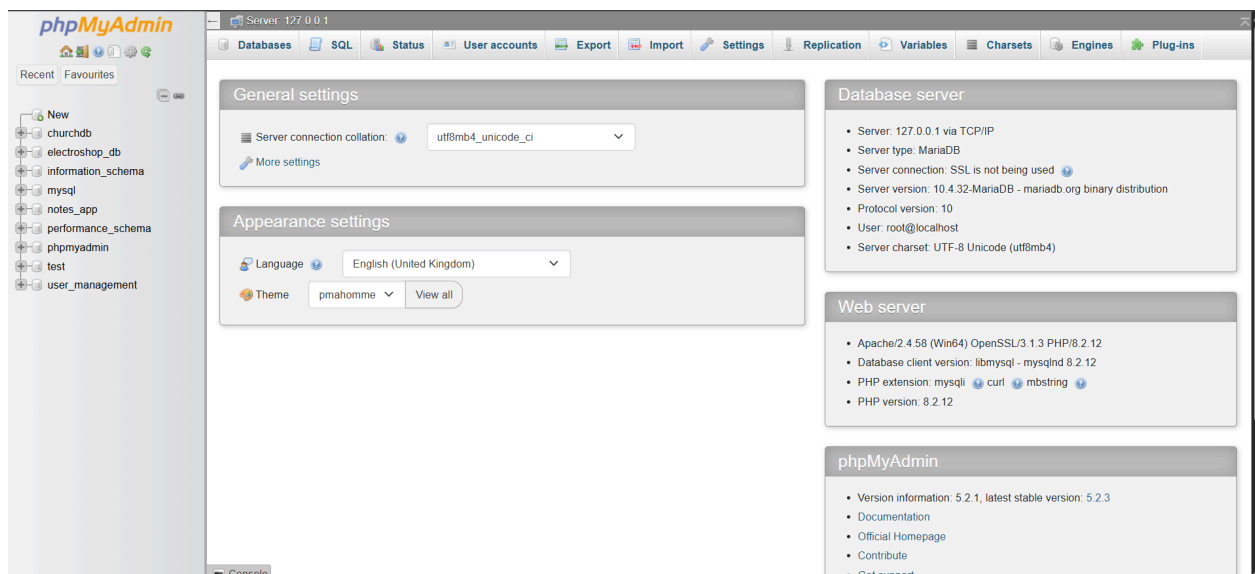


Fig 4: Hello World Test

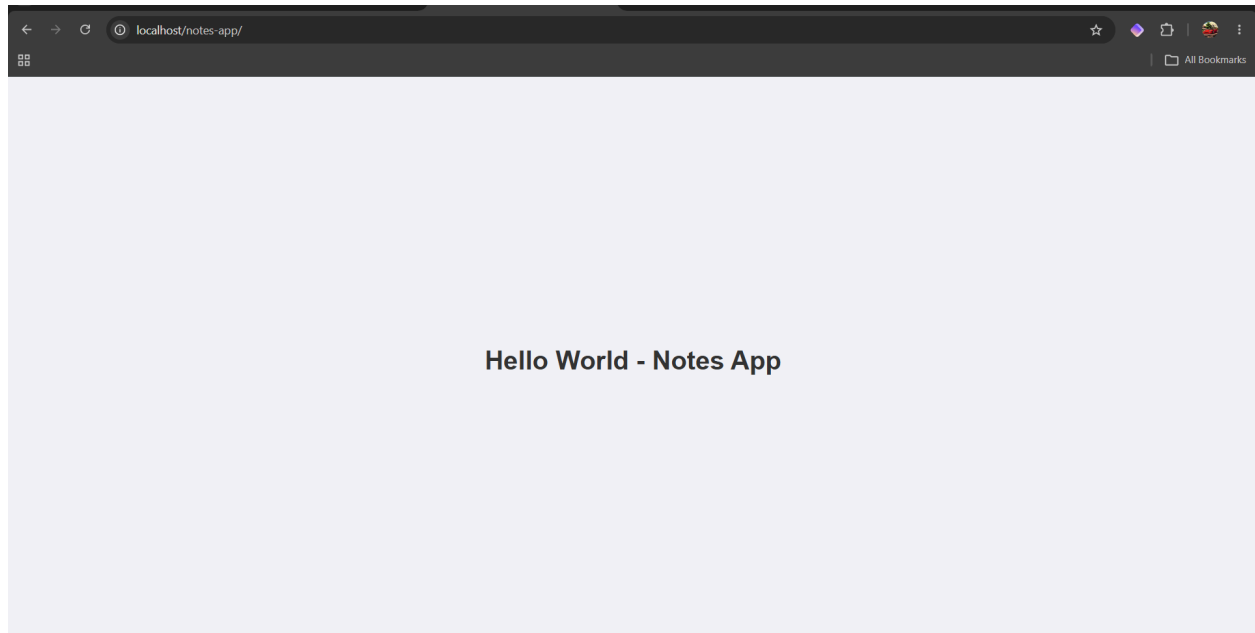
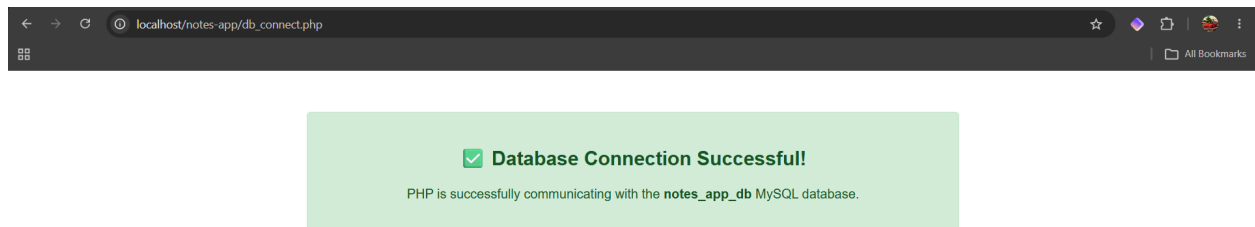


Fig 5: Database Connection Test



Week 2: Wireframes and GUI Design

Fig 1: Figma wireframe detailing the planned layout and user interface for the NoteKeeper login page.

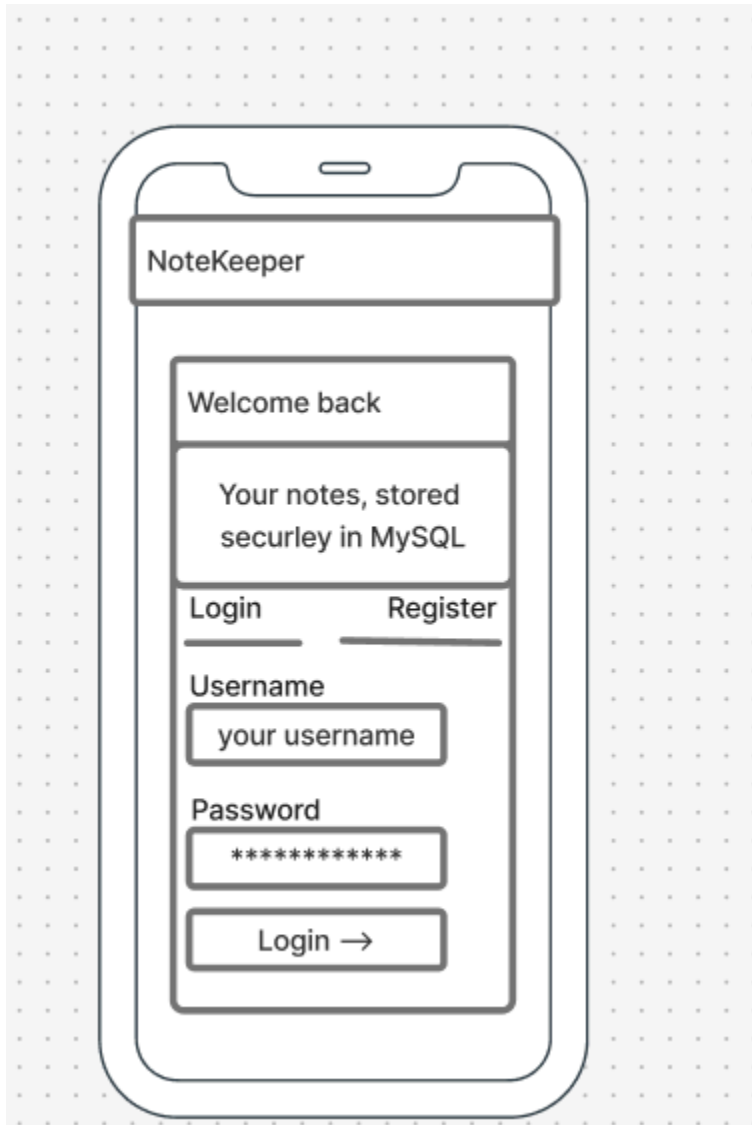


Fig 2: Browser showing the Login page with username and password fields used to authenticate registered users

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

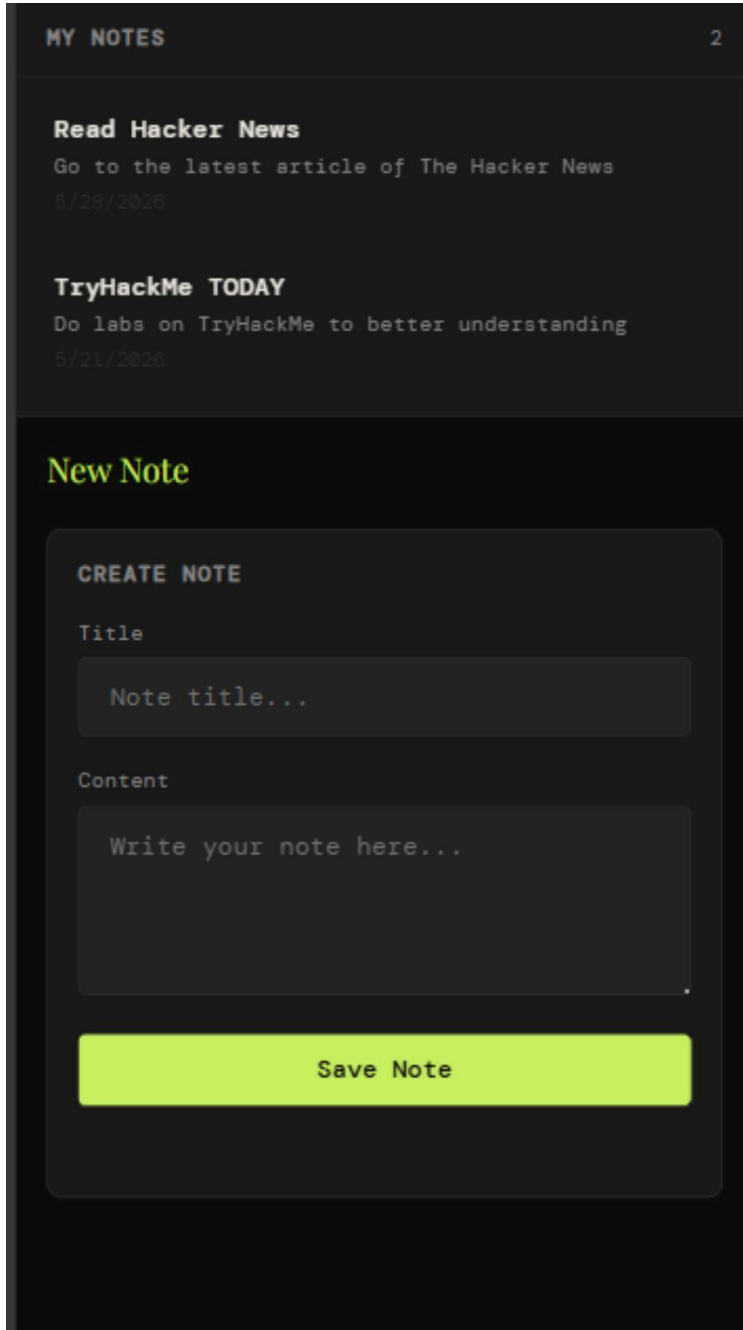
Login Register

Username
your username

Password
.....

Login →

Fig 3: Mobile View Mockup



The mockup shows a mobile application interface with a dark theme. At the top, a header bar contains the text "MY NOTES" on the left and the number "2" on the right. Below the header, there are two note cards. The first card has the title "Read Hacker News" and the text "Go to the latest article of The Hacker News" followed by the date "5/28/2026". The second card has the title "TryHackMe TODAY" and the text "Do labs on TryHackMe to better understanding" followed by the date "5/21/2026". Below these cards, there is a section titled "New Note" in a light green font. Under this section is a form titled "CREATE NOTE". The form has two input fields: "Title" with the placeholder text "Note title..." and "Content" with the placeholder text "Write your note here...". At the bottom of the form is a large green button labeled "Save Note".

MY NOTES 2

Read Hacker News
Go to the latest article of The Hacker News
5/28/2026

TryHackMe TODAY
Do labs on TryHackMe to better understanding
5/21/2026

New Note

CREATE NOTE

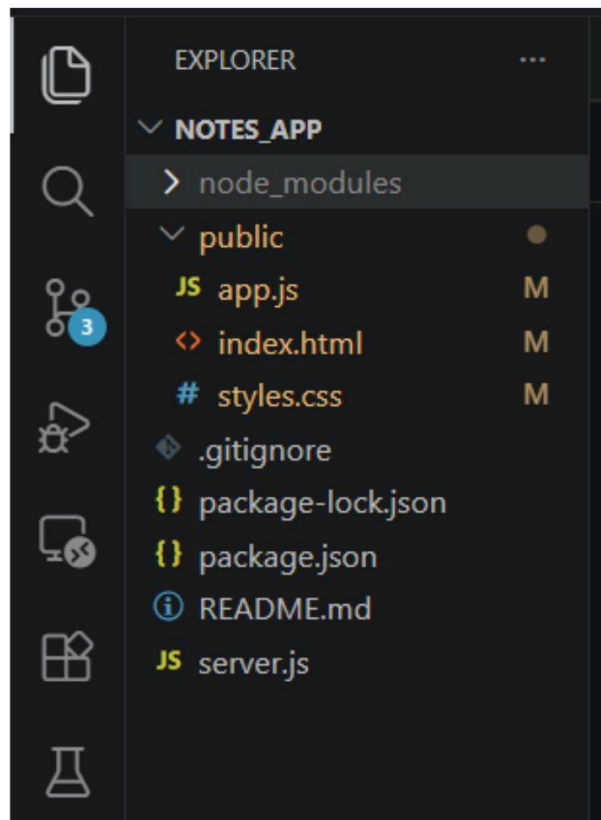
Title
Note title...

Content
Write your note here...

Save Note

Fig 4: Color Theme and Navigation Structure

	Background
	Surface
	Accent
	Text
	Muted
	Error



Week 3: JavaScript and PHP Basics

Fig1: Javascript Form Validation

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

Email

Password

Create Account →

Please fill in all fields

Fig 2: Password Strength Checker

The image shows a dark-themed web interface for a login and registration system. At the top, it says "Welcome back." followed by "Your notes, stored securely in MySQL." Below this are two tabs: "Login" and "Register", with "Register" being the active tab. The form contains three input fields: "Username" with the value "linet", "Email" with the value "linetkamathi@gmail.com", and "Password" which is masked with dots. Below the password field is a strength indicator consisting of a horizontal bar that is approximately one-third filled with orange, and the word "Medium" below it. At the bottom is a large orange button labeled "Create Account →".

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

linet

Email

linetkamathi@gmail.com

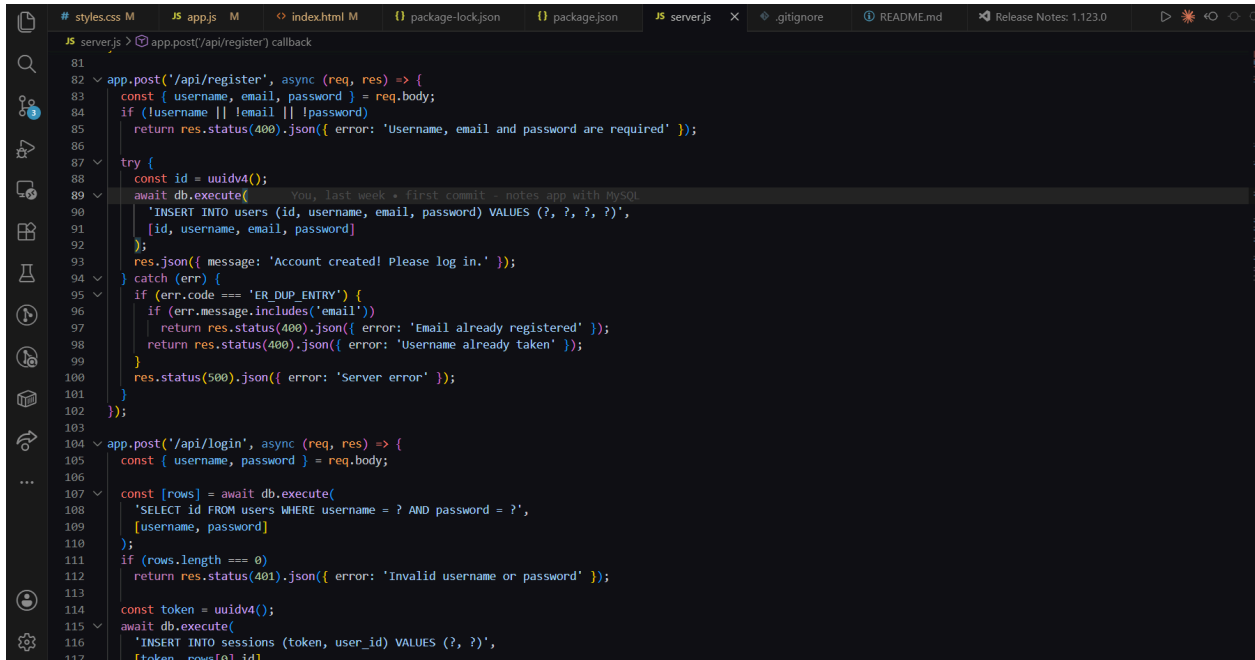
Password

.....

Medium

Create Account →

Fig 3: Shows the Express.js route syntax used in server.js to handle HTTP requests



The image shows a code editor with a dark theme. The top of the editor has a tab bar with several files: # styles.css M, JS app.js M, index.html M, package-lock.json, package.json, JS server.js (active), .gitignore, README.md, and Release Notes: 1.123.0. The main editor area displays the content of server.js. The code defines two Express.js routes using the `app.post` method. The first route is for `/api/register`, which handles user registration. It checks if the username, email, and password are provided. If not, it returns a 400 status with an error message. If provided, it generates a unique ID, inserts the user into the database, and returns a success message. It also handles errors like duplicate entries or server errors. The second route is for `/api/login`, which handles user login. It checks if the username and password are provided, queries the database for the user, and returns an error if the credentials are invalid. It also generates a token and inserts a session into the database. The code is as follows:

```
81 server.js > app.post('/api/register') callback
82
83 app.post('/api/register', async (req, res) => {
84   const { username, email, password } = req.body;
85   if (!username || !email || !password)
86     return res.status(400).json({ error: 'Username, email and password are required' });
87
88   try {
89     const id = uuidv4();
90     await db.execute(
91       'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
92       [id, username, email, password]
93     );
94     res.json({ message: 'Account created! Please log in.' });
95   } catch (err) {
96     if (err.code === 'ER_DUP_ENTRY') {
97       if (err.message.includes('email'))
98         return res.status(400).json({ error: 'Email already registered' });
99       return res.status(400).json({ error: 'Username already taken' });
100     }
101     res.status(500).json({ error: 'Server error' });
102   }
103 });
104
105 app.post('/api/login', async (req, res) => {
106   const { username, password } = req.body;
107
108   const [rows] = await db.execute(
109     'SELECT id FROM users WHERE username = ? AND password = ?',
110     [username, password]
111   );
112   if (rows.length === 0)
113     return res.status(401).json({ error: 'Invalid username or password' });
114
115   const token = uuidv4();
116   await db.execute(
117     'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
118     [token, rows[0].id]
```

Fig 4 shows the database connection configuration and initDB function in server.js.

```
const DB_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '',
  database: 'notes_app',
  multipleStatements: true
};

let db;

async function initDB() {
  const tempConn = await mysql.createConnection({
    host: DB_CONFIG.host,
    port: DB_CONFIG.port,
    user: DB_CONFIG.user,
    password: DB_CONFIG.password
  });

  await tempConn.execute(`CREATE DATABASE IF NOT EXISTS \`${DB_CONFIG.database}\``);
  await tempConn.end();

  db = await mysql.createPool(DB_CONFIG);
}
```

Fig 5: Dynamic User Input Handling

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds)

SELECT * FROM `notes`

Profiling

[Edit inline]

[Edit]

[Explain SQL]

[Create PHP code]

[Refresh]

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

		id	user_id	title	content	created_at
☐	Edit Copy Delete	14f49b21-31f0-4204-aafd-8f7d2e02a11a	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Machine Learning	How Kenya has integrated Machine Learning	2026-06-03 12:37:30
☐	Edit Copy Delete	9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐	Edit Copy Delete	9f938a0c-f96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14

Week 4 Task Schedule

Task 1 - Understanding Server-Side Programming

Fig 1: Simple Processing Page

```
app.post('/api/register', async (req, res) => {
  const { username, email, password } = req.body;
  if (!username || !email || !password)
    return res.status(400).json({ error: 'Username, email and password are required' });

  try {
    const id = uuidv4();
    await db.execute(
      'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
      [id, username, email, password]
    );
    res.json({ message: 'Account created! Please log in.' });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      if (err.message.includes('email'))
        return res.status(400).json({ error: 'Email already registered' });
      return res.status(400).json({ error: 'Username already taken' });
    }
    res.status(500).json({ error: 'Server error' });
  }
});
```

Fig 2: Dynamic Welcome Page

The screenshot shows the NoteKeeper web application. At the top, the title 'NoteKeeper' is on the left, and 'logged in as chendler' with a 'logout' button is on the right. A sidebar on the left titled 'MY NOTES' shows a count of 3 and lists three items: 'Machine Learning' (dated 07/08/2020), 'Read Hacker News' (dated 07/08/2020), and 'TryHackMe' (dated 07/08/2020). The main content area is titled 'New Note' and contains a 'CREATE NOTE' form. The form has two input fields: 'Title' with placeholder text 'Note title...' and 'Content' with placeholder text 'Write your note here...'. A green 'Save Note' button is at the bottom of the form.

Fig3: Form Submission System

Welcome back.

Your notes, stored securely in MySQL.

[Login](#) [Register](#)

Username

linet

Email

linetkamathi@gmail.com

Password

.....

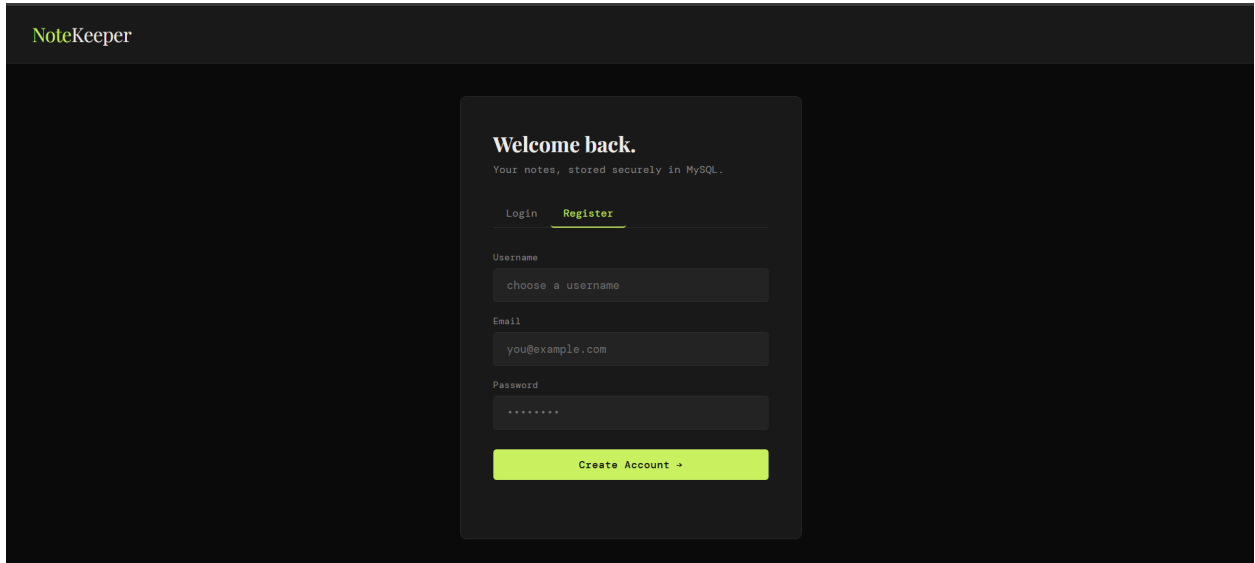
Medium

Create Account →

✓ Account created! Please log in.

Task 2 - HTML Forms and PHP Integration

Fig 1: Registration Form



The screenshot shows the registration form of the NoteKeeper application. The header is dark gray with the 'NoteKeeper' logo in green and white. The main content area is black, featuring a central white card. The card has a heading 'Welcome back.' and a subtext 'Your notes, stored securely in MySQL.'. Below this, there are two tabs: 'Login' and 'Register', with 'Register' being the active tab. The form fields are labeled 'Username', 'Email', and 'Password'. The 'Username' field contains the placeholder text 'choose a username'. The 'Email' field contains the placeholder text 'you@example.com'. The 'Password' field contains a series of dots. At the bottom of the card, there is a green button labeled 'Create Account →'.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

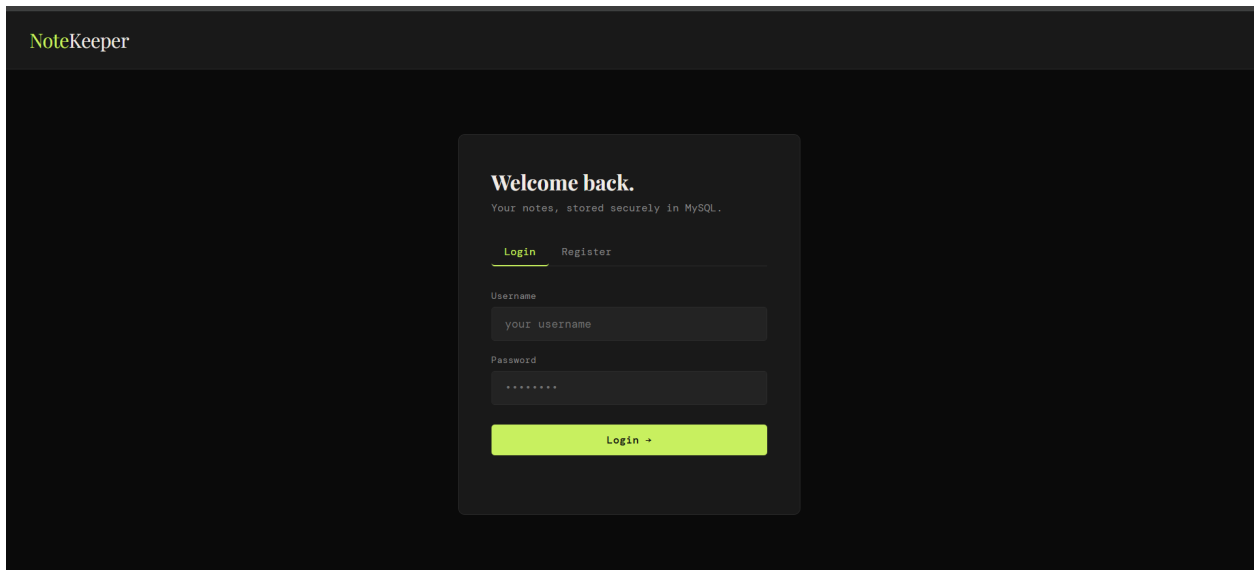
Username
choose a username

Email
you@example.com

Password
.....

Create Account →

Fig 2: Login Form



The screenshot shows the login form of the NoteKeeper application. The header is dark gray with the 'NoteKeeper' logo in green and white. The main content area is black, featuring a central white card. The card has a heading 'Welcome back.' and a subtext 'Your notes, stored securely in MySQL.'. Below this, there are two tabs: 'Login' and 'Register', with 'Login' being the active tab. The form fields are labeled 'Username' and 'Password'. The 'Username' field contains the placeholder text 'your username'. The 'Password' field contains a series of dots. At the bottom of the card, there is a green button labeled 'Login →'.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

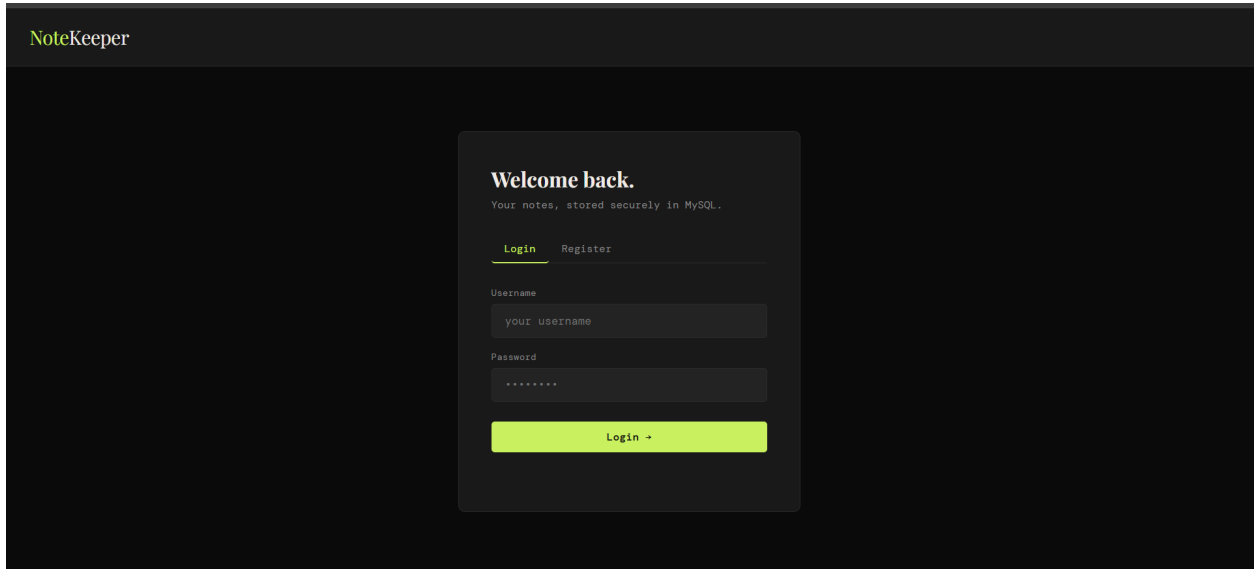
Username
your username

Password
.....

Login →

Task 3 - Simple Authentication

Fig 1: Basic Login Page



The image shows a web application interface for 'NoteKeeper'. The page has a dark theme. At the top left, the 'NoteKeeper' logo is displayed in a light green font. The main content area is a dark gray rectangle. In the center of this area is a white login form. The form contains the following elements: a heading 'Welcome back.' followed by the text 'Your notes, stored securely in MySQL.'; two links, 'Login' (underlined) and 'Register'; a 'Username' label above a text input field containing the placeholder text 'your username'; a 'Password' label above a password input field with masked characters '*****'; and a bright green 'Login →' button at the bottom.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

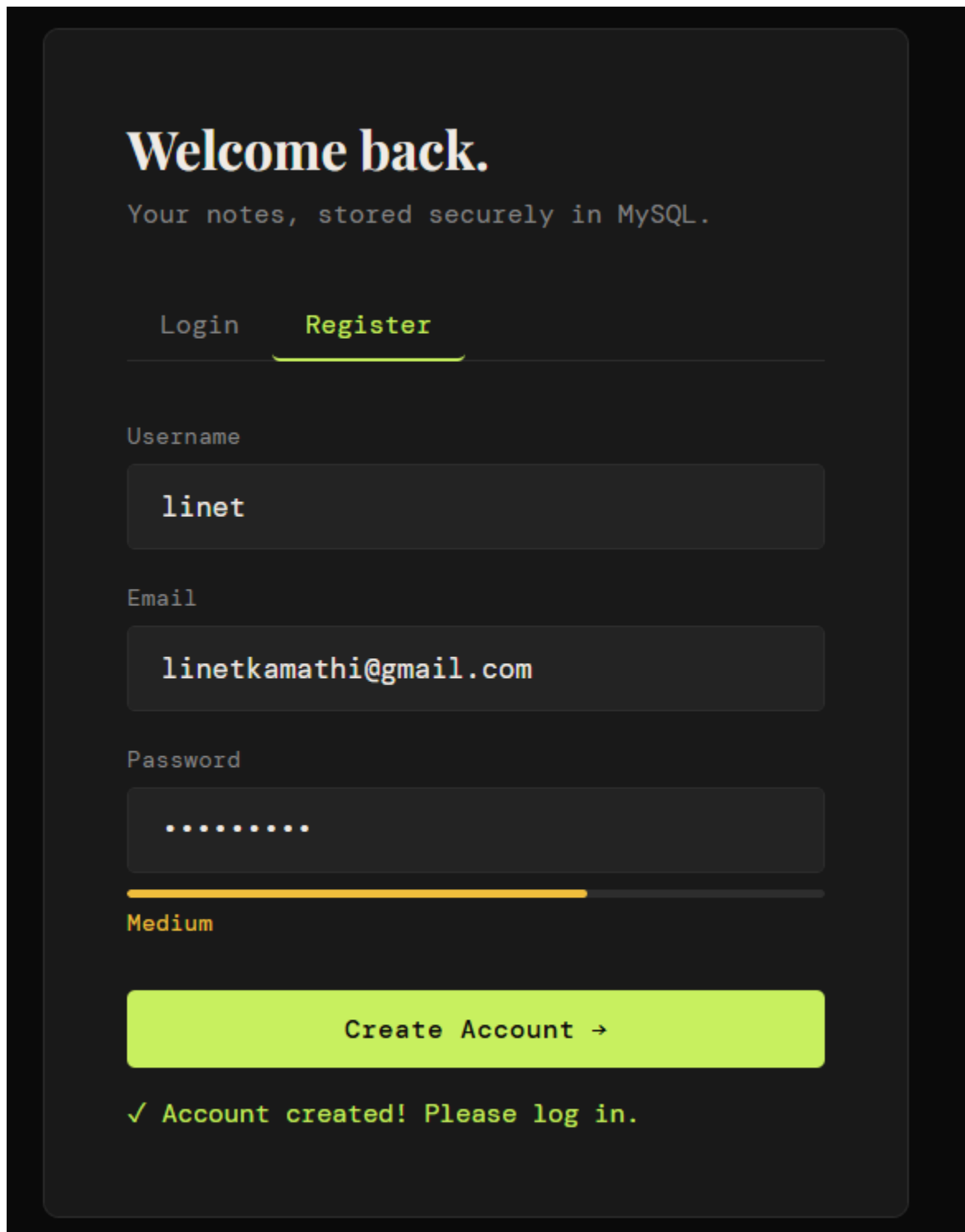
[Login](#) [Register](#)

Username
your username

Password

Login →

Fig 2: Username/passwords validation



Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

linet

Email

linetkamathi@gmail.com

Password

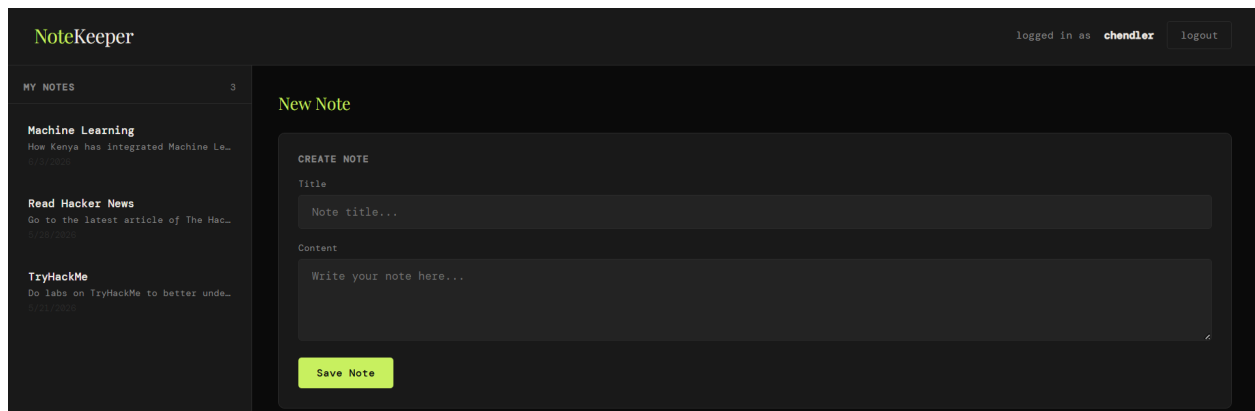
.....

Medium

Create Account →

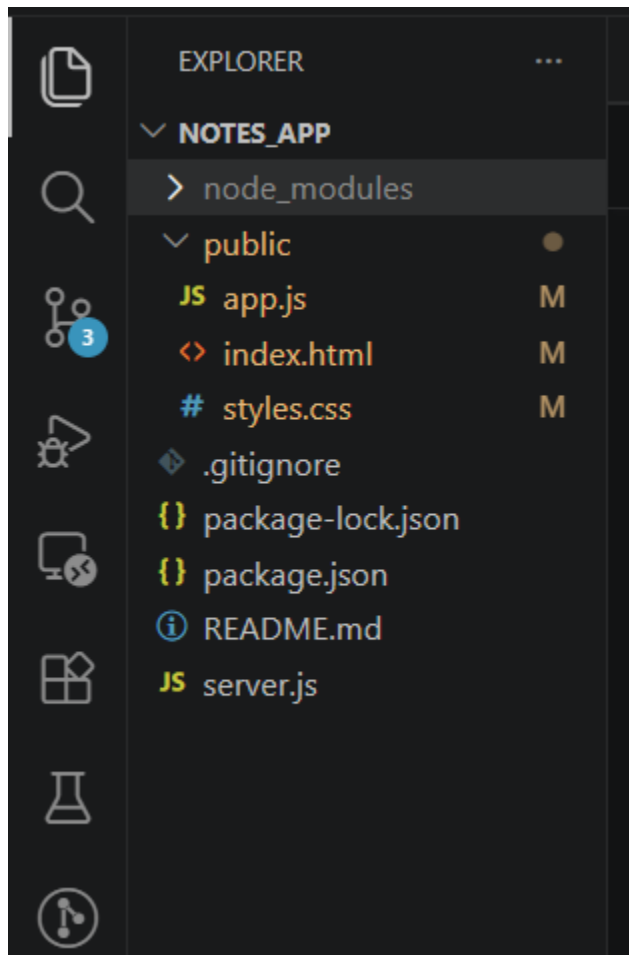
✓ Account created! Please log in.

Fig 3: Session-based welcome page



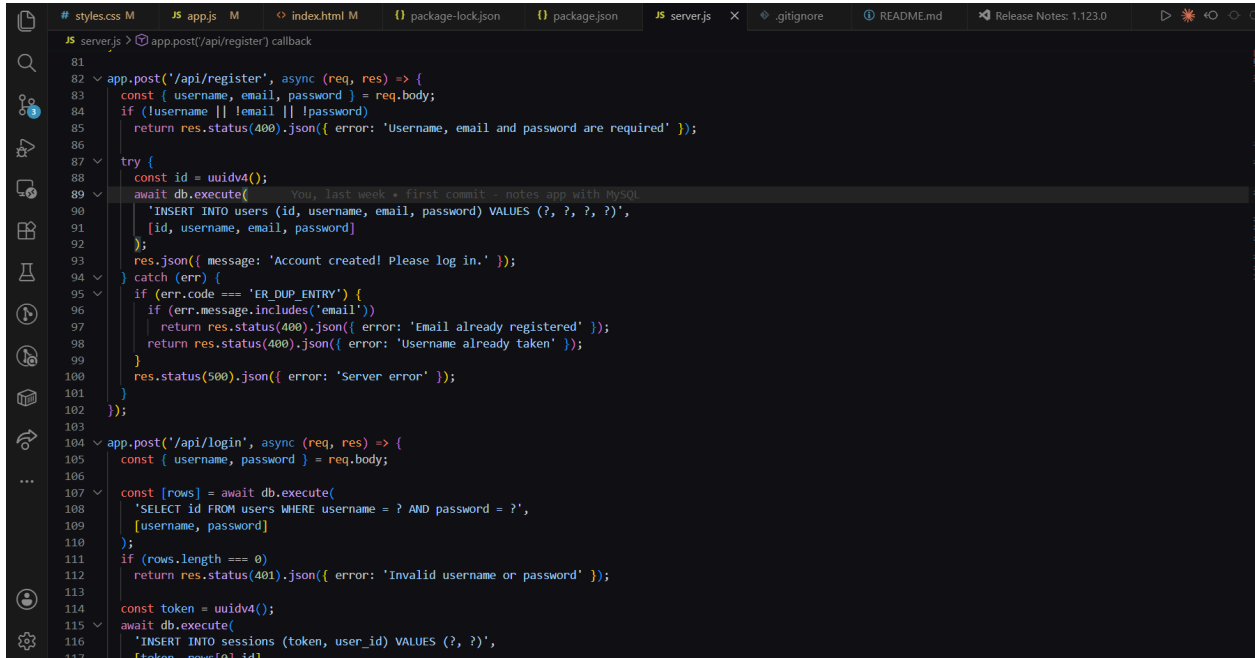
Task 4 - Backend Folder Organization

Fig 1: Project Structure



Task 5 - Introduction to Modern Backend Alternatives

Fig : PHP Modern Alternative ([Node.js](#))



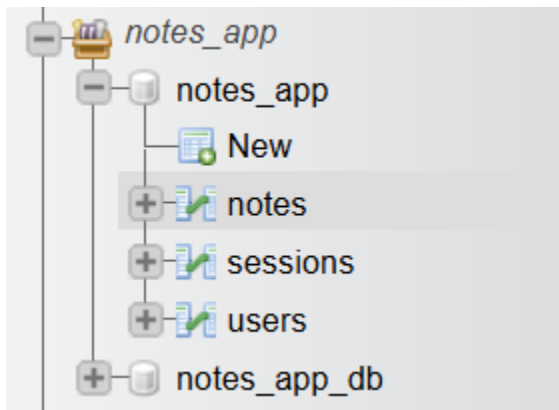
The screenshot shows a code editor with a dark theme. The top bar displays several open files: # styles.css M, JS app.js M, index.html M, package-lock.json, package.json, JS server.js, .gitignore, README.md, and Release Notes: 1.123.0. The main editor area shows the content of server.js, which is a Node.js application using Express.js and MySQL. The code includes a registration endpoint (app.post('/api/register')) and a login endpoint (app.post('/api/login')). The registration endpoint checks for required fields (username, email, password) and handles database insertions with error handling for duplicates and server errors. The login endpoint checks for valid credentials and generates a token for successful logins. The code is written in JavaScript with ES6 syntax, including arrow functions and const/let for variable declarations. The editor has a sidebar on the left with icons for file explorer, search, and other development tools.

```
81
82 ✓ app.post('/api/register', async (req, res) => {
83   const { username, email, password } = req.body;
84   if (!username || !email || !password)
85     return res.status(400).json({ error: 'Username, email and password are required' });
86
87   try {
88     const id = uuidv4();
89     await db.execute(`
90       INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?),
91       [id, username, email, password]
92     `);
93     res.json({ message: 'Account created! Please log in.' });
94   } catch (err) {
95     if (err.code === 'ER_DUP_ENTRY') {
96       if (err.message.includes('email'))
97         return res.status(400).json({ error: 'Email already registered' });
98       return res.status(400).json({ error: 'Username already taken' });
99     }
100    res.status(500).json({ error: 'Server error' });
101  }
102 });
103
104 ✓ app.post('/api/login', async (req, res) => {
105   const { username, password } = req.body;
106
107   const [rows] = await db.execute(
108     `SELECT id FROM users WHERE username = ? AND password = ?`,
109     [username, password]
110   );
111   if (rows.length === 0)
112     return res.status(401).json({ error: 'Invalid username or password' });
113
114   const token = uuidv4();
115   await db.execute(
116     `INSERT INTO sessions (token, user_id) VALUES (?, ?),
117     [token, rows[0].id]`
118   );
119   res.json({ token });
120 }
```

Week 5 - Advanced Web Design and Development

Task 1: Database Creation

Fig 1: notes-app database



Task 2: Creating Tables

Fig 1: notes-app tables

Table	Action	Rows	Type	Collation	Size	Overhead
☐ notes	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ sessions	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	-
3 tables	Sum	8	InnoDB	utf8mb4_general_ci	112.0 KiB	0 B

Task 3: CRUD Operations

Fig 1: Add Records (INSERT)

+ -			id	user_id	title	content	created_at	
☐			Delete	14f49b21-31f0-4204-aafd-8f7d2e02a11a	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Machine Learning	How Kenya has integrated Machine Learning	2026-06-03 12:37:30
☐			Delete	9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐			Delete	9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14
☐ Check all With selected: Edit Copy Delete Export								

Fig 2: View Records (SELECT)

NoteKeeper

MY NOTES3

Machine Learning

How Kenya has integrated Machine Le...

Read Hacker News

Go to the latest article of The Hac...

5/28/2026

TryHackMe

Do labs on TryHackMe to better unde...

5/21/2026

New Note

CREATE NOTE

Title

Note title...

Content

Write your note here...

Save Note

Fig 3: Delete Records (DELETE)

				id	user_id	title	content	created_at
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14

Fig 4: Edit Records (UPDATE)

				id	user_id	title	content	created_at
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe TODAY	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14
☐ Check all With selected: Edit Copy Delete Export								

Task 4: Connecting PHP to Database

Fig 1: Database Connection

```
const DB_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '',
  database: 'notes_app',
  multipleStatements: true
};

let db;

async function initDB() {
  const tempConn = await mysql.createConnection({
    host: DB_CONFIG.host,
    port: DB_CONFIG.port,
    user: DB_CONFIG.user,
    password: DB_CONFIG.password
  });

  await tempConn.execute(`CREATE DATABASE IF NOT EXISTS \`${DB_CONFIG.database}\``);
  await tempConn.end();

  db = await mysql.createPool(DB_CONFIG);
}
```

Task 5: Fetching Database Records

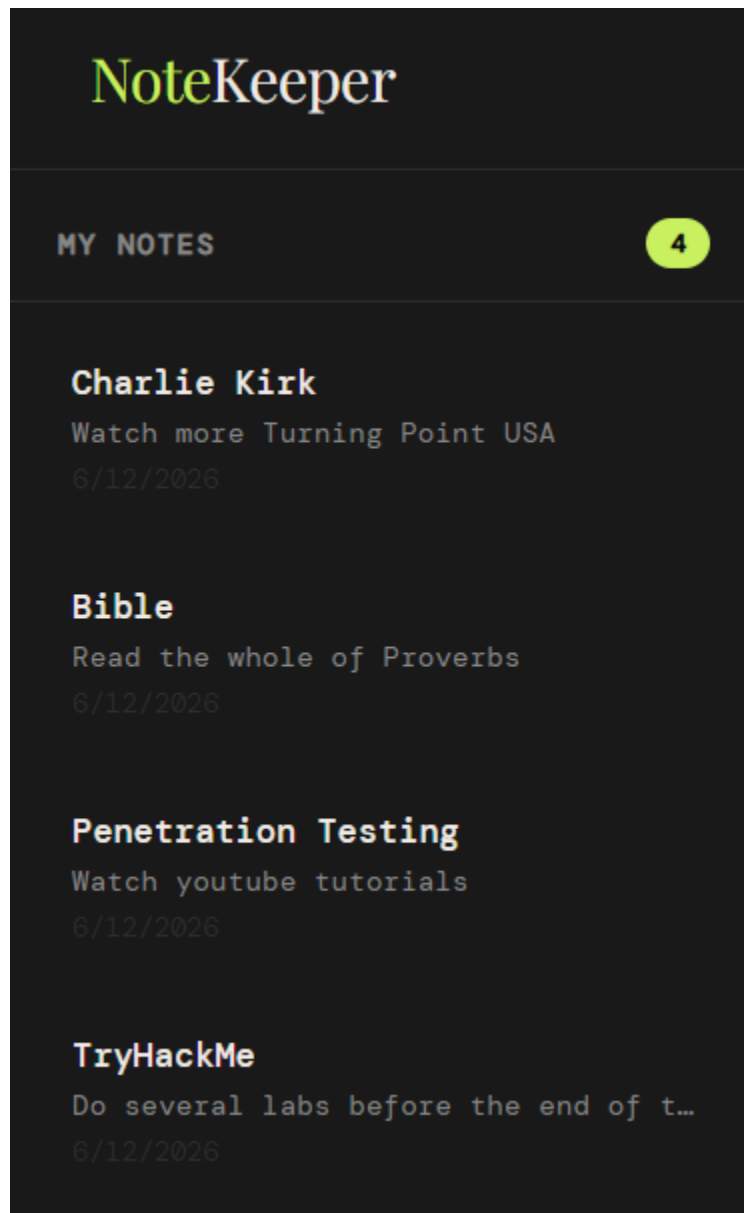
Fig 1: Database Records

```
app.put('/api/notes/:id', requireAuth, async (req, res) => {
  const { title, content } = req.body;
  if (!title || !content)
    return res.status(400).json({ error: 'Title and content required' });

  await db.execute(
    'UPDATE notes SET title = ?, content = ? WHERE id = ? AND user_id = ?',
    [title, content, req.params.id, req.userId]
  );
  res.json({ message: 'Note updated!' });
});
```

Creativity: Note Counter

Fig 1: shows the dynamic note count badge in the sidebar header displaying the total number of notes.



Week 6: Database Integration and CRUD Operations

Practical Task 1

Fig 1: Add a Note (INSERT)

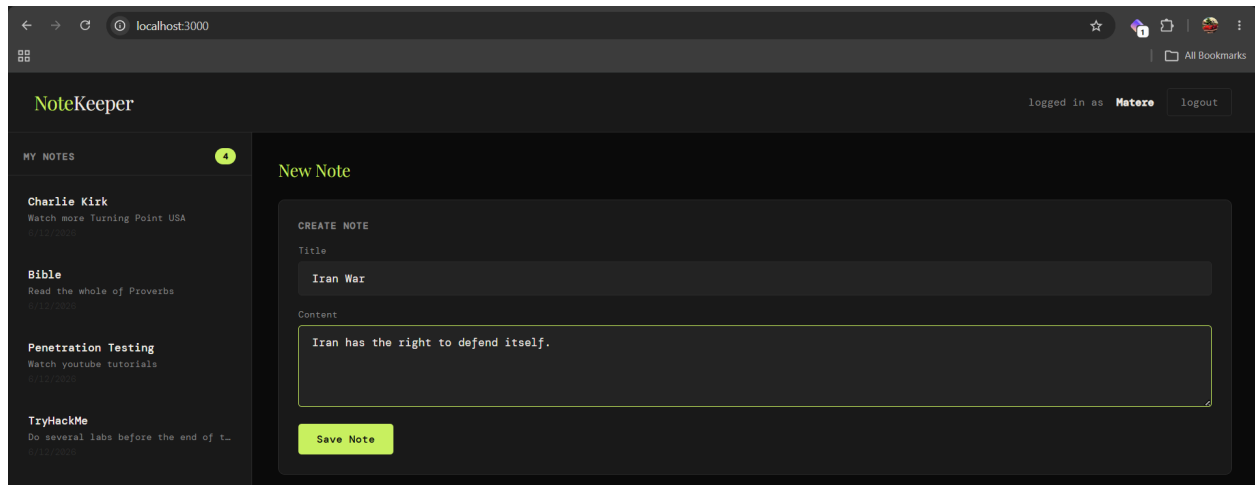


Fig 2: View Notes (SELECT)

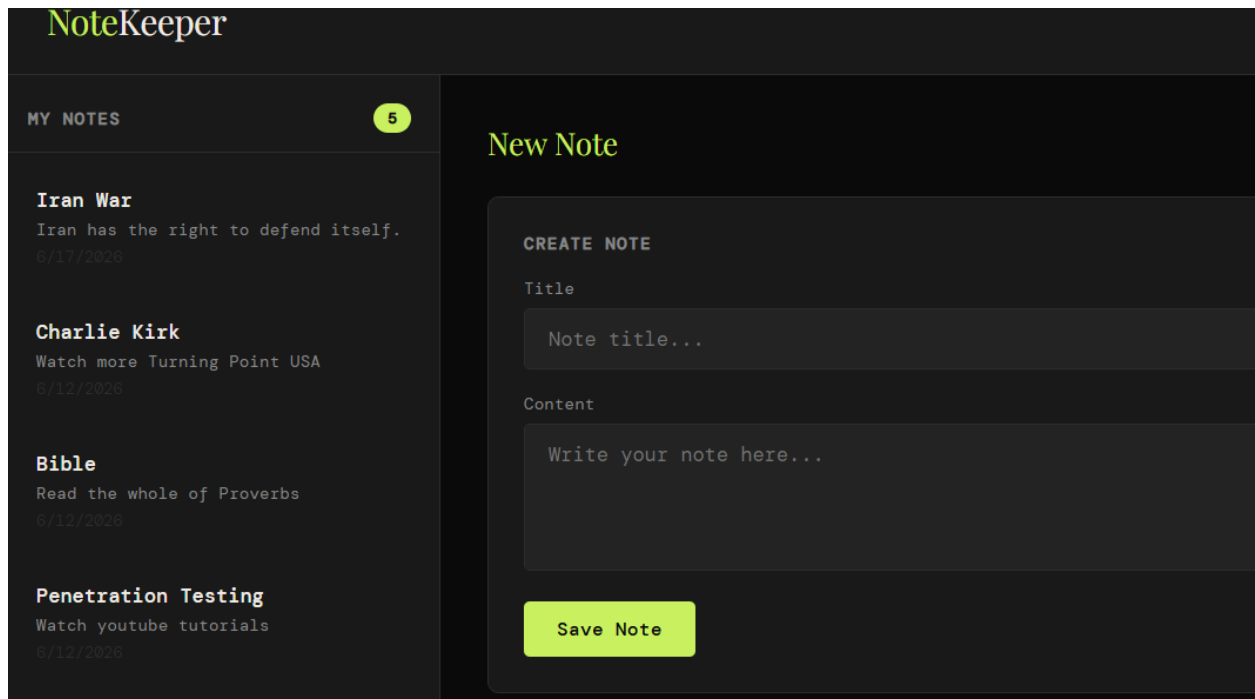


Fig 3: Edit a Note (UPDATE)

New Note

CREATE NOTE

Title

Pen Tester

Content

Watch youtube tutorials

Update Note

Fig 4: Delete a Note (DELETE)

		id	user_id	title	content	created_at
☐	 Edit  Copy  Delete	44de630b-b69d-4329-9c6f-ad2a23efb42e	954c422b-6eb7-486f-92ed-99341205d3f2	Charlie Kirk	Watch more Turning Point USA	2026-06-12 04:53:18
☐	 Edit  Copy  Delete	6daa1bf7-84f3-41fa-af38-648203afca9f	954c422b-6eb7-486f-92ed-99341205d3f2	TryHackMe	Do several labs before the end of the week.	2026-06-12 04:50:01
☐	 Edit  Copy  Delete	7fe950ea-1a33-4bae-9cce-cf6e33656dca	954c422b-6eb7-486f-92ed-99341205d3f2	Iran War	Iran has the right to defend itself.	2026-06-17 07:56:39
☐	 Edit  Copy  Delete	bd2baec9-358b-467f-88dd-bd13f521a874	954c422b-6eb7-486f-92ed-99341205d3f2	Bible	Read the whole of Proverbs	2026-06-12 04:52:54

Fig 5: Database Export

```
> week1
> week2
> week3
> week4
v week5
  JS server.js
  .gitignore
  Notes Application Docum...
  notes_app.sql
  {} package-lock.json
71
72
73
74
75
76
77
78
79
80
81
82
<div class= sidebar
  <div class="sideba
    <h3>My Notes</h3>
    <span id="noteCo
  </div>
  <div class="notes-
    <div class="empt
  </div>
</div>
<div class="main-par
  <div class="panel-
```

Practical Task 2

Fig 1: Note with Category Added

New Note

CREATE NOTE

Title

Cipher Text

Category

Study

Content

Why is it called Caesar Cipher.

Save Note

Fig 2: Records Stored with Category

				id	user_id	title	content	category	created_at
☐	Edit	Copy	Delete	065373d4-ef12-453c-9a9a-43778fa81694	954c422b-6eb7-486f-92ed-99341205d3f2	Bible Study	Join the Bible study today at 8pm.	Personal	2026-06-17 08:29:58
☐	Edit	Copy	Delete	38e99915-e346-424c-857e-4fac777a9a82	954c422b-6eb7-486f-92ed-99341205d3f2	Drink More Water	Water helps with everyday tasks	General	2026-06-17 08:28:51
☐	Edit	Copy	Delete	4e52ad21-953c-494e-8829-fae1206837c6	954c422b-6eb7-486f-92ed-99341205d3f2	Tech Companies	Research about several tech companies in Nairobi.	Work	2026-06-17 08:29:36
☐	Edit	Copy	Delete	820f7cd8-5ae5-4865-83ec-692f802c00f3	954c422b-6eb7-486f-92ed-99341205d3f2	Penetration Testing	Watch tutorials on how to Pen Test.	Study	2026-06-17 08:30:30
	☐ Check all	With selected:	Edit	Copy	Delete	Export			

Fig 3: View Records

MY NOTES

4

Penetration Testing

Watch tutorials on how to Pen Test.

07/17/2026

Bible Study

Join the Bible study today at 8pm.

07/17/2026

Tech Companies

Research about several tech compani...

07/17/2026

Drink More Water

Water helps with everyday tasks.

07/17/2026

New Note

CREATE NOTE

Title

Note title...

Category

Study

Content

Write your note here...

Save Note

Fig 4: Edit a Record

New Note

CREATE NOTE

Title

Tech Companies & Software

Category

Study

Content

Research about several tech companies in Nairobi.

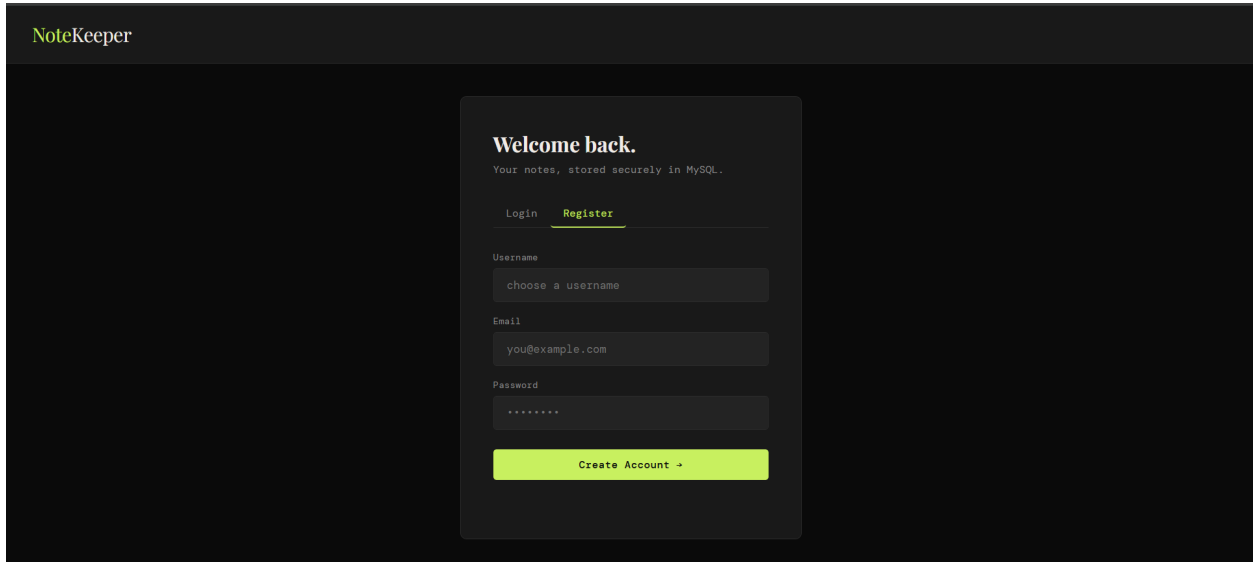
Update Note

Fig 5: Delete a Record

		id	user_id	title	content	category	created_at
	Edit Copy Delete	065373d4-ef12-453c-9a9a-43778fa81694	954c422b-6eb7-486f-92ed-99341205d3f2	Bible Study	Join the Bible study today at 8pm.	Personal	2026-06-17 08:29:58
	Edit Copy Delete	4e52ad21-953c-494e-8829-fae1206837c6	954c422b-6eb7-486f-92ed-99341205d3f2	Tech Companies & Software	Research about several tech companies in Nairobi.	Work	2026-06-17 08:29:36
	Edit Copy Delete	820f7cd8-5ae5-4865-83ec-692f802c00f3	954c422b-6eb7-486f-92ed-99341205d3f2	Penetration Testing	Watch tutorials on how to Pen Test.	Study	2026-06-17 08:30:30
	Check all	With selected: Edit Copy Delete Export					

Practical Task 3 (Challenge Task)

Fig 1: User Registration



The image shows a user registration form for a web application. The form is centered on a dark background. At the top left, the text "NoteKeeper" is displayed in a light green font. The form itself is a light gray box with a white border. Inside the box, the text "Welcome back." is followed by "Your notes, stored securely in MySQL.". Below this, there are two tabs: "Login" and "Register", with "Register" being the active tab. The form contains three input fields: "Username" with the placeholder text "choose a username", "Email" with the placeholder text "you@example.com", and "Password" with the placeholder text "*****". At the bottom of the form is a green button with the text "Create Account →".

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

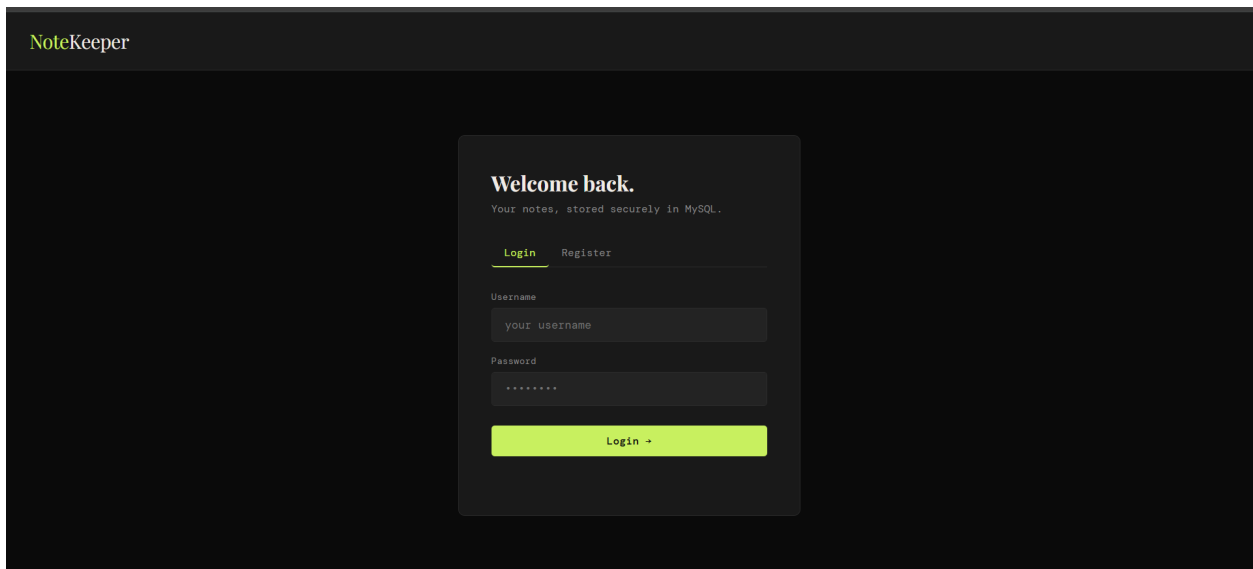
Username
choose a username

Email
you@example.com

Password

Create Account →

Fig 2: User Login



The image shows a user login form for a web application. The form is centered on a dark background. At the top left, the text "NoteKeeper" is displayed in a light green font. The form itself is a light gray box with a white border. Inside the box, the text "Welcome back." is followed by "Your notes, stored securely in MySQL.". Below this, there are two tabs: "Login" and "Register", with "Login" being the active tab. The form contains two input fields: "Username" with the placeholder text "your username" and "Password" with the placeholder text "*****". At the bottom of the form is a green button with the text "Login →".

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

Username
your username

Password

Login →

Fig 3: View Records

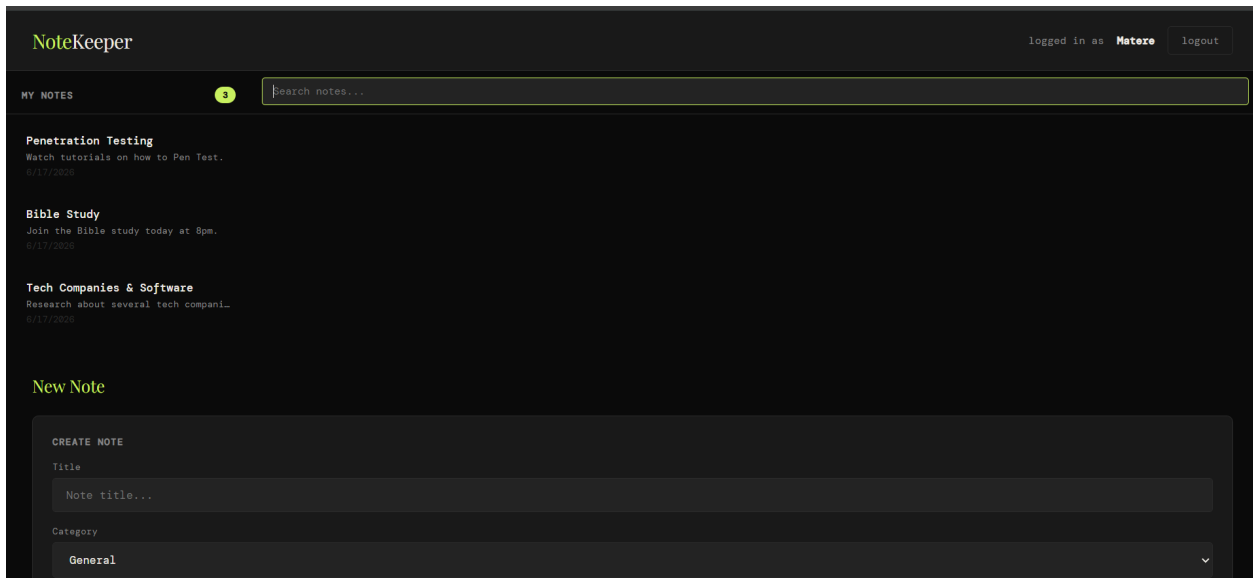


Fig 4: Search Functionality

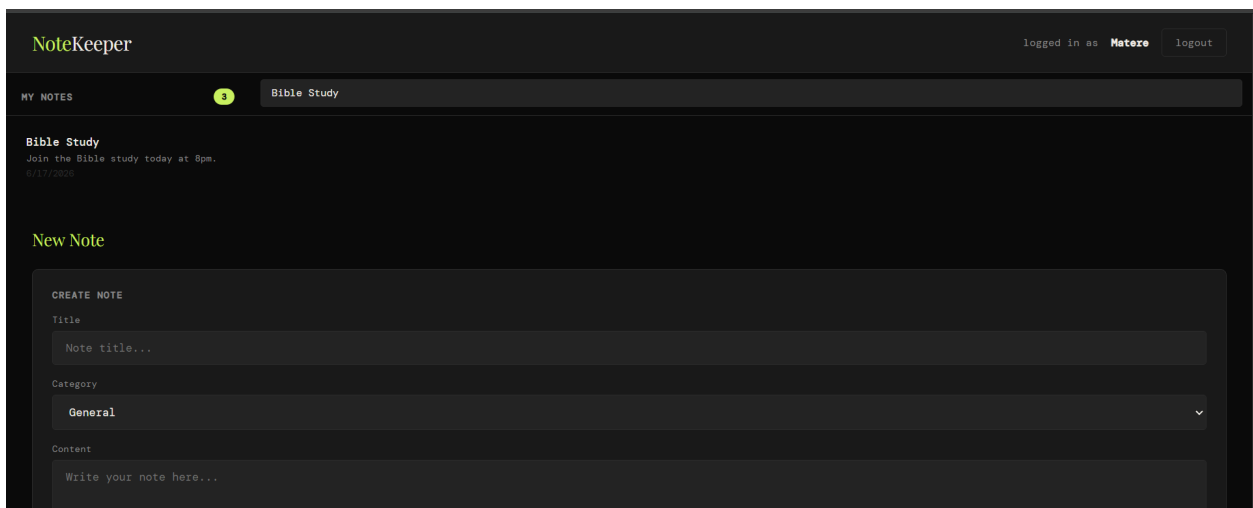


Fig 5: Edit Record

New Note

CREATE NOTE

Title

Tech Companies & Software

Category

Study

Content

Research about several tech companies in Nairobi.

Update Note

Fig 6: Delete Record

		id	user_id	title	content	category	created_at
☐	Edit	065373d4-ef12-453c-9a9a-43778fa81694	954c422b-6eb7-486f-92ed-99341205d3f2	Bible Study	Join the Bible study today at 8pm.	Personal	2026-06-17 08:29:58
☐	Edit	4e52ad21-953c-494e-8829-fae1206837c6	954c422b-6eb7-486f-92ed-99341205d3f2	Tech Companies & Software	Research about several tech companies in Nairobi.	Work	2026-06-17 08:29:36
☐	Edit	82077cd8-5ae5-4865-83ec-692f802c00f3	954c422b-6eb7-486f-92ed-99341205d3f2	Penetration Testing	Watch tutorials on how to Pen Test.	Study	2026-06-17 08:30:30

☐ Check all With selected: Edit Copy Delete Export

Fig 7: Responsive Design

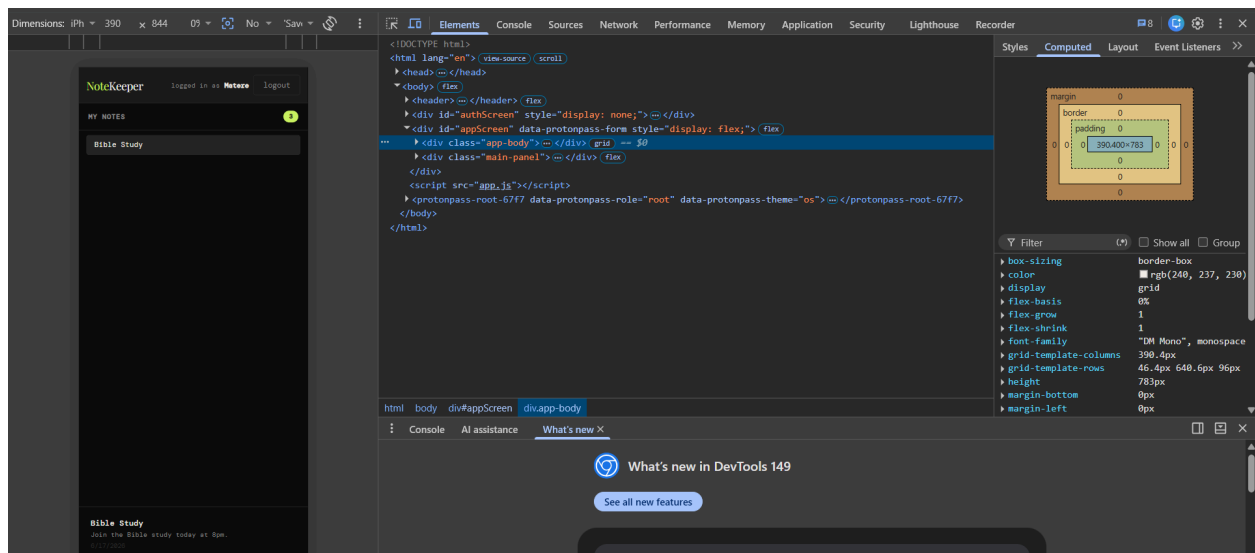


Fig 8: Form Validation

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

choose a username

Email

you@example.com

Password

.....

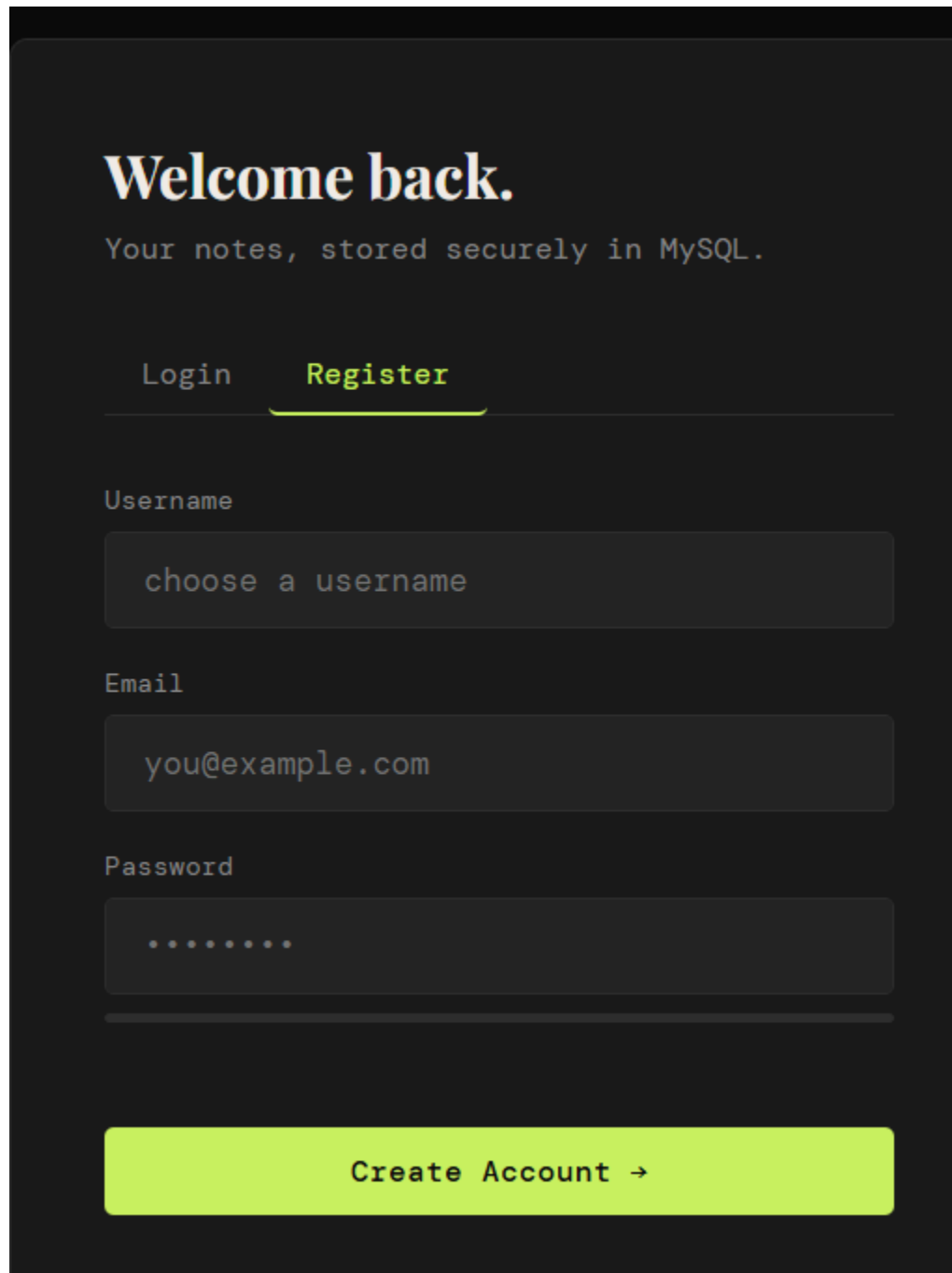
Create Account →

Please fill in all fields

Week 7: User Authentication and Session Management

Practical Task 1: Student Portal Login System

Fig 1: Registration

The image shows a registration form on a dark background. At the top, it says "Welcome back." in a large, bold, white font. Below this, in a smaller, lighter font, it says "Your notes, stored securely in MySQL." There are two tabs: "Login" and "Register". The "Register" tab is selected and highlighted with a green underline. Below the tabs, there are three input fields: "Username" with the placeholder text "choose a username", "Email" with the placeholder text "you@example.com", and "Password" with a series of dots. At the bottom, there is a large green button with the text "Create Account →".

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

choose a username

Email

you@example.com

Password

.....

Create Account →

Fig 2: Login

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

your username

Password

.....

Login →

Fig 3: Dashboard

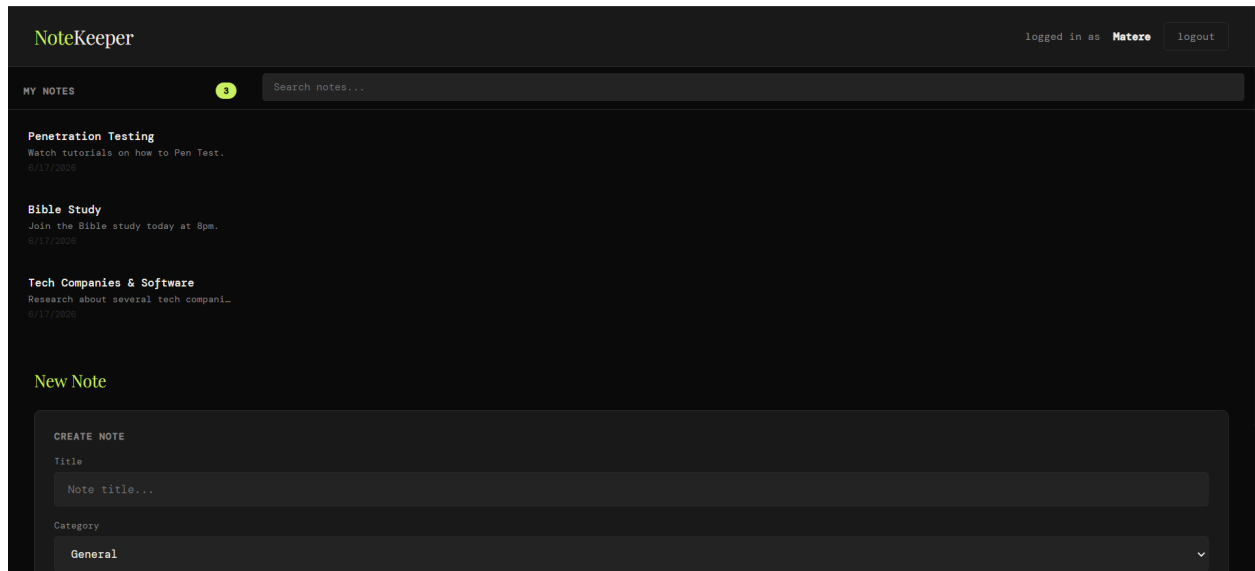


Fig 4: Logout

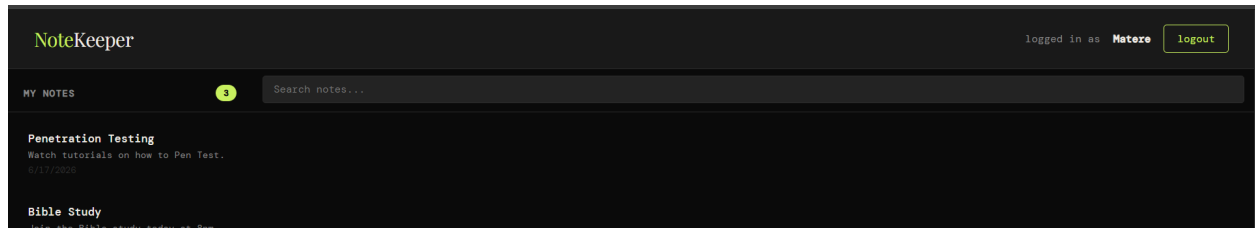
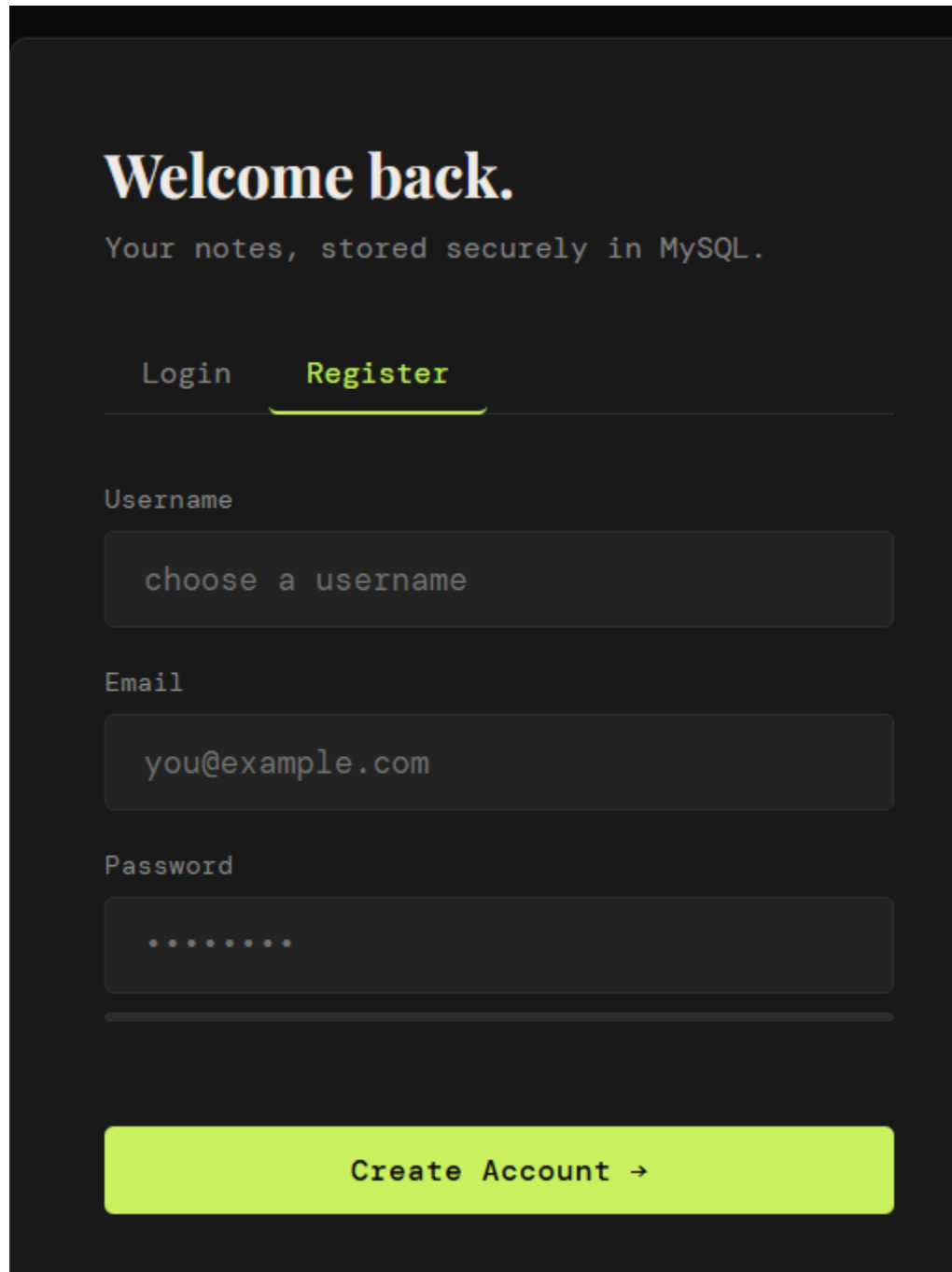


Fig 5: Database Script

> week1	71	<div class= sidebar
> week2	72	<div class="sideba
> week3	73	<h3>My Notes</h3
> week4	74	<span id="noteCo
> week5	75	</div>
JS server.js	76	<div class="notes-
.gitignore	77	<div class="empt
Notes Application Docum...	78	</div>
notes_app.sql U	79	</div>
package-lock.json	80	
	81	<div class="main-par
	82	<div class="panel-

Practical Task 2: Employee Portal

Fig 1: Registration



The image shows a registration form for an Employee Portal. The form is set against a dark background. At the top, it says "Welcome back." in a large, bold, white font, followed by "Your notes, stored securely in MySQL." in a smaller, white font. Below this, there are two tabs: "Login" and "Register". The "Register" tab is highlighted with a yellow underline. Under the "Register" tab, there are three input fields: "Username" with the placeholder text "choose a username", "Email" with the placeholder text "you@example.com", and "Password" with a series of dots. Below the "Password" field, there is a horizontal line. At the bottom of the form, there is a large yellow button with the text "Create Account →".

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

choose a username

Email

you@example.com

Password

.....

Create Account →

Fig 2: Authentication

```
app.post('/api/login', async (req, res) => {
  const { username, password } = req.body;

  const [rows] = await db.execute(
    'SELECT id FROM users WHERE username = ? AND password = ?',
    [username, password]
  );
  if (rows.length === 0)
    return res.status(401).json({ error: 'Invalid username or password' });

  const token = uuidv4();
  await db.execute(
    'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
    [token, rows[0].id]
  );

  res.json({ token, username });
});
```

Fig 3: Protected Page

```
124 app.post('/api/logout', requireAuth, async (req, res) => {
125   const token = req.headers['authorization'];
126   await db.execute('DELETE FROM sessions WHERE token = ?', [token]);
127   res.json({ message: 'Logged out' });
128 });
129
130 app.get('/api/notes', requireAuth, async (req, res) => {
131   const [rows] = await db.execute(
132     'SELECT * FROM notes WHERE user_id = ? ORDER BY created_at DESC',
133     [req.userId]
134   );
135   res.json(rows);
136 });
137
138 app.post('/api/notes', requireAuth, async (req, res) => {
139   const { title, content, category } = req.body;
140   if (!title || !content)
141     return res.status(400).json({ error: 'Title and content required' });
142
143   const id = uuidv4();
144   await db.execute(
145     'INSERT INTO notes (id, user_id, title, content, category) VALUES (?, ?, ?, ?, ?)',
146     [id, req.userId, title, content, category || 'General']
147   );
148   res.json({ id, title, content, category });
149 });
```

You, 22 hours ago • week6 - add category

Fig 4: Session Management

		token	user_id	created_at
☐	 Edit  Copy  Delete	7d90e248-9ecf-4b4f-93c3-9392f1929fed	954c422b-6eb7-486f-92ed-99341205d3f2	2026-06-18 07:29:33
 ☐ Check all With selected:  Edit  Copy  Delete  Export				

Fig 5: Session Deleted on Logout

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

SELECT * FROM `sessions`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

token user_id created_at

Query results operations

Create view

Bookmark this SQL query

Label: ☐ Let every user access this bookmark

Bookmark this SQL query

Practical Task 3: Multi-User Authentication System

Fig 1: Role Selection on Register

Welcome back.

Your notes, stored securely in MySQL.

Login

Register

Username

choose a username

Email

you@example.com

Role

Student



Student

Lecturer

Administrator

Create Account →

Fig 2: Register Three Users

			id	username	email	password	role	created_at	
☐	Edit	Copy	Delete	1400bac7-527c-4d15-ba02-5d18eae54dac	JohnKiriaku	kiriakujohn@gmail.com	Player8-Ouch7-Recluse5-Sporty1-Flatbed6	administrator	2026-06-18 08:03:19
☐	Edit	Copy	Delete	1a6126e2-feb9-4c6b-a352-81effd5bad5c	BigYahu	yahuuben@gmail.com	Thorn1-Uranium6-Overhaul1-List7-Kung4	lecturer	2026-06-18 08:02:31
☐	Edit	Copy	Delete	679fa6a4-76f1-4ea4-b905-71d3d29613e6	Charles	charlesplays@gmail.com	Shelf7-Uptown5-Conductor8-Bubbly5-Handsfree5	student	2026-06-18 08:01:28

Fig 3: Role Badge on Dashboard

NoteKeeper

logged in as **BigYahu** **LECTURER** [logout](#)

MY NOTES

0

Search notes...

No notes yet.
Create your first one →

New Note

CREATE NOTE

Title

Note title...

Category

General

Content

Write your note here...

Fig 4: Session Management

		token	user_id	created_at
☐	Edit Copy Delete	a04b8c86-706e-4ed0-a33e-136fce2b1a69	1a6126e2-feb9-4c6b-a352-81effd5bad5c	2026-06-18 08:04:21
☐ Check all		With selected: Edit Copy Delete Export		

Fig 5: Access Restriction

```
async function requireAuth(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).json({ error: 'No token provided' });

  const [rows] = await db.execute(
    'SELECT user_id FROM sessions WHERE token = ?', [token]
  );
  if (rows.length === 0) return res.status(401).json({ error: 'Invalid or expired session' });

  req.userId = rows[0].user_id;
  next();
}
```

Bonus Features

1. Remember Me: localStorage Keeps Session

Fig 1: Browser DevTools Application tab

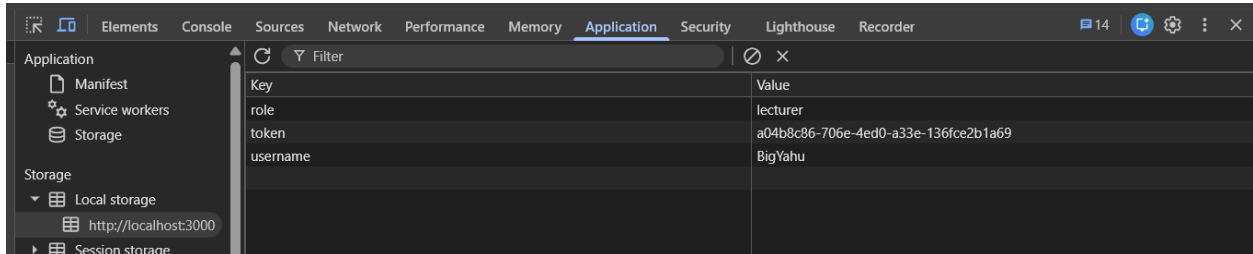
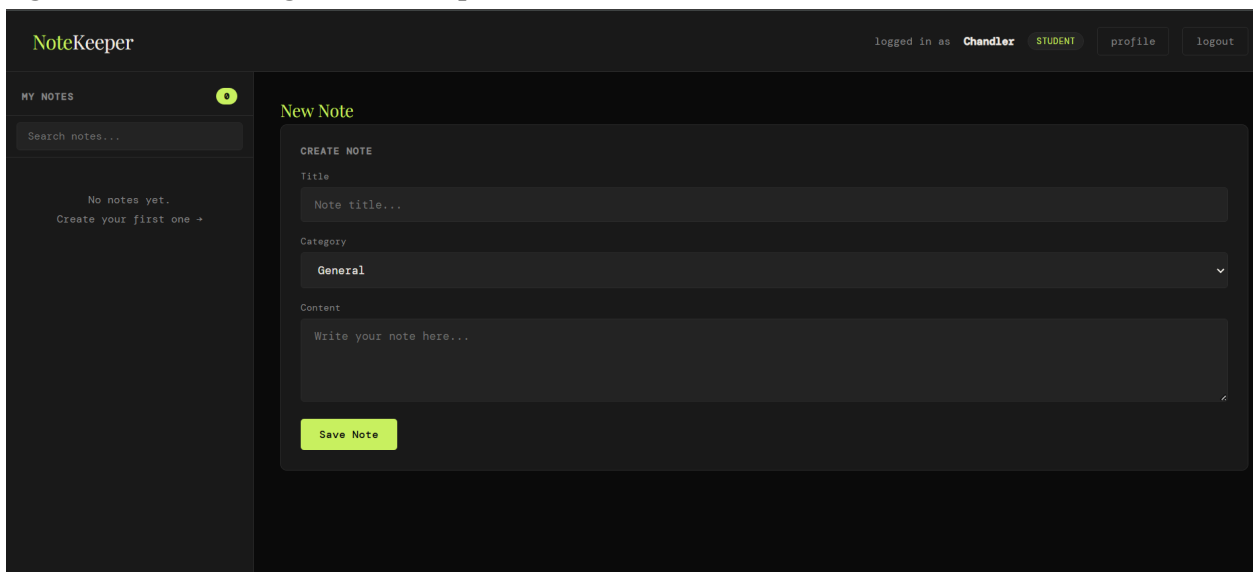


Fig 2: Browser showing the NoteKeeper dashboard



2. Email Verification

Fig 3: Browser showing the registration success message

Welcome back.

Your notes, stored securely in MySQL.

Login

Register

Forgot Password

Username

Chandler

Email

chandlermxtere@gmail.com

Role

Student



Password

.....

Strong

Create Account →

✓ Account created! Please check your email to verify your account.

Fig 4: Gmail inbox showing the NoteKeeper email verification message sent via Nodemailer using Gmail SMTP

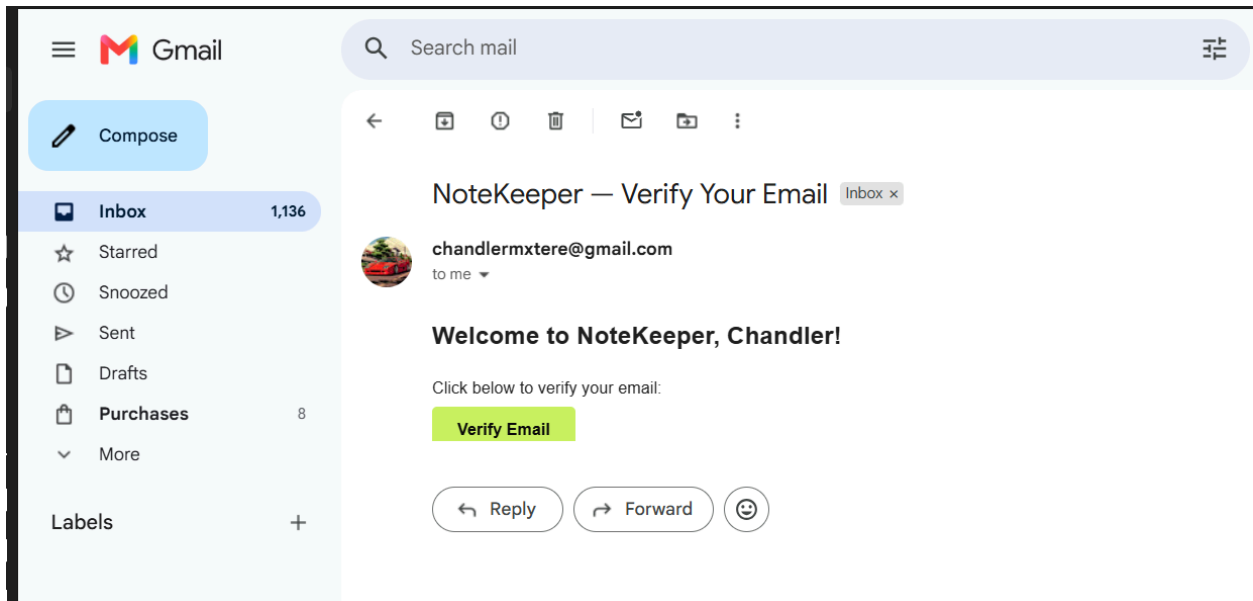


Fig 5: Browser showing the email verified confirmation page after clicking the verification link, confirming the token-based email verification system works correctly

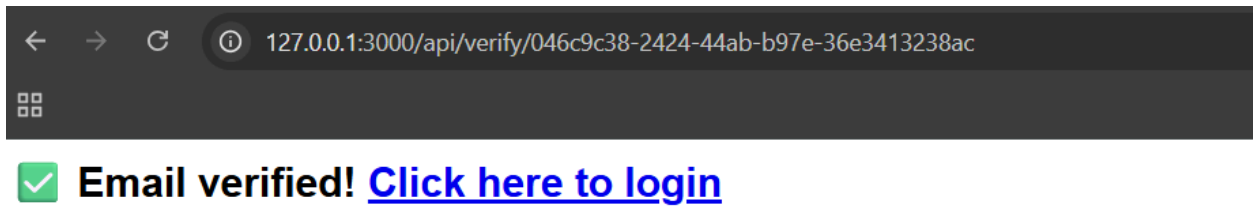
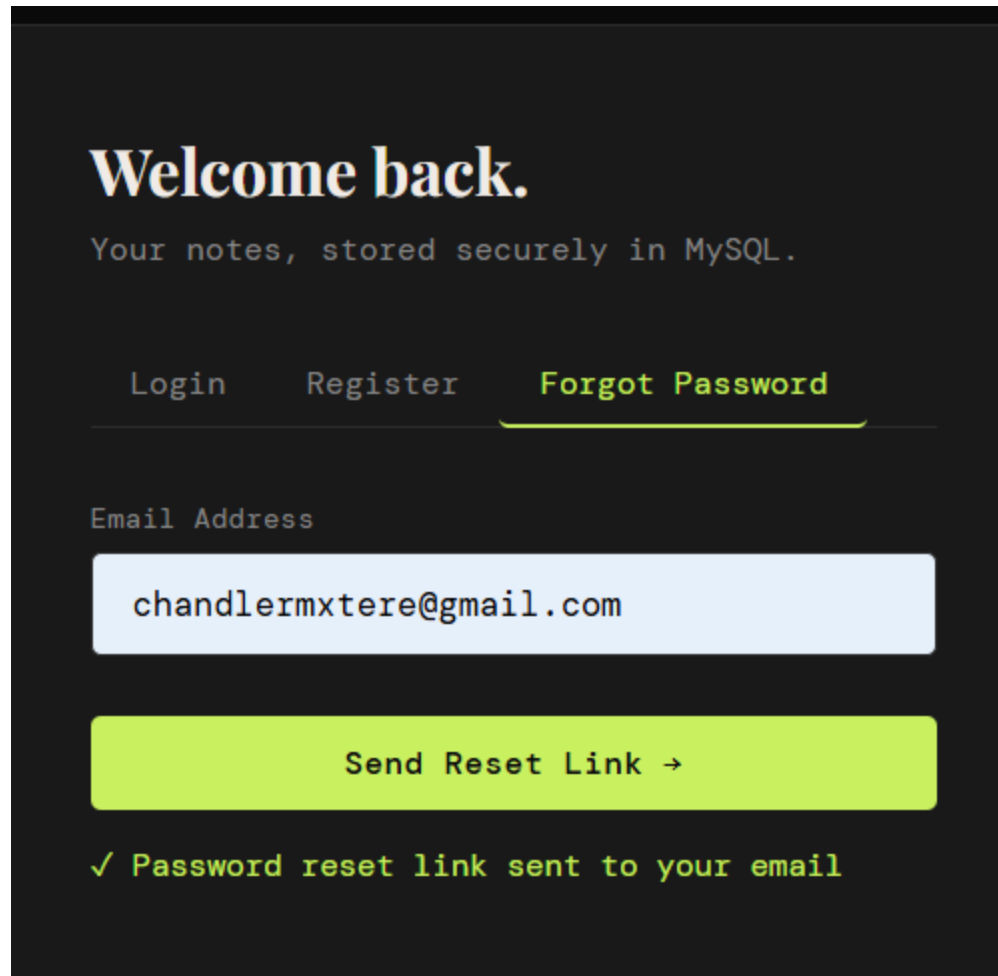


Fig 6: phpMyAdmin Browse view of the users table showing is_verified set to 1 confirming the verification status is updated and stored in the MySQL database

	id	username	email	password	role	is_verified	verify_token	reset_token	reset_expires	two_fa_secret	two_fa_enabled
	e584e197-60cf-466e-b593-850786fbfedb	Chandler	chandlermxtere@gmail.com	Pronto6-Art1-Retrieval4-Jockey7-Blizzard5	student	1	NULL	NULL	NULL	NULL	0

3. Password Reset

Fig 7: Browser showing the Forgot Password form with a success message confirming a password reset email was sent to the registered email address



Welcome back.

Your notes, stored securely in MySQL.

Login Register **Forgot Password**

Email Address

chandlermxtere@gmail.com

Send Reset Link →

✓ Password reset link sent to your email

Fig 8: Gmail inbox showing the NoteKeeper password reset email with the Reset Password link

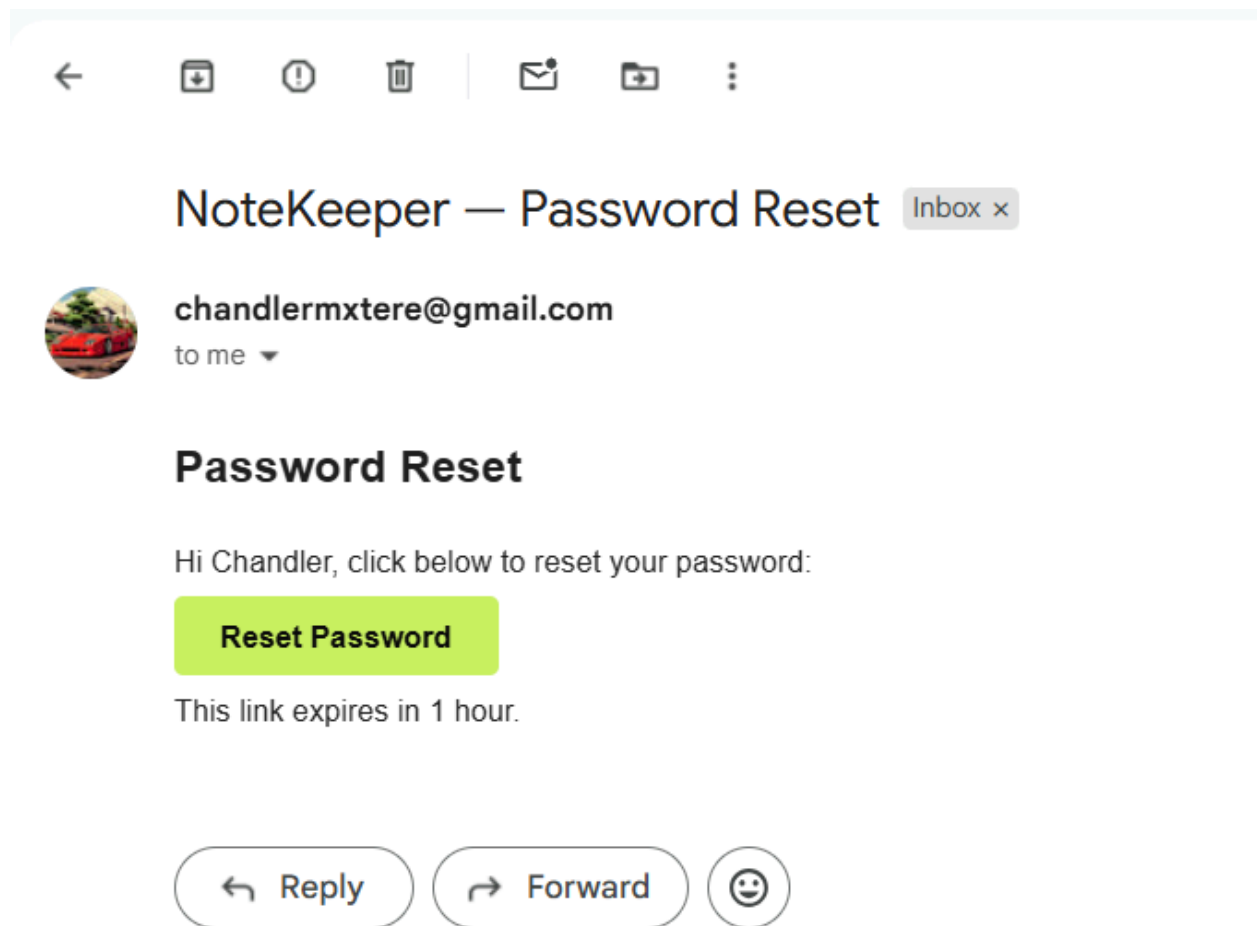
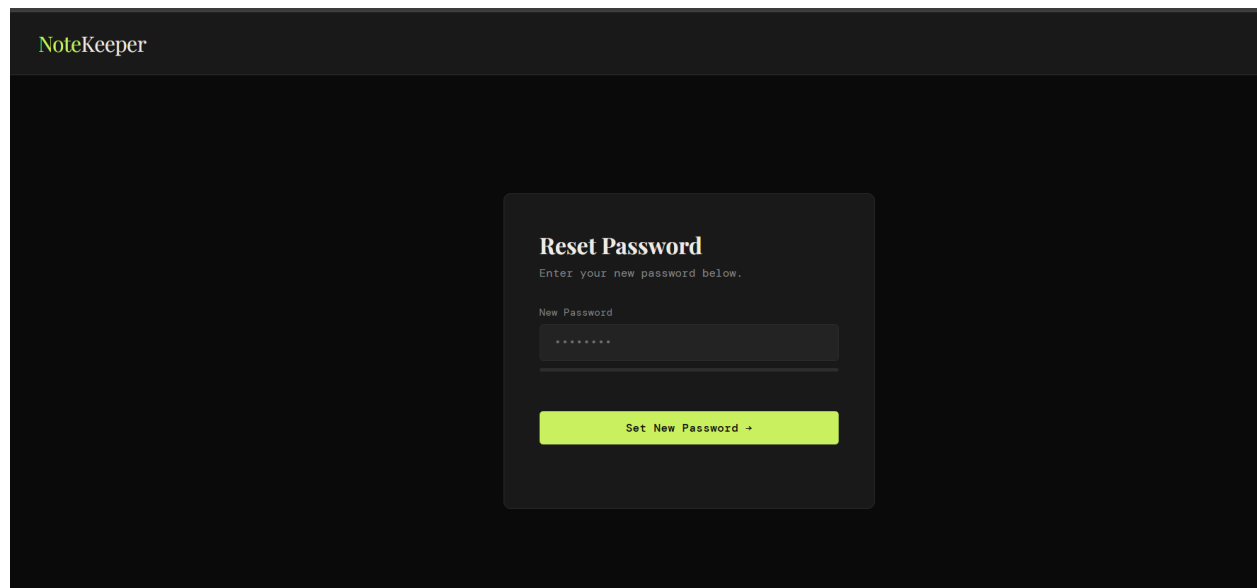


Fig 9: Browser showing the reset password page with the new password field and live strength indicator



4. 2FA: Google Authenticator

Fig 10: Browser showing the NoteKeeper profile panel with the QR code generated for Google Authenticator setup

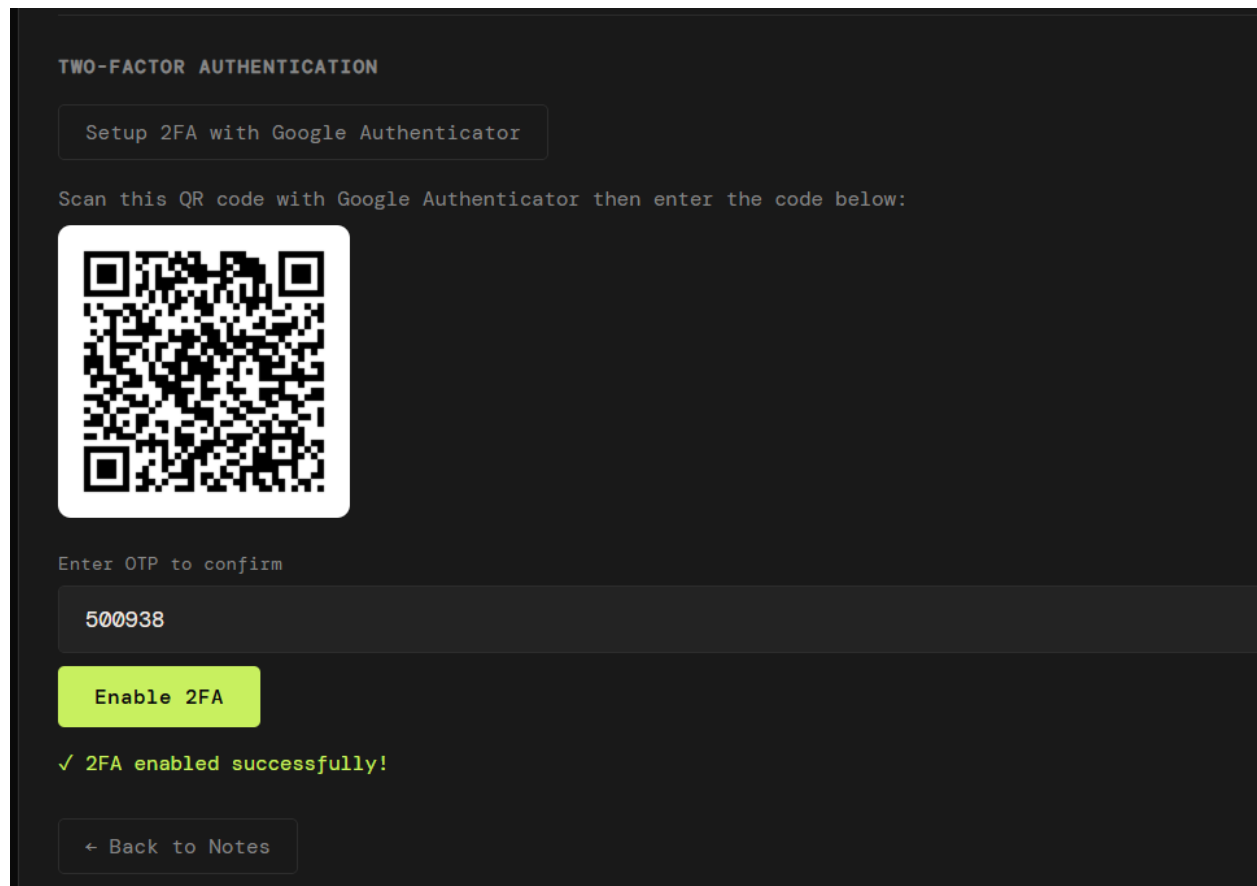


Fig 11: Browser showing the 2FA enabled success message after the OTP was verified and confirmed against the stored secret

TWO-FACTOR AUTHENTICATION

Setup 2FA with Google Authenticator

Scan this QR code with Google Authenticator then enter the code below:



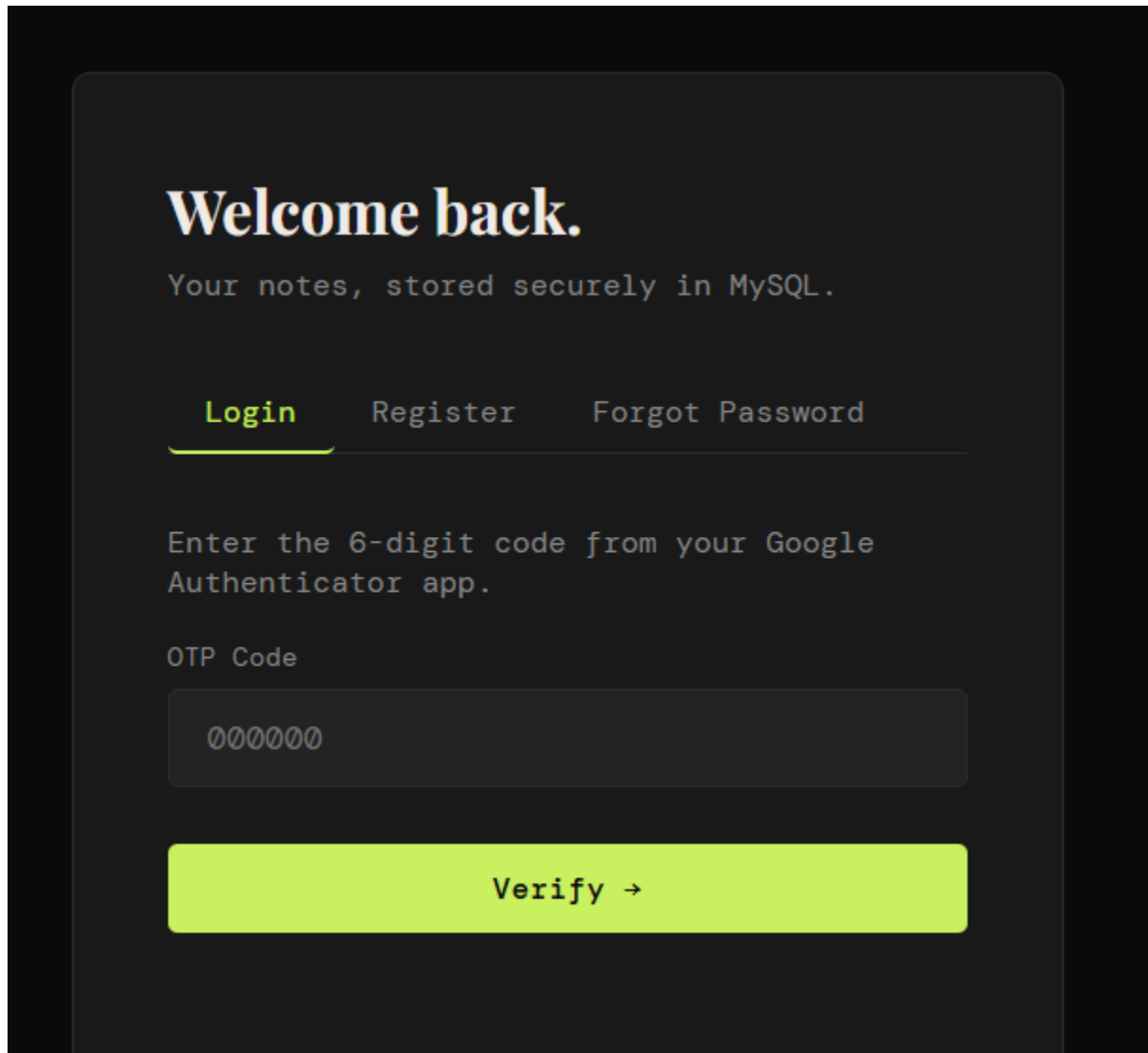
Enter OTP to confirm

787604

Enable 2FA

✓ 2FA enabled successfully!

Fig 12: Browser showing the OTP input screen appearing after username and password are verified, confirming 2FA is enforced before granting dashboard access



5. Profile Management

Fig 13: Browser showing the full profile panel displaying username, email, role, email verification status, 2FA status and account creation date

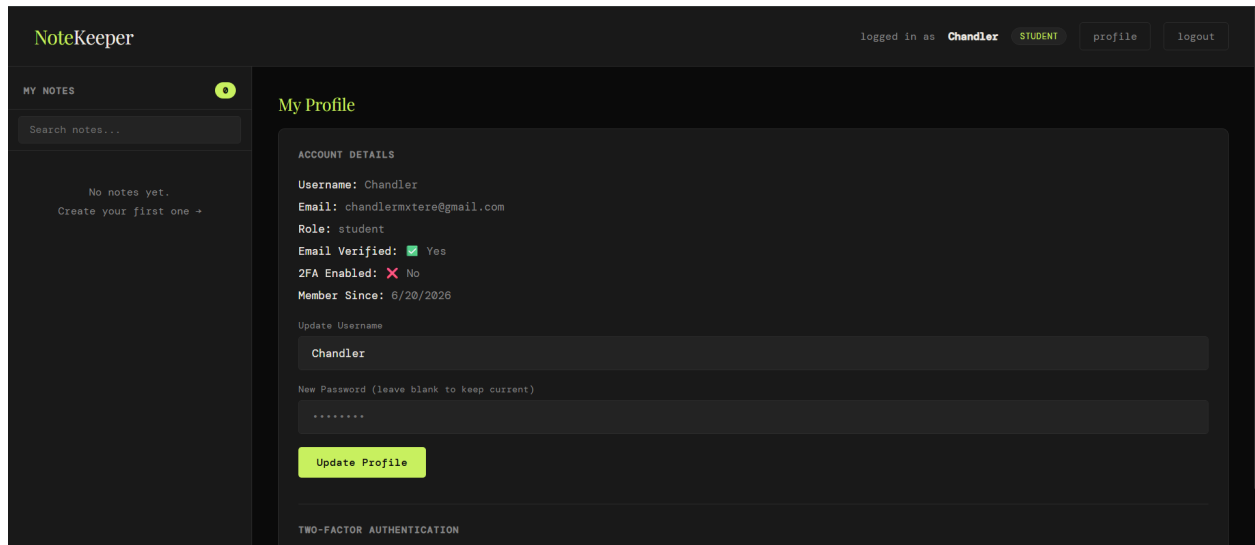
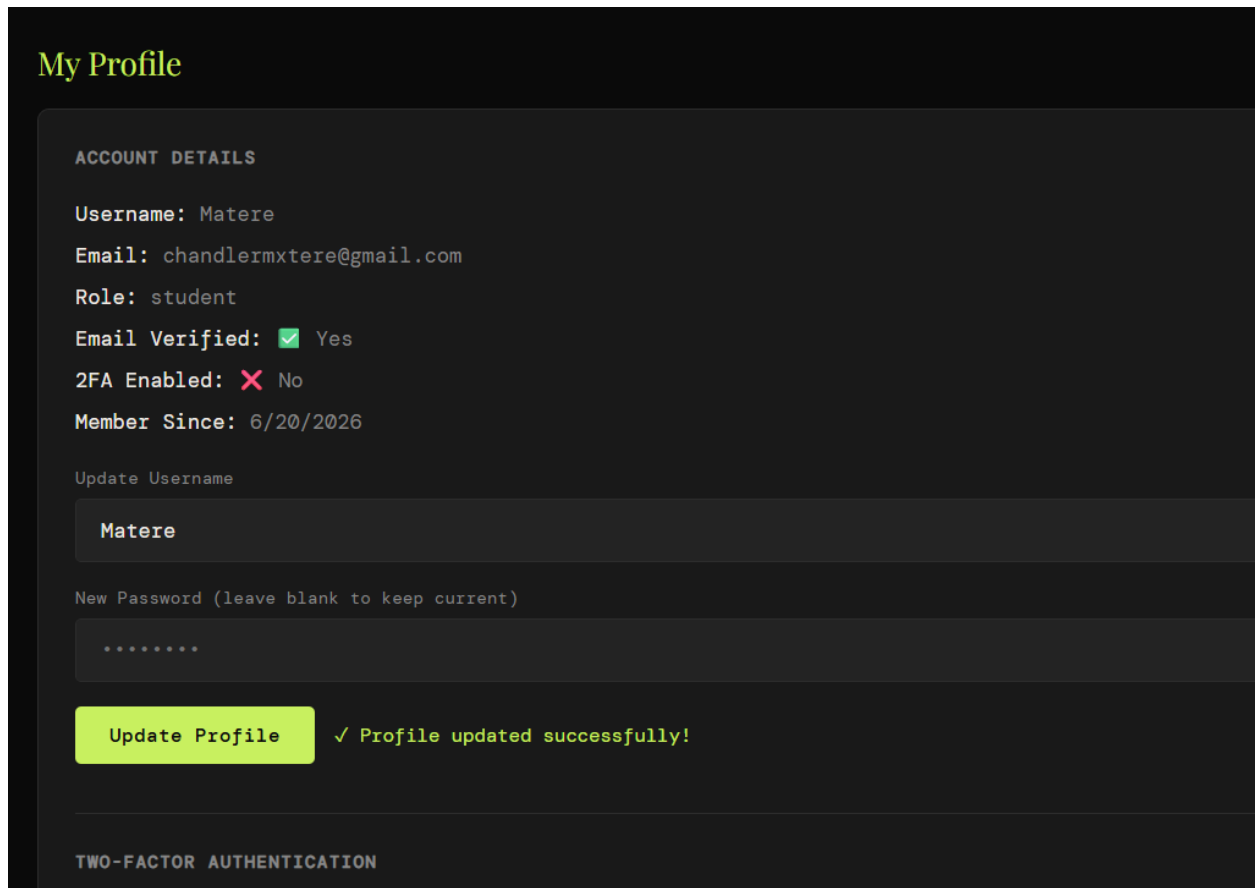


Fig 14: Browser showing the profile updated success message with the new username reflected in the header after submitting changes



Week 8: Responsive Web Design and Mobile-First Development

Task 1: Create a Responsive Personal Profile Page

Fig 1: Desktop view of the personal profile page showing the hero section with responsive profile image, name, bio and navigation built using HTML, CSS Flexbox and a blue gradient theme

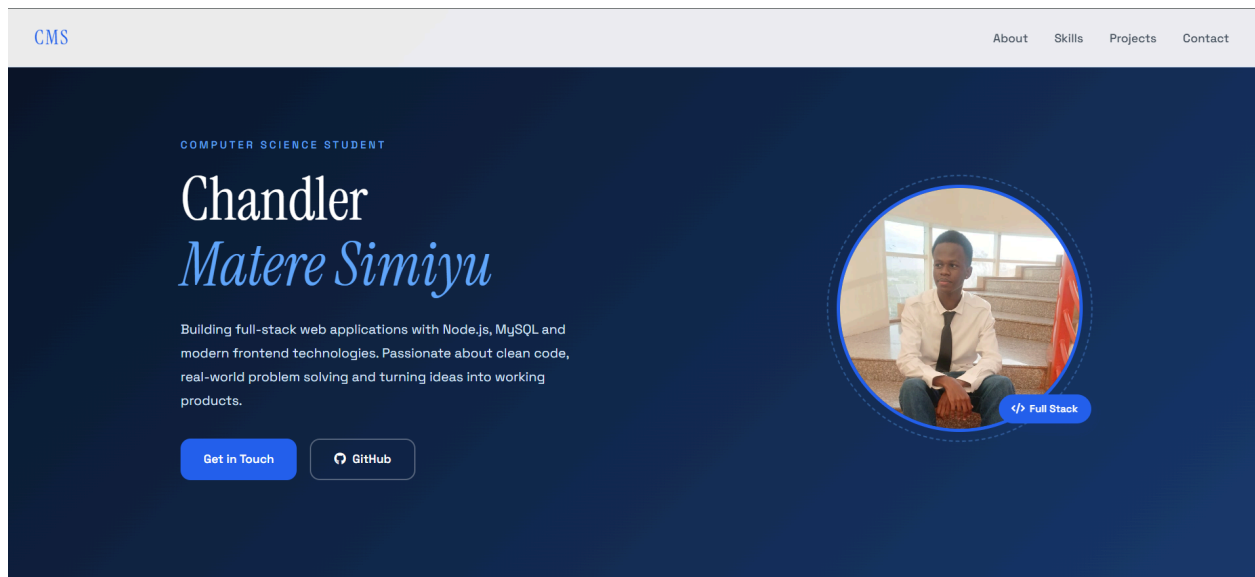


Fig 2: Tablet view of the profile page showing the responsive layout adjusting to a narrower viewport with the profile image stacking above the text content using CSS media queries at 768px breakpoint

CMS

About Skills Projects Contact

COMPUTER SCIENCE STUDENT

Chandler *Matere Simiyu*

Building full-stack web applications with Node.js, MySQL and modern frontend technologies.
Passionate about clean code, real-world problem solving and turning Ideas into working products.

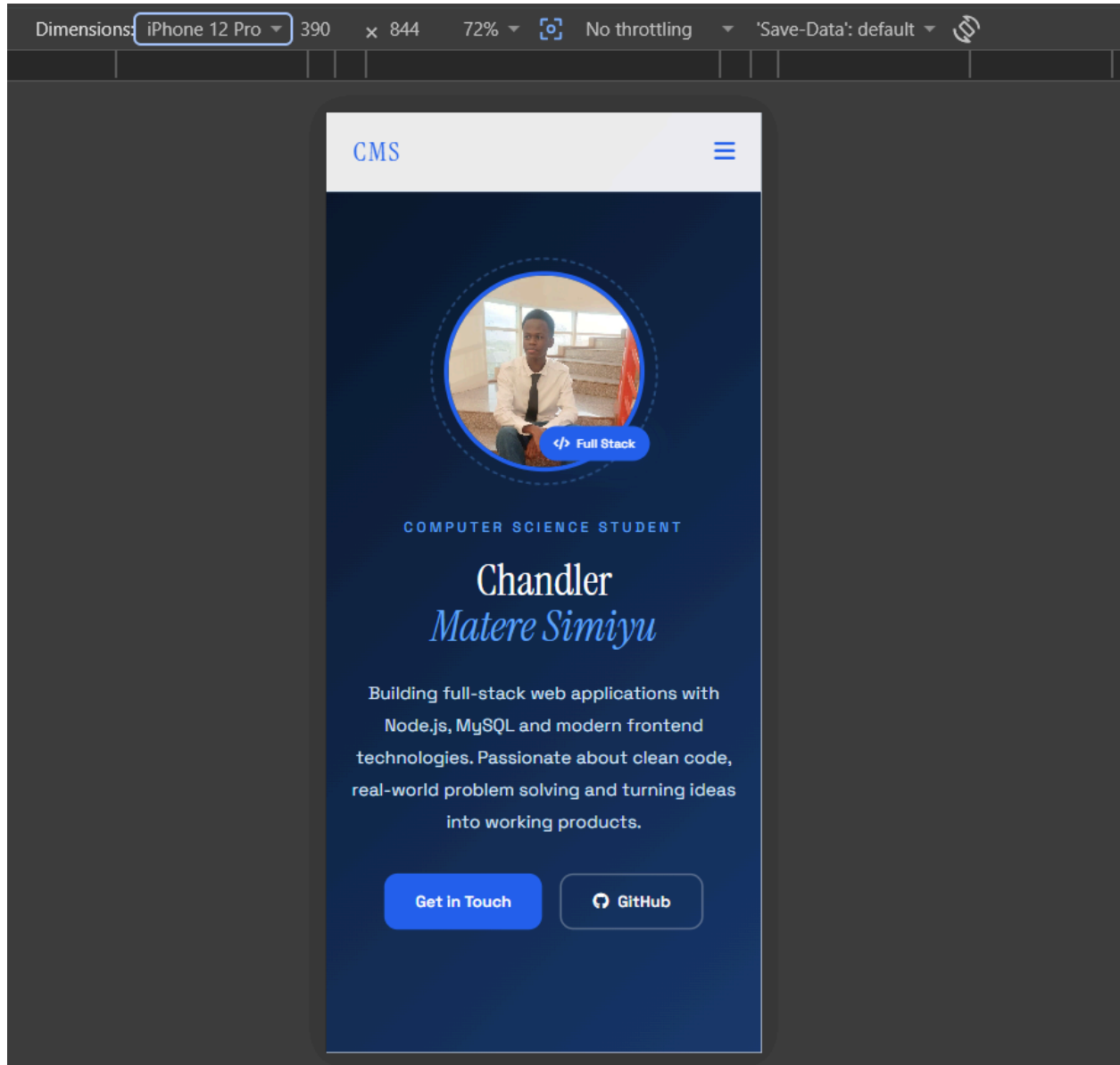
Get in Touch

GitHub



Full Stack

Fig 3: Mobile view of the profile page showing the single column layout with the hamburger navigation menu and centered content, demonstrating the mobile first responsive design at the 480px breakpoint



Task 2: Build a Responsive Product Showcase

Fig 1: Desktop view of the product showcase displaying four fashion products

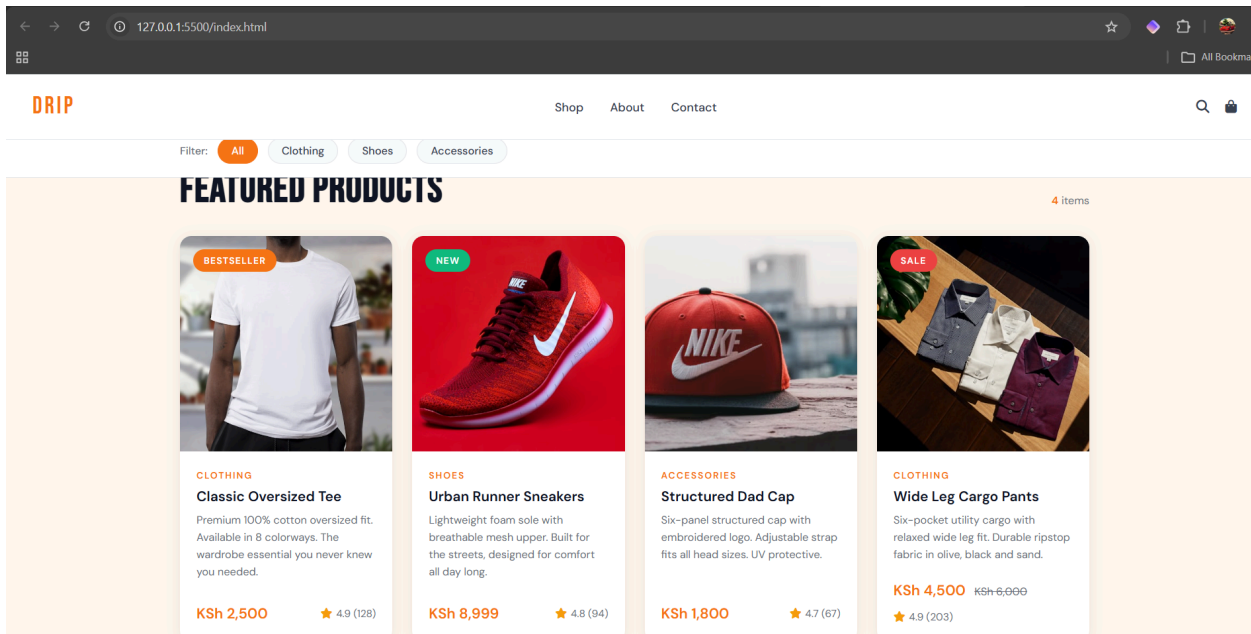


Fig 2: Tablet view of the product showcase at the 600px breakpoint showing two products per row

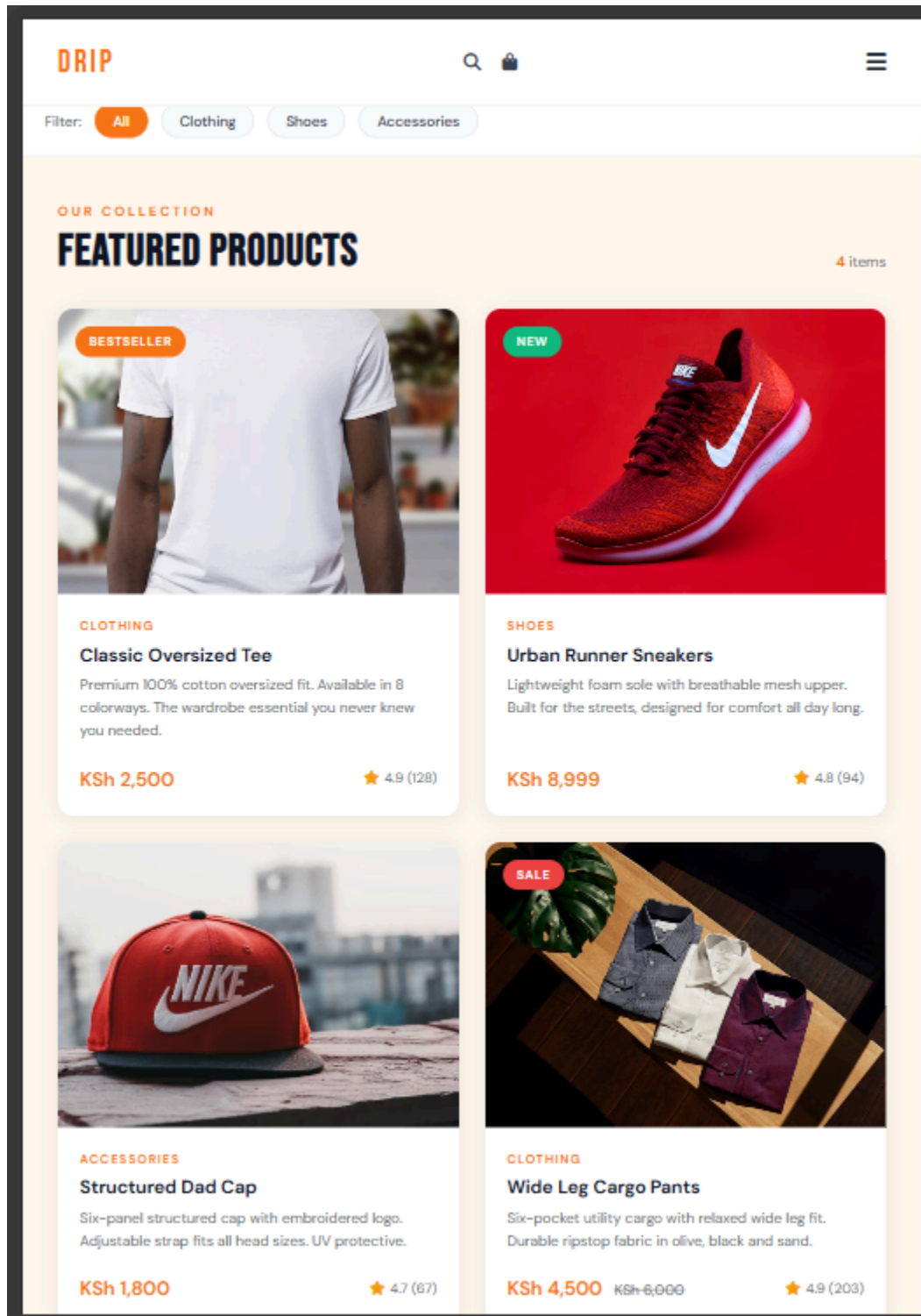


Fig 3: Mobile view of the product showcase showing one product per row approach as the base layout

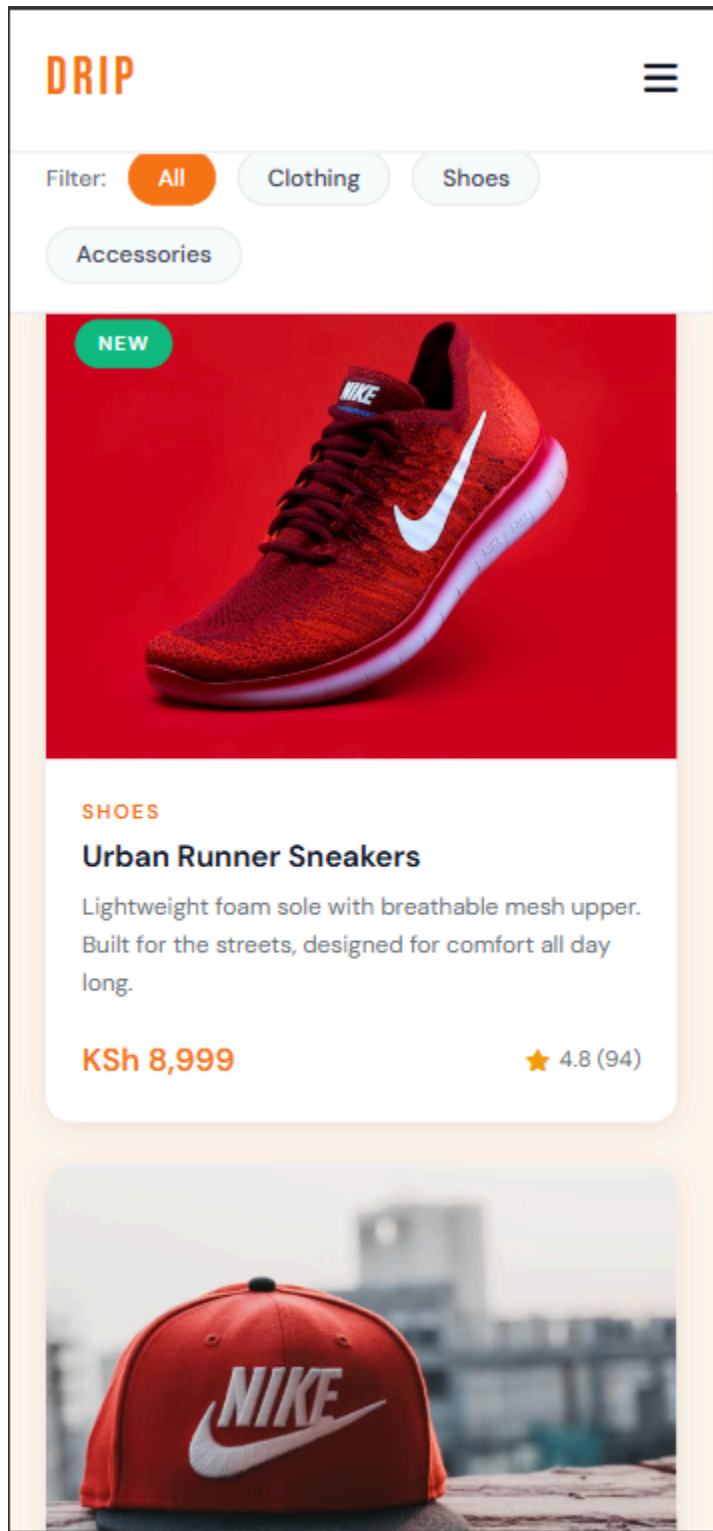
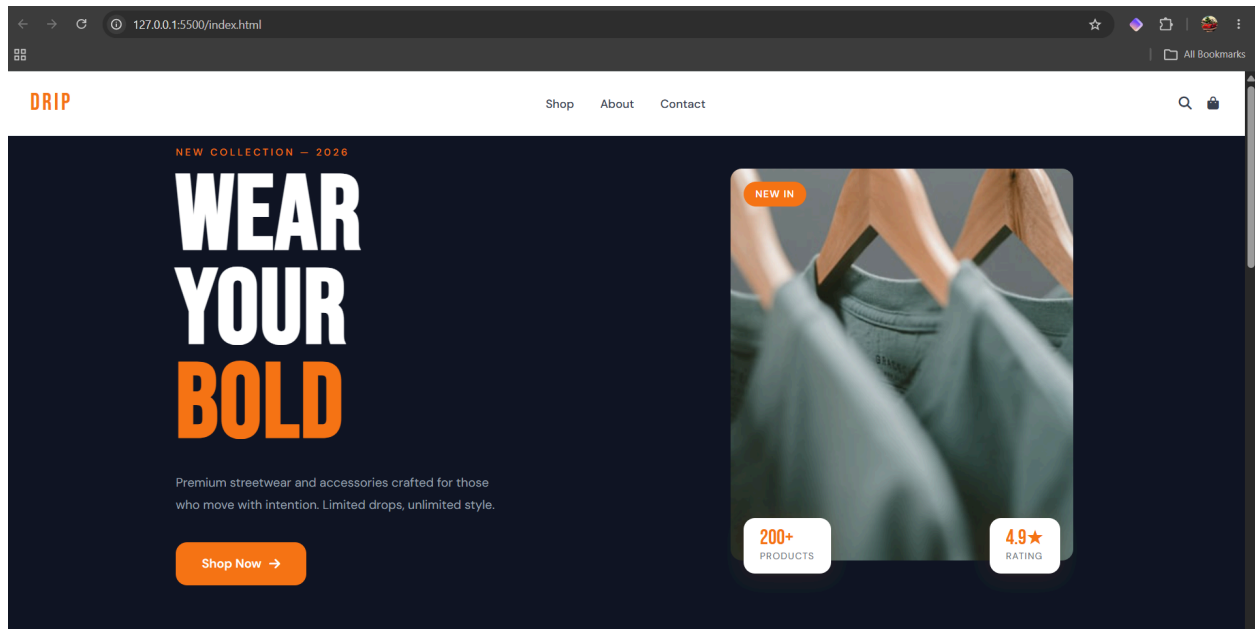


Fig 4: Desktop hero section of the DRIP fashion showcase showing the responsive typography



Extra Practical Assignment

Fig. 1: Mobile Viewport Rendering of the Hero Section and Collapsed Navigation Menu

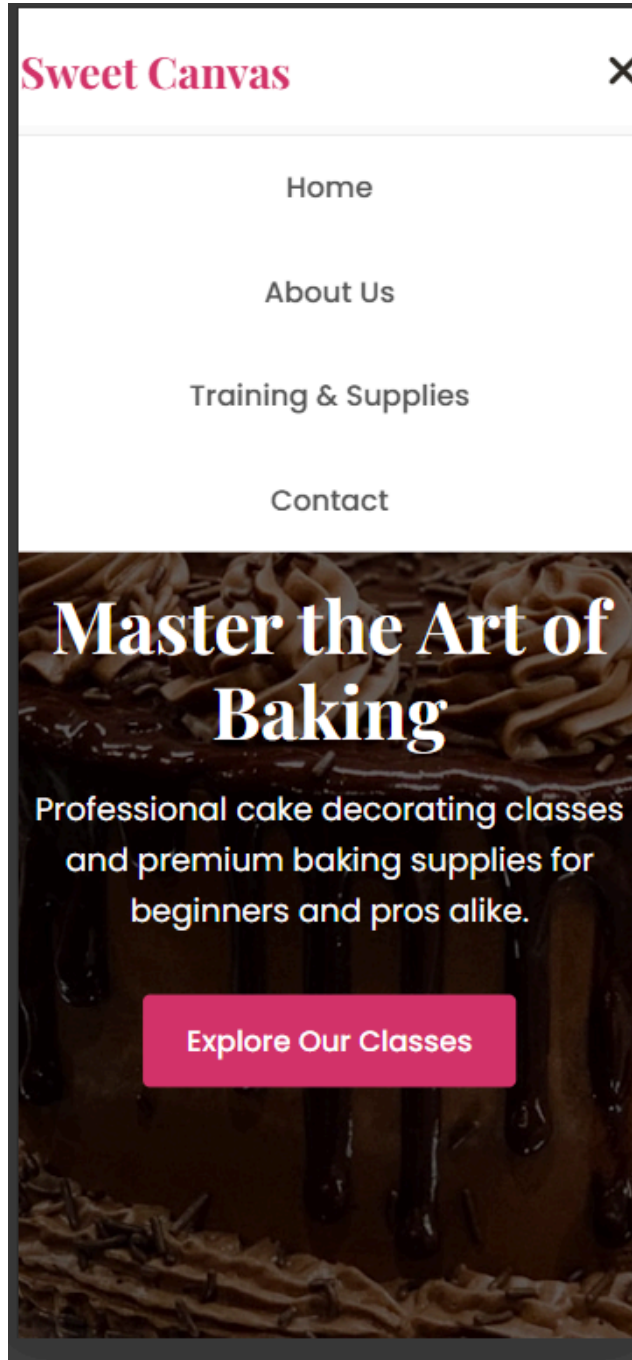


Fig. 2: Desktop Viewport Demonstrating the 3-Column CSS Grid Implementation for Services

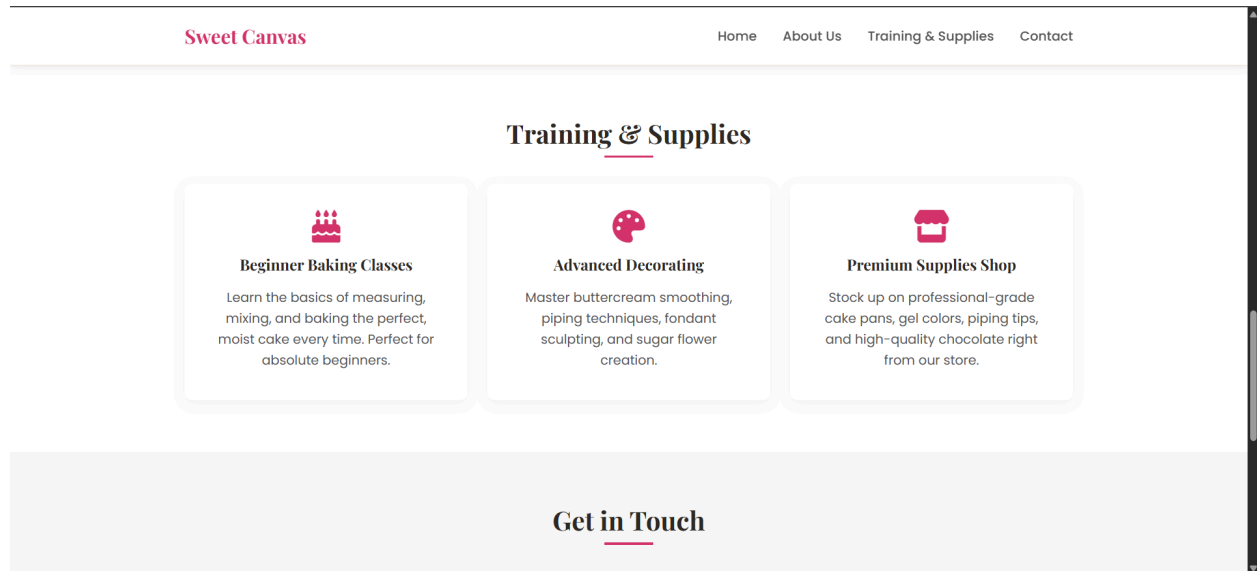
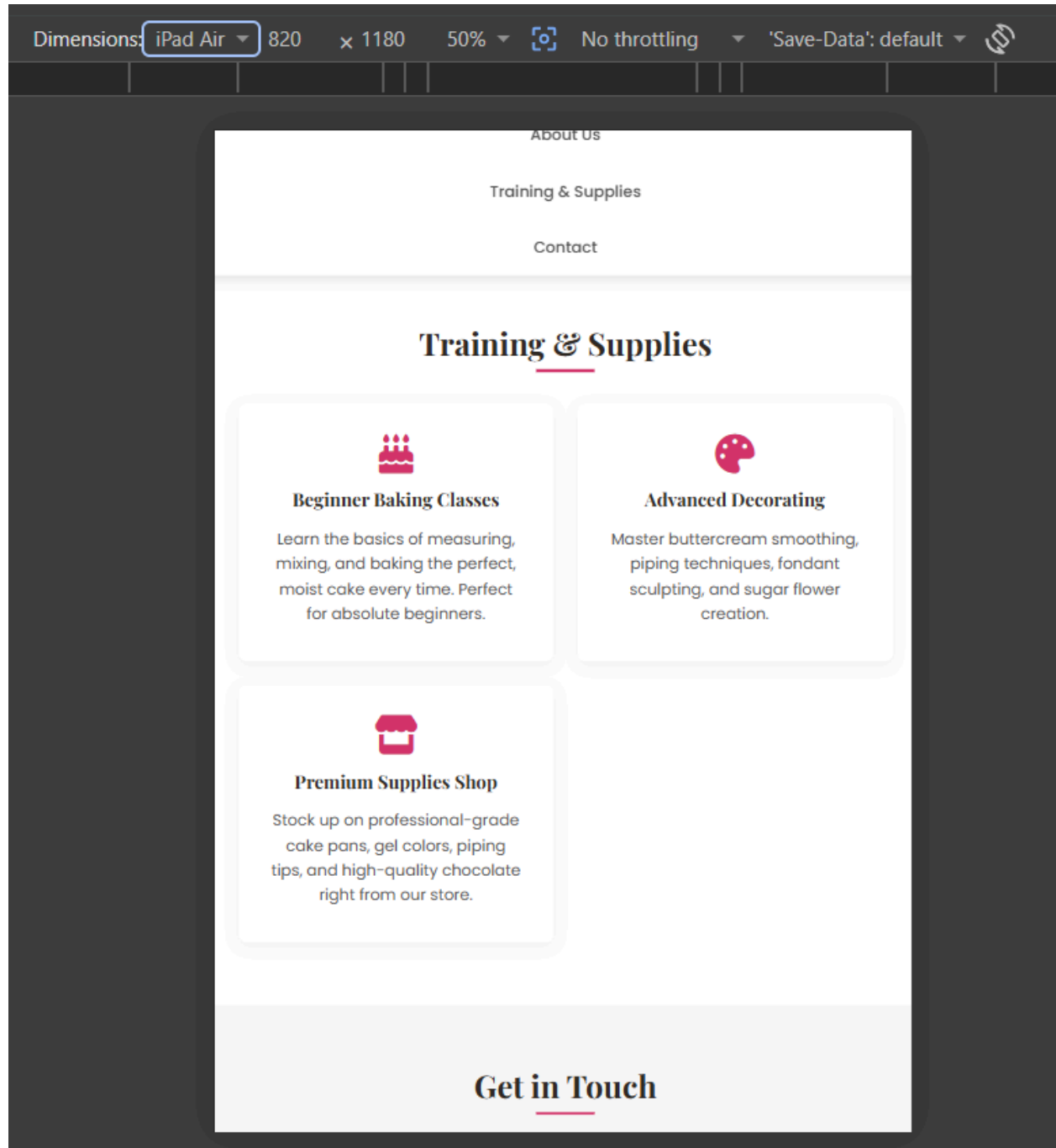


Fig. 3: Tablet/Desktop Viewport Demonstrating the 2-Column Grid Layout for the Contact Section



Week 9: Java Web Services: Servlet Lifecycle, Session Management, and Cookies

Practical Activity: Student Login System

Fig 1: Browser showing the NoteKeeper login form with username and password fields demonstrating the login page equivalent to login.jsp

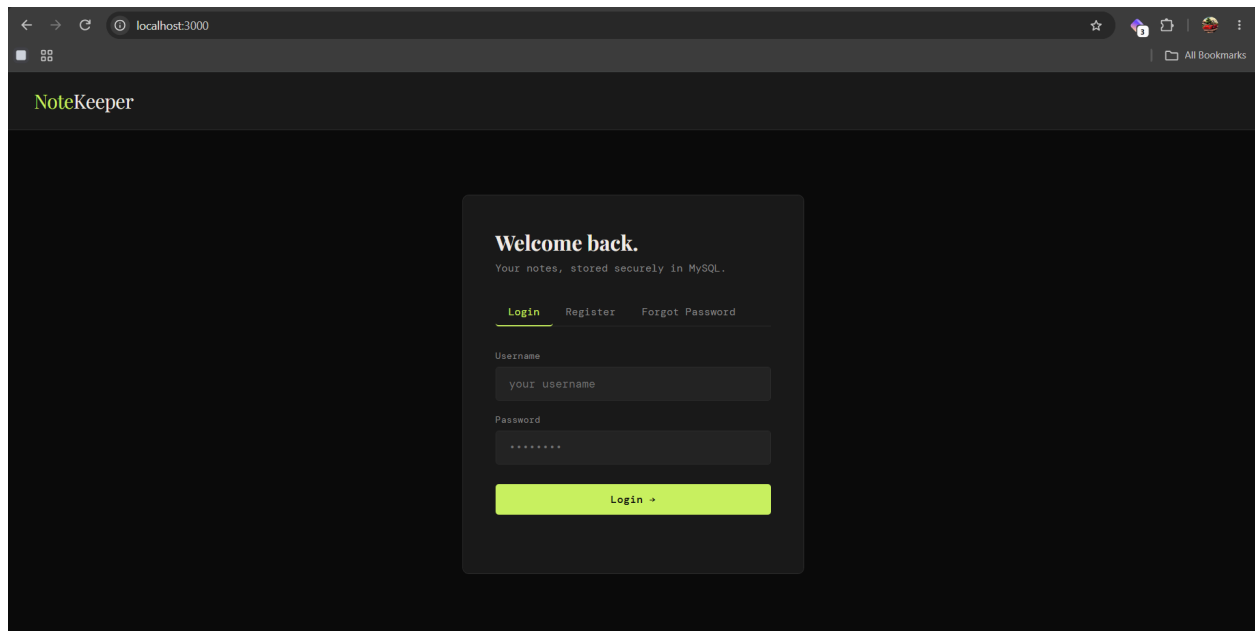


Fig 2: Browser showing the NoteKeeper dashboard after successful login displaying the welcome message with the logged in username confirming session creation

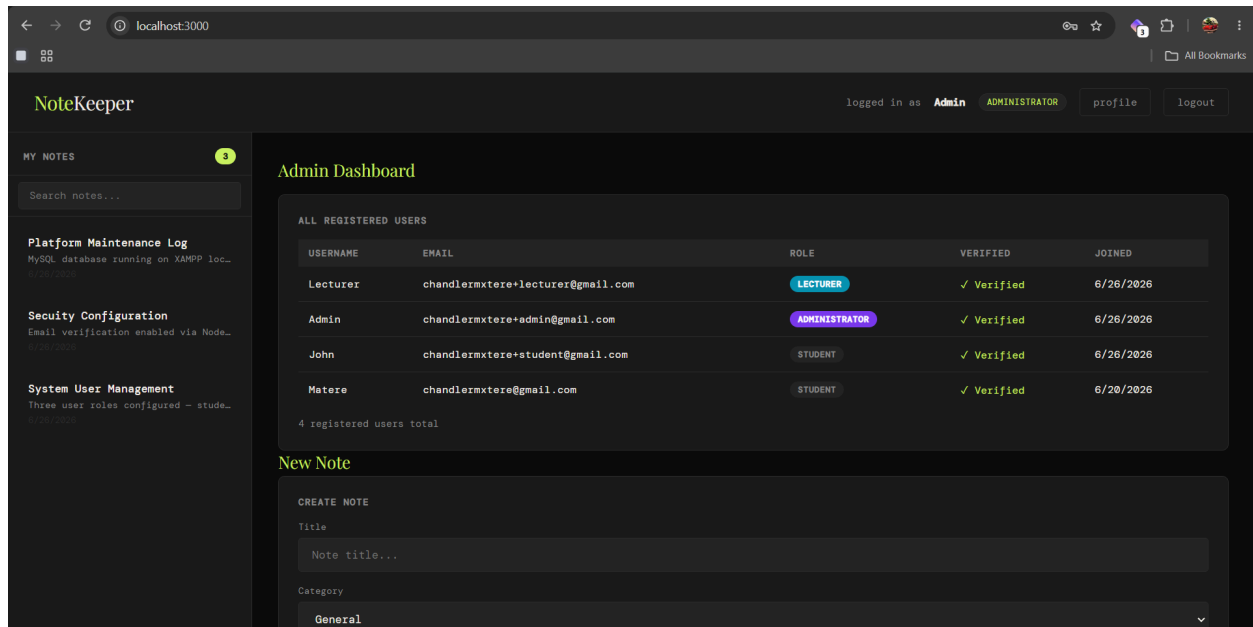


Fig 3: VS Code showing the requireAuth middleware in server.js demonstrating how unauthenticated requests are blocked and sessions are validated on every protected route

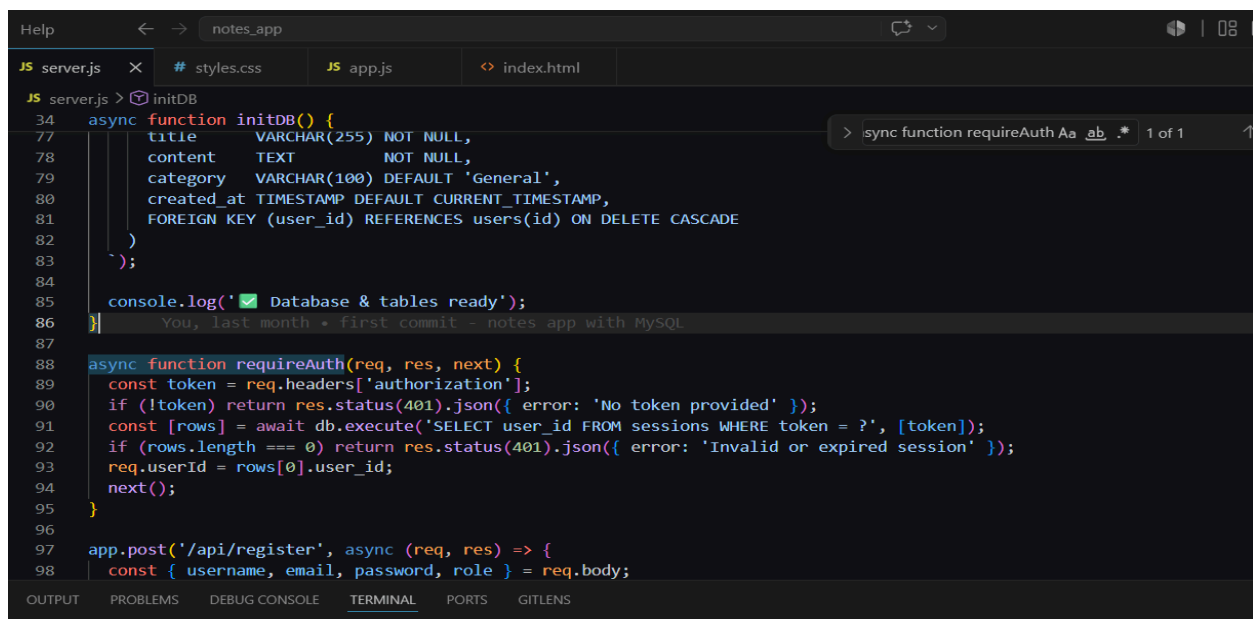
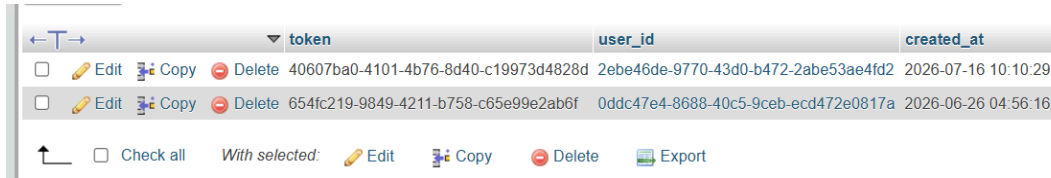
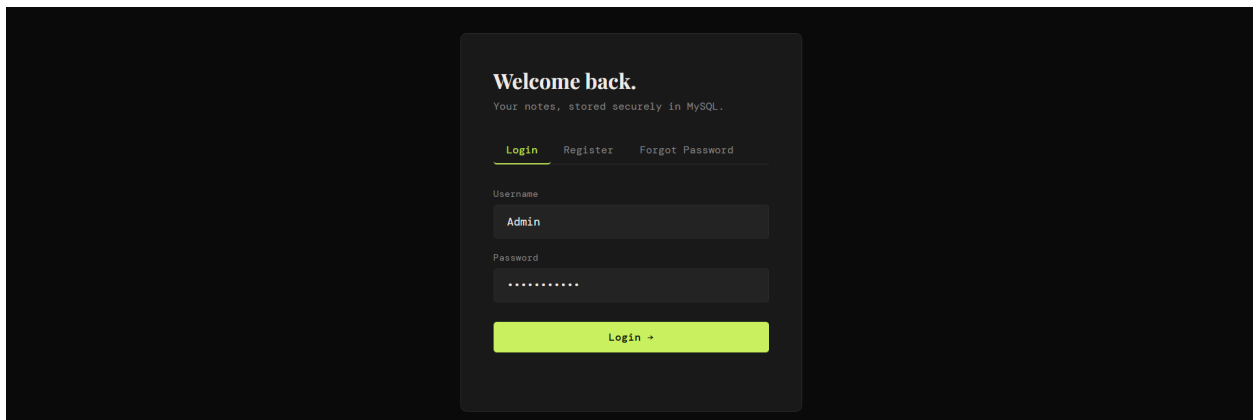


Fig 4: phpMyAdmin Browse view of the sessions table showing the active session token linked to the user_id confirming server side session creation after login



	token	user_id	created_at
☐ Edit Copy Delete	40607ba0-4101-4b76-8d40-c19973d4828d	2ebe46de-9770-43d0-b472-2abe53ae4fd2	2026-07-16 10:10:29
☐ Edit Copy Delete	654fc219-9849-4211-b758-c65e99e2ab6f	0ddc47e4-8688-40c5-9ceb-ecd472e0817a	2026-06-26 04:56:16

Fig 5: Browser showing the login page after clicking logout confirming the session was destroyed and the user was redirected back to the login page



Class Exercise 1: Student Login Web Application

Fig 1: Browser showing the NoteKeeper register and login tabs demonstrating the login page with username and password input fields

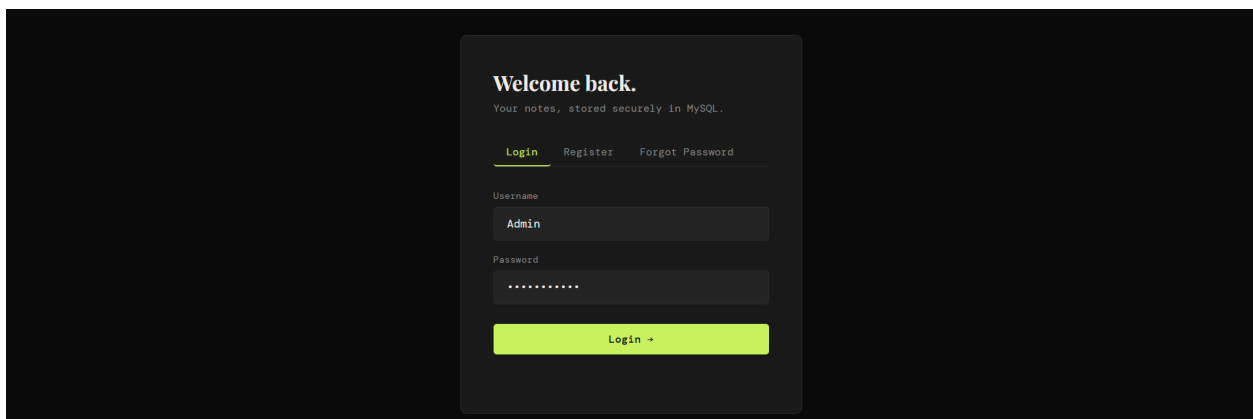


Fig 2: Browser showing JavaScript validation error when submitting empty username field confirming client side validation is implemented

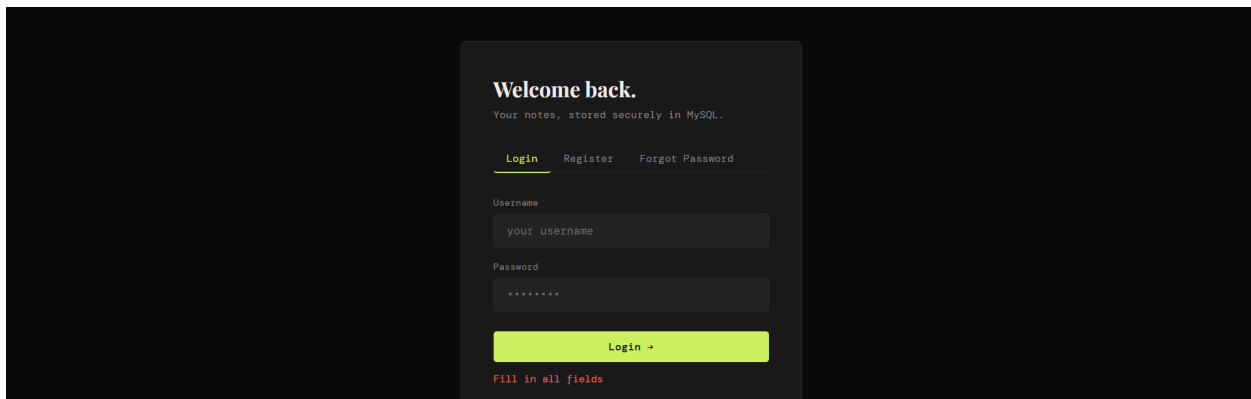


Fig 3: Browser showing the dashboard after successful login with the username displayed in the header confirming the welcome page and session username display

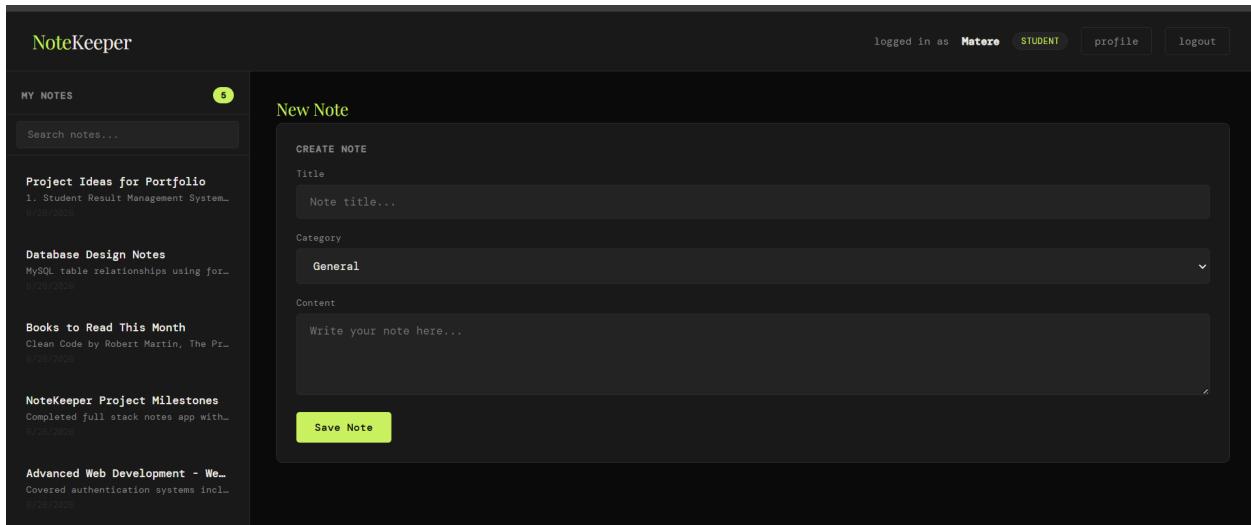
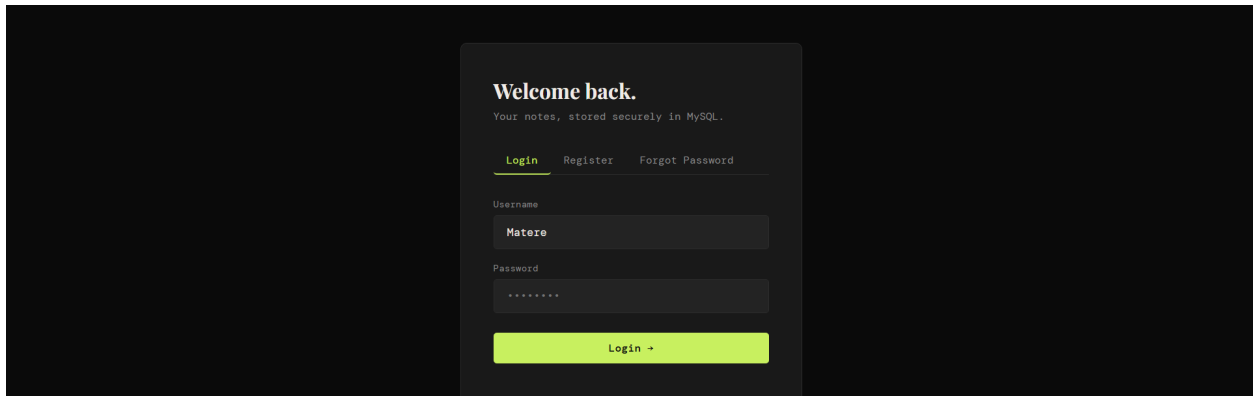


Fig 4: phpMyAdmin Browse view of the sessions table showing a newly created session token and its creation timestamp after login

				token	user_id	created_at
☐				40607ba0-4101-4b76-8d40-c19973d4828d	2ebe46de-9770-43d0-b472-2abe53ae4fd2	2026-07-16 10:10:29
☐				654fc219-9849-4211-b758-c65e99e2ab6f	0ddc47e4-8688-40c5-9ceb-ecd472e0817a	2026-06-26 04:56:16

☐ Check all With selected: Edit Copy Delete Export

Fig 5: Browser showing the login page after clicking the logout button confirming the session was invalidated and the user was redirected



Class Exercise 2: Session Tracking and Cookies Enhancement

Fig 1: Browser DevTools Application tab showing the token stored in localStorage confirming the logged in username is tracked client side without cookies

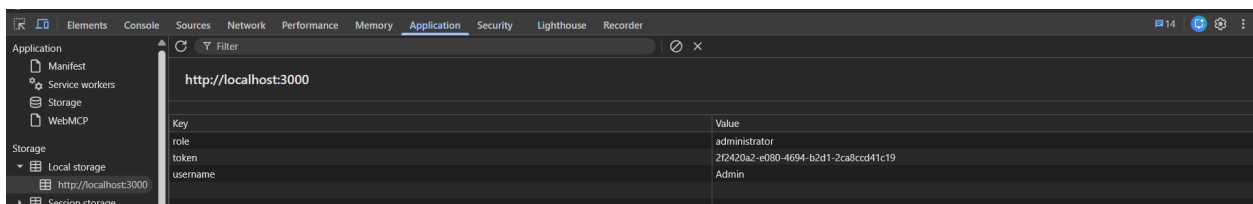


Fig 2: Browser showing the NoteKeeper dark theme applied across all pages demonstrating the theme preference stored via CSS variables in styles.css

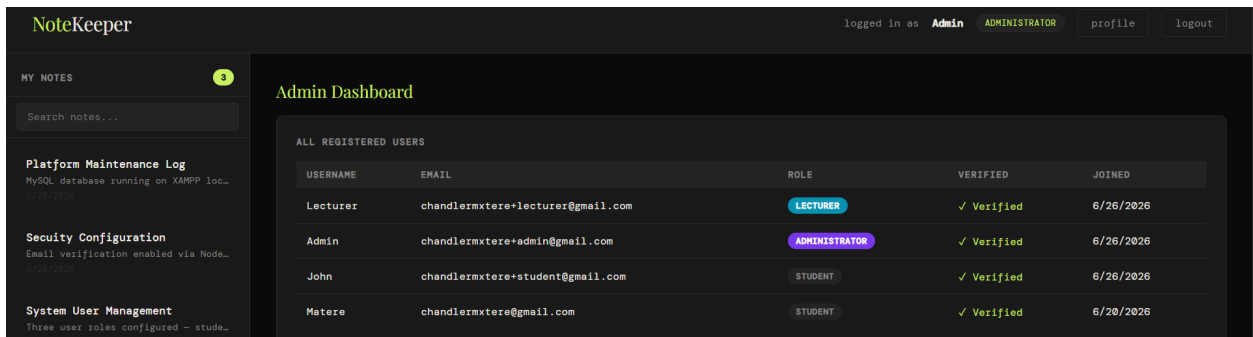


Fig 3: Browser DevTools Application tab showing the token and username values in localStorage confirming session ID equivalent and login persistence

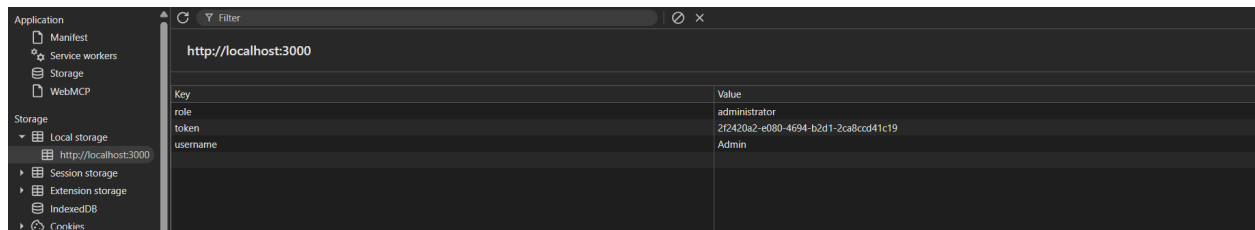


Fig 4: VS Code showing the login route in server.js with the is_verified check and session token INSERT query demonstrating how login time is recorded via the created_at timestamp in the sessions table



Assignment: Remember Me Feature

Fig 1: Browser showing the NoteKeeper login page

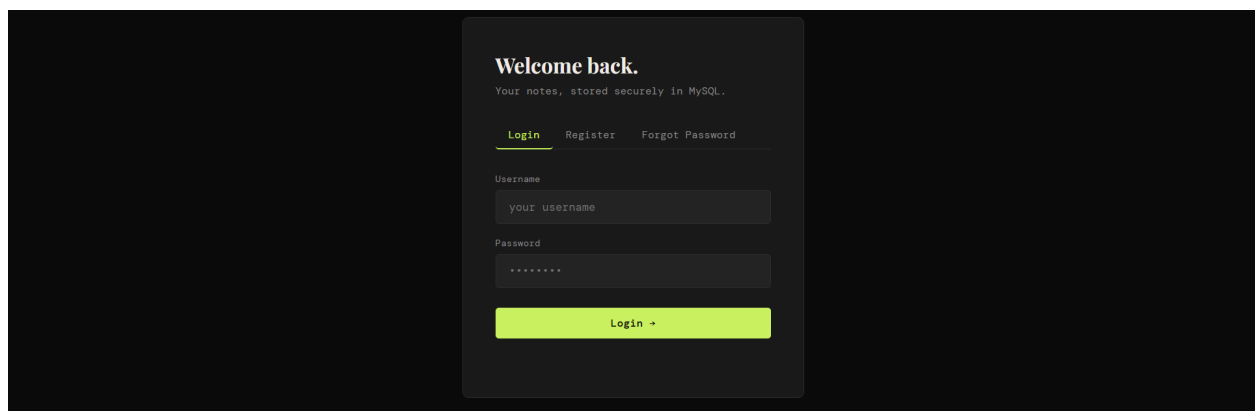


Fig 2: Browser DevTools Application tab showing token and username persisted in localStorage after login confirming the Remember Me functionality

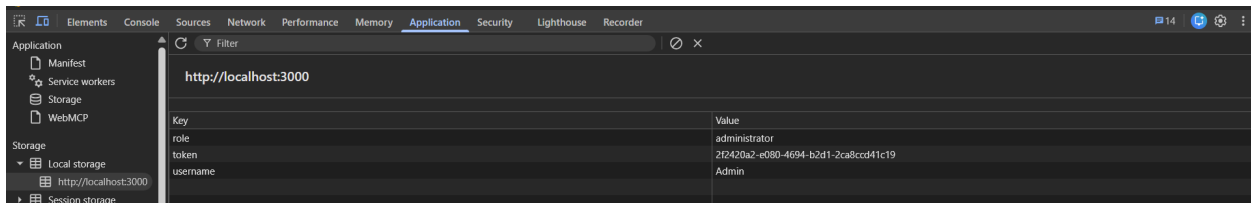


Fig 3: Browser showing the app still logged in after closing and reopening confirming the Remember Me feature works across browser sessions

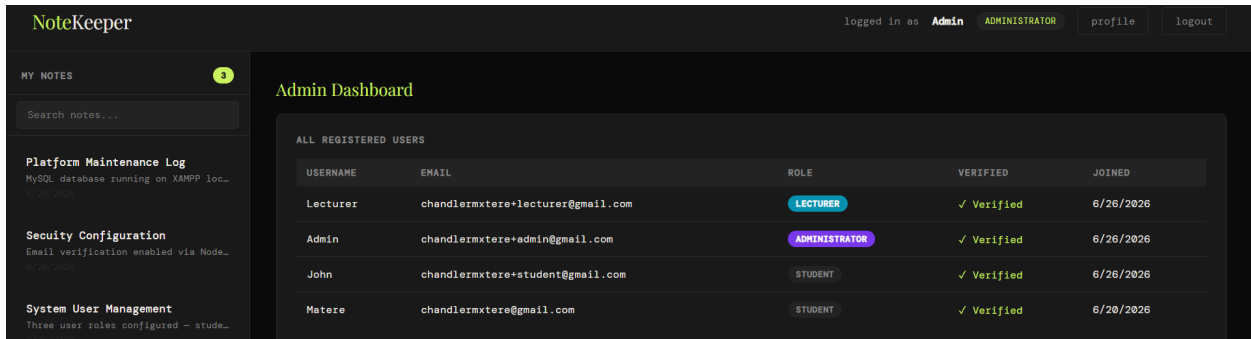
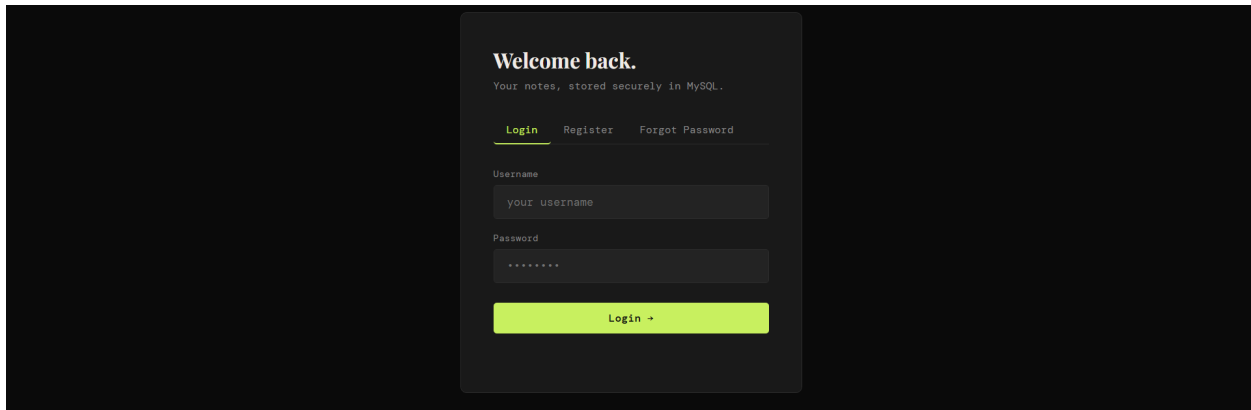


Fig 4: Browser showing the login page after logout confirming the token was cleared from both localStorage and the MySQL sessions table



Creativity Section

Fig 1: Gmail inbox showing the NoteKeeper email verification message sent via Nodemailer demonstrating an additional security layer not present in basic servlet implementations

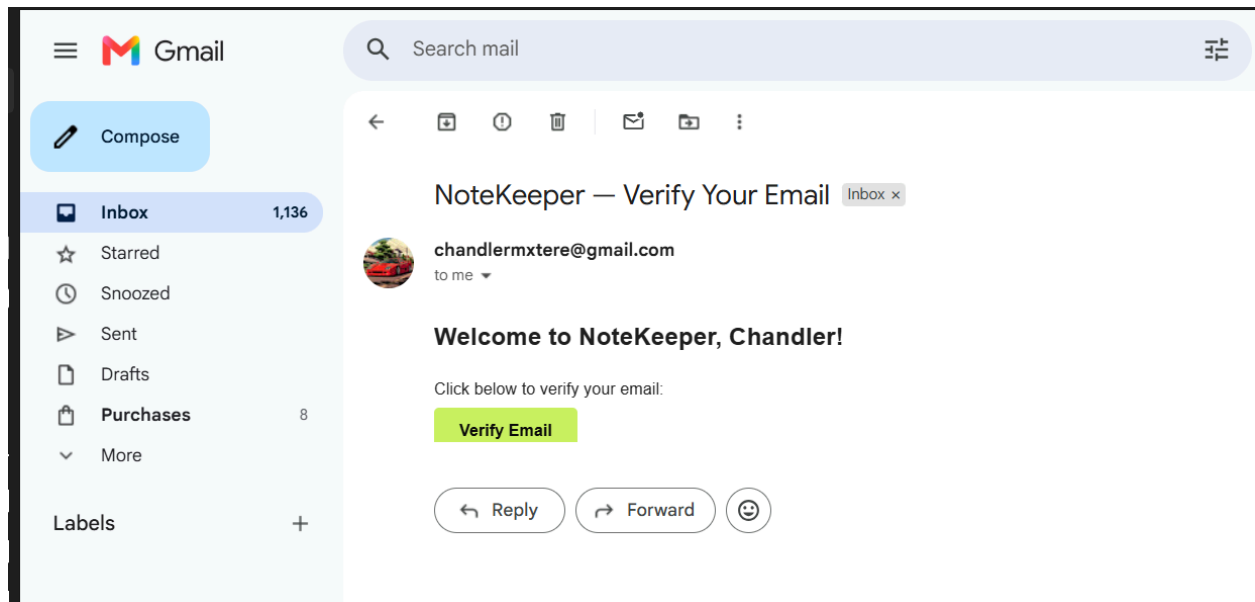


Fig 2: phpMyAdmin Browse view of the users table showing is_verified=1 confirming only verified users can create sessions

		id	username	email	password	role	is_verified	verify_token	reset_token	reset_expires	two_fa_secret
		0ddc47e4-8688-40c5-9ceb-ecd472e0817a	Lecturer	chandlermxtere+lecturer@gmail.com	Lecturer2026#@	lecturer	1	NULL	NULL	NULL	NULL
		2ebe46de-9770-43d0-b472-2abe53ae4fd2	Admin	chandlermxtere+admin@gmail.com	Admin2026#@	administrator	1	NULL	NULL	NULL	FA4WIUKOGBEHQ
		a9d8ba83-e20d-4bd5-baa0-747a17c391b1	John	chandlermxtere+student@gmail.com	Student2026#@	student	1	NULL	NULL	NULL	NULL
		e584e197-60cf-466e-b593-850786fbfedb	Matere	chandlermxtere@gmail.com	Recognize7-Shingle3-Splashing9-Pretended9-Bronco6	student	1	NULL	NULL	NULL	KBCXIKTHFJJSQ

Student Summary

NoteKeeper was built using Node.js, Express and MySQL managed through VS Code as the development environment. A user receives a session token when they successfully log in and the server generates a unique UUID, inserts it into the MySQL sessions table and returns it to the frontend. This approach

mirrors Java Servlet session management where the server calls `request.getSession()` to create a persistent session tied to a unique ID. The session lifecycle follows the same three phases — initialization on login, active state during use and destruction on logout.

Week Reflection

Both Node.js session management and Java Servlet `HttpSession` solve the same fundamental problem — HTTP is stateless so both architectures create a persistent connection state that survives across multiple requests. The token is stored in `localStorage` to give the frontend explicit control over when and how it is attached to backend requests, which is the standard approach for modern single-page applications. The `requireAuth` middleware blocks any request without a valid token by returning a 401 Unauthorized error, which signals the frontend to redirect the user back to the login page. On logout the token is permanently deleted from both `localStorage` and the MySQL sessions table, completely terminating that specific session.

WEEK 10: Dynamic HTML Generation and JDBC Basics

Practical Activity: Dynamic HTML and JDBC Basics

Fig 1: phpMyAdmin showing the notes_app database with all three tables listed demonstrating the database creation equivalent to creating studentdb

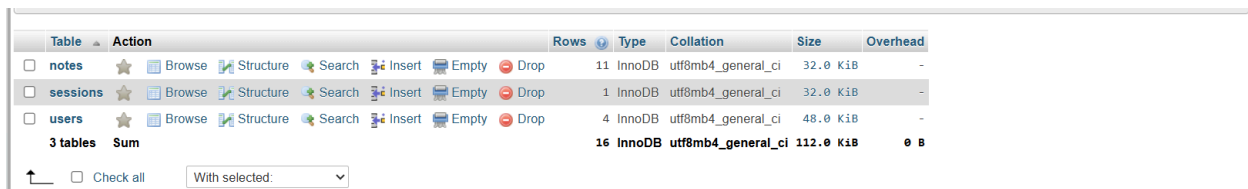
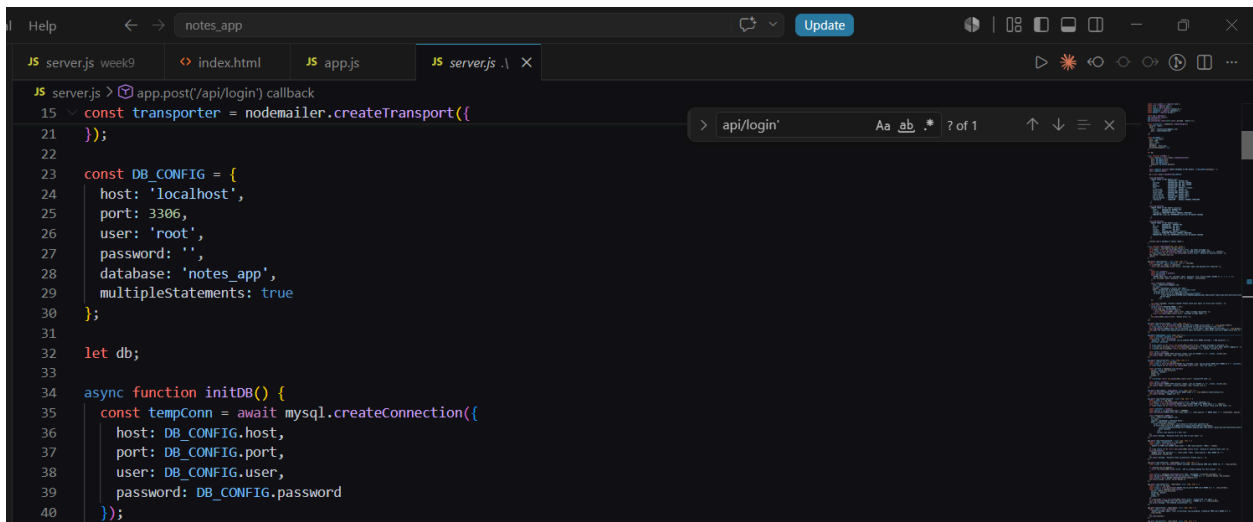


Table	Action	Rows	Type	Collation	Size	Overhead
☐ notes		11	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ sessions		1	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ users		4	InnoDB	utf8mb4_general_ci	48.0 KiB	-
3 tables Sum		16	InnoDB	utf8mb4_general_ci	112.0 KiB	0 B

☐ Check all With selected:

Fig 2: VS Code showing the DB_CONFIG object and initDB() function in server.js demonstrating the MySQL connection equivalent to JDBC connection setup



```
JS server.js week9 > index.html JS app.js JS server.js .\ X
JS server.js > app.post(/api/login) callback
15 const transporter = nodemailer.createTransport({
21 });
22
23 const DB_CONFIG = {
24   host: 'localhost',
25   port: 3306,
26   user: 'root',
27   password: '',
28   database: 'notes_app',
29   multipleStatements: true
30 };
31
32 let db;
33
34 async function initDB() {
35   const tempConn = await mysql.createConnection({
36     host: DB_CONFIG.host,
37     port: DB_CONFIG.port,
38     user: DB_CONFIG.user,
39     password: DB_CONFIG.password
40   });
```


Fig 3: VS Code showing the CREATE TABLE statements in server.js demonstrating how the users and notes tables are created programmatically on server startup

```
47 await db.execute(`
48   CREATE TABLE IF NOT EXISTS users (
49     id          VARCHAR(36) PRIMARY KEY,
50     username    VARCHAR(100) NOT NULL UNIQUE,
51     email       VARCHAR(255) NOT NULL UNIQUE,
52     password    VARCHAR(100) NOT NULL,
53     role        VARCHAR(20)  DEFAULT 'student',
54     is_verified TINYINT(1)   DEFAULT 0,
55     verify_token VARCHAR(100) DEFAULT NULL,
56     reset_token  VARCHAR(100) DEFAULT NULL,
57     reset_expires DATETIME   DEFAULT NULL,
58     two_fa_secret VARCHAR(100) DEFAULT NULL,
59     two_fa_enabled TINYINT(1) DEFAULT 0,
60     created_at   TIMESTAMP   DEFAULT CURRENT_TIMESTAMP
61   )
62 `);
63
```

Fig 4: Browser showing the NoteKeeper note creation form with title, category and content fields demonstrating the web form used to insert records into the database

The screenshot displays the NoteKeeper web application interface. At the top, the header shows the user is logged in as 'Mateo' with the role 'STUDENT', and includes links for 'profile' and 'logout'. The main content area is divided into two sections. On the left, under 'MY NOTES', there is a search bar and a list of notes including 'Project Ideas for Portfolio', 'Database Design Notes', 'Books to Read This Month', and 'NoteKeeper Project Milestones'. On the right, the 'New Note' form is visible, featuring a 'Title' input field, a 'Category' dropdown menu set to 'General', and a large 'Content' text area. A 'Save Note' button is located at the bottom of the form.

Fig 5: Browser showing the NoteKeeper dashboard sidebar dynamically rendering all notes fetched from MySQL demonstrating dynamic HTML generation equivalent to JSP page display

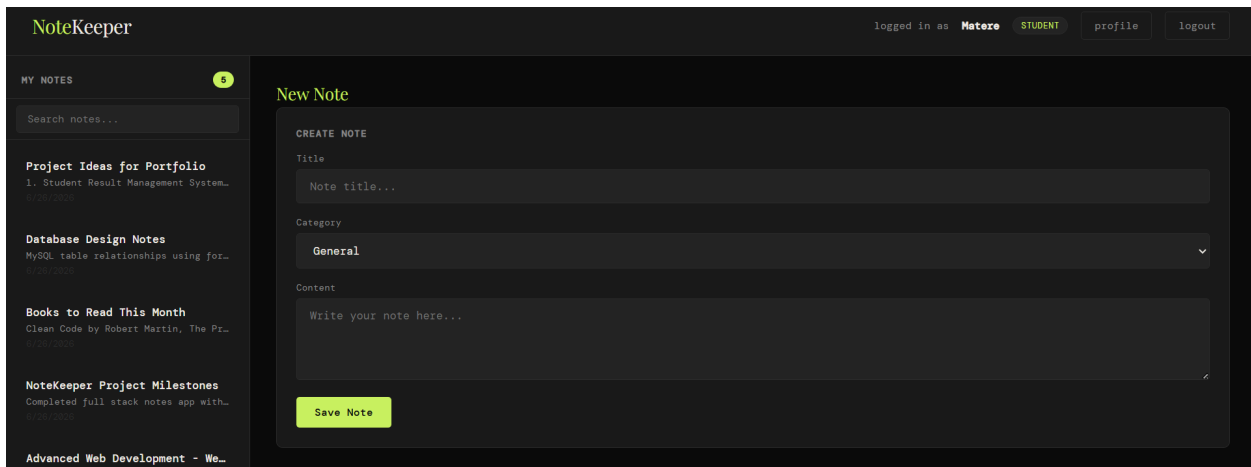


Fig 6: phpMyAdmin Browse view of the notes table showing all inserted records with their id, user_id, title, content, category and created_at columns confirming successful database insertion

New information_schema mysql notes_app New notes sessions users performance_schema phpmyadmin test Show all Number of rows: 25 Filter rows: Search this table Sort by key: None						
Extra options						
	id	user_id	title	content	category	created_at
☐	01cd24b3-2660-4176-8699-8dce0279f0c6	2ebe46de-9770-43d0-b472-7abe53ae4fd2	Platform Maintenance Log	MySQL database running on XAMPP locally Sessions ...	General	2026-06-26 05:26:35
☐	39c3b994-22ea-4fad-b0b6-0a949de5c261	e584e197-60cf-466e-b593-850786fbfdeb	Advanced Web Development - Week 7 Summary	Covered authentication systems including JWT token...	Study	2026-06-26 03:41:41
☐	752dc655-dbdd-497c-aa81-e425de2347f6	e584e197-60cf-466e-b593-850786fbfdeb	Books to Read This Month	Clean Code by Robert Martin, The Pragmatic Program...	Personal	2026-06-26 03:43:07
☐	7df2311e-06b8-4a82-8c9a-923ca094dbdb	2ebe46de-9770-43d0-b472-7abe53ae4fd2	Security Configuration	Email verification enabled via Nodemailer. Passwor...	Work	2026-06-26 05:26:07
☐	80e4a794-ceda-48d3-bb02-18f741e12f47	0ddc47e4-8688-40c5-9ceb-ecd472e0817a	BIT3208 Week 7 Observations	Students have demonstrated good understanding of a...	Work	2026-06-26 05:28:01

Creativity Section

Fig 1: Browser showing the Administrator dashboard with the dynamically generated users table pulled from MySQL confirming dynamic HTML generation based on database records

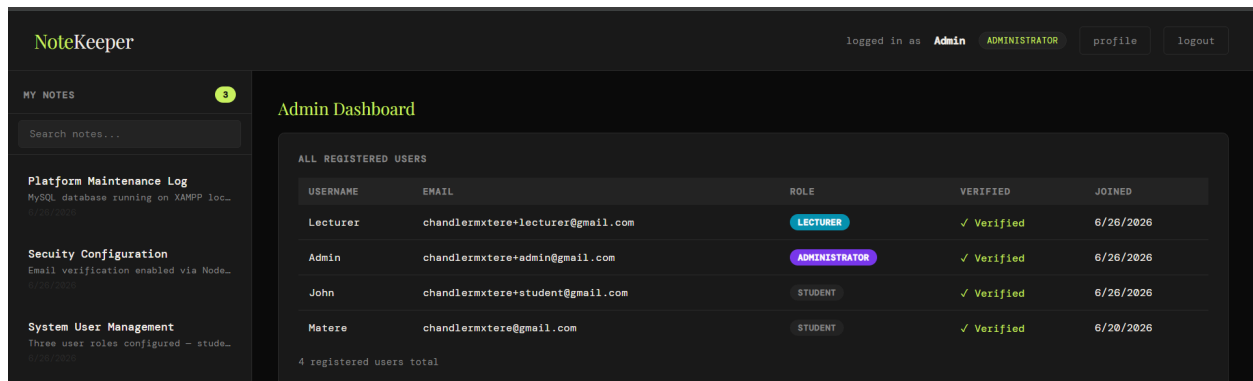
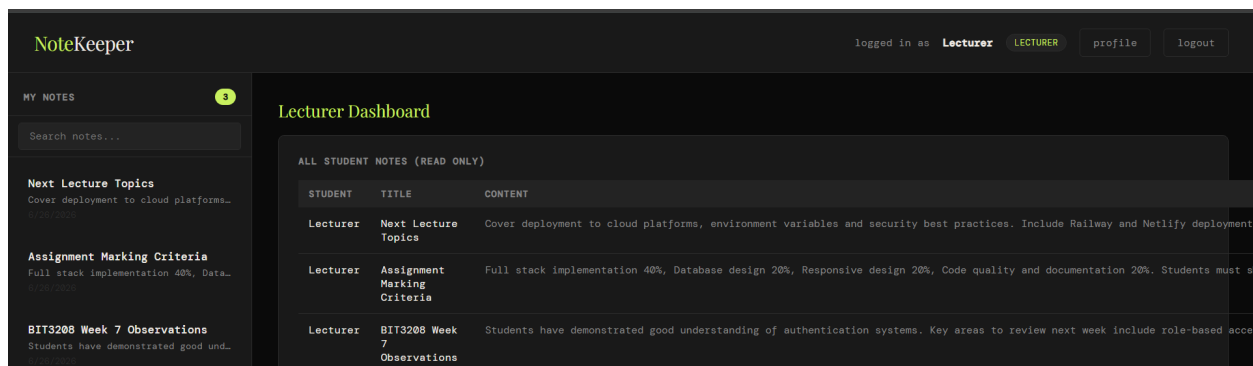


Fig 2: Browser showing the Lecturer dashboard with the dynamically generated student notes table confirming role-based dynamic content rendering from the database



Student Summary

NoteKeeper uses a MySQL database connected to the Node.js application using the mysql2 driver package, which handles all communication between the server and the database. Records were inserted by capturing user input from the HTML note creation form and passing the values into a parameterized SQL INSERT query executed by the server. When the dashboard loads, a SELECT query fetches all saved notes from the database and the results are rendered dynamically in the sidebar without a page refresh. The direct equivalent of Java's JDBC API in this stack is the mysql2 package, which provides the same connection, query execution and result handling capabilities.

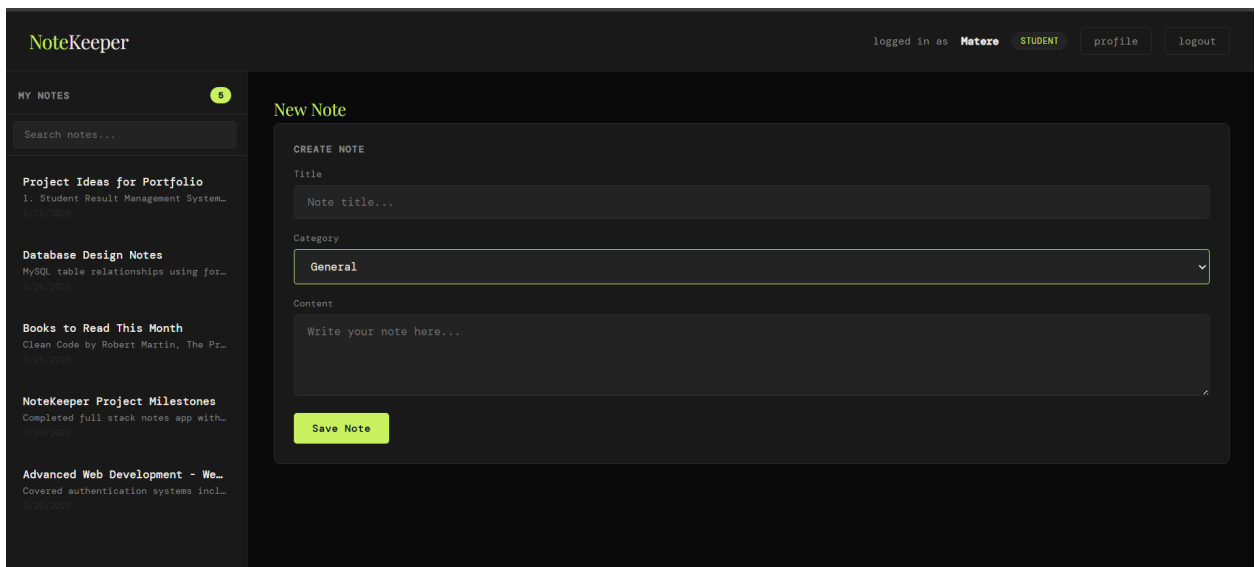
Week Reflection

Both the mysql2 driver in Node.js and JDBC in Java serve the same fundamental purpose bridging the application to the MySQL database to execute queries and return results. The initDB() function in server.js is the direct equivalent of opening a JDBC Statement and calling .executeUpdate("CREATE TABLE IF NOT EXISTS..."), ensuring tables exist before the application accepts any requests.

WEEK 11: Building Dynamic Pages with Full Database Integration (CRUD)

Practical Activity — Full CRUD Implementation

Fig 1: Browser showing the NoteKeeper note creation form demonstrating the Add Student equivalent with title, category and content fields



The screenshot displays the NoteKeeper web application interface. The top navigation bar includes the 'NoteKeeper' logo, a user status indicator 'logged in as Mateo STUDENT', and links for 'profile' and 'logout'. The left sidebar, titled 'MY NOTES', features a search bar and a list of notes including 'Project Ideas for Portfolio', 'Database Design Notes', 'Books to Read This Month', 'NoteKeeper Project Milestones', and 'Advanced Web Development - We...'. The main content area is titled 'New Note' and contains a 'CREATE NOTE' form. This form has three fields: 'Title' (with placeholder 'Note title...'), 'Category' (a dropdown menu currently showing 'General'), and 'Content' (a text area with placeholder 'Write your note here...'). A green 'Save Note' button is positioned at the bottom of the form.

Fig 2: Browser showing the sidebar listing all notes dynamically fetched from MySQL demonstrating the View All Students equivalent

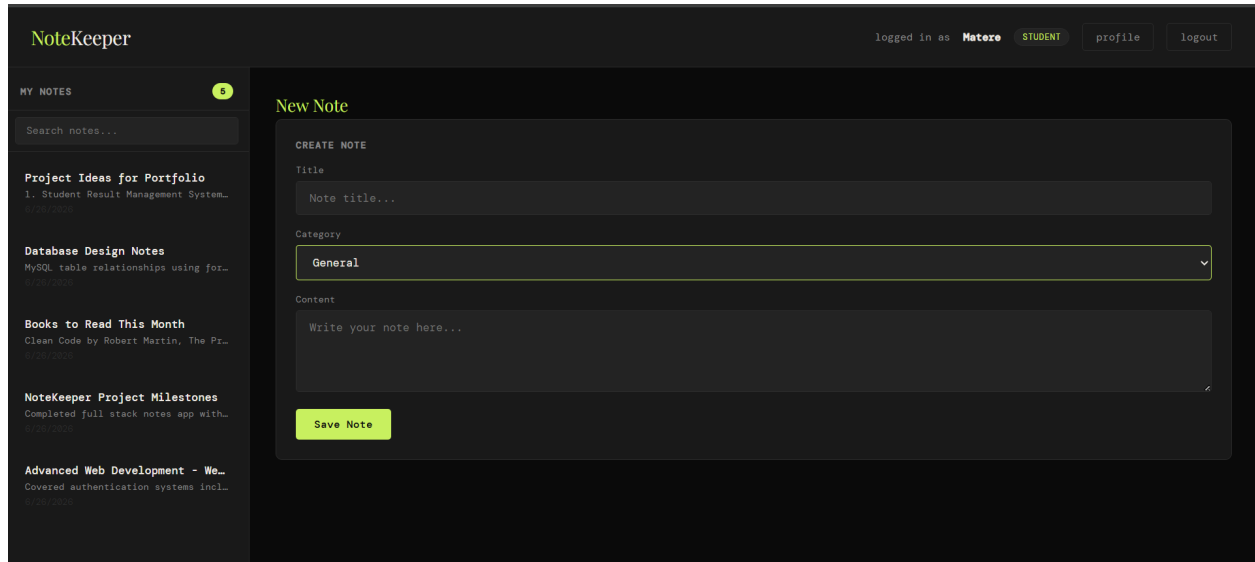
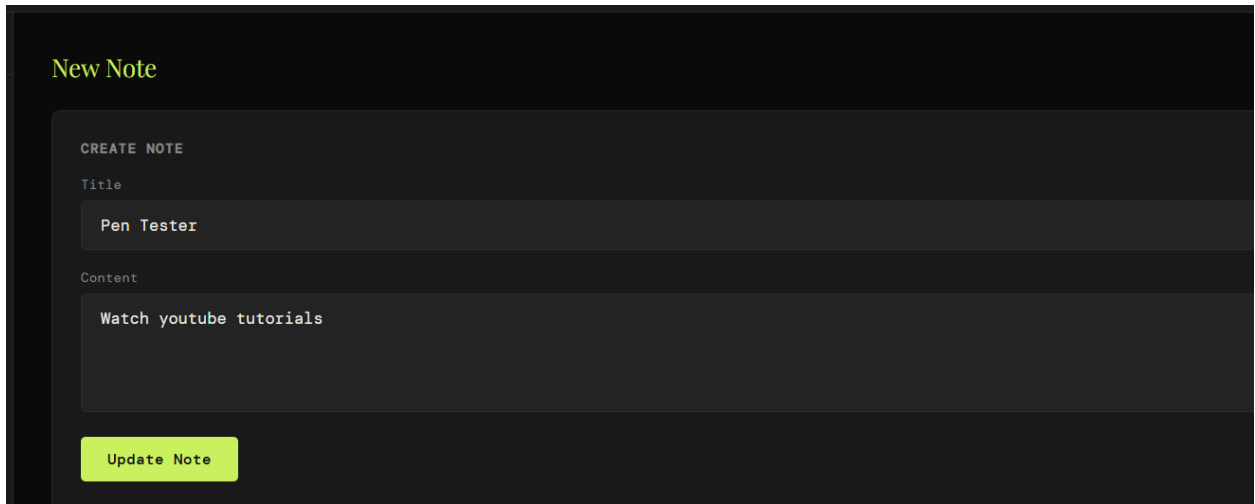
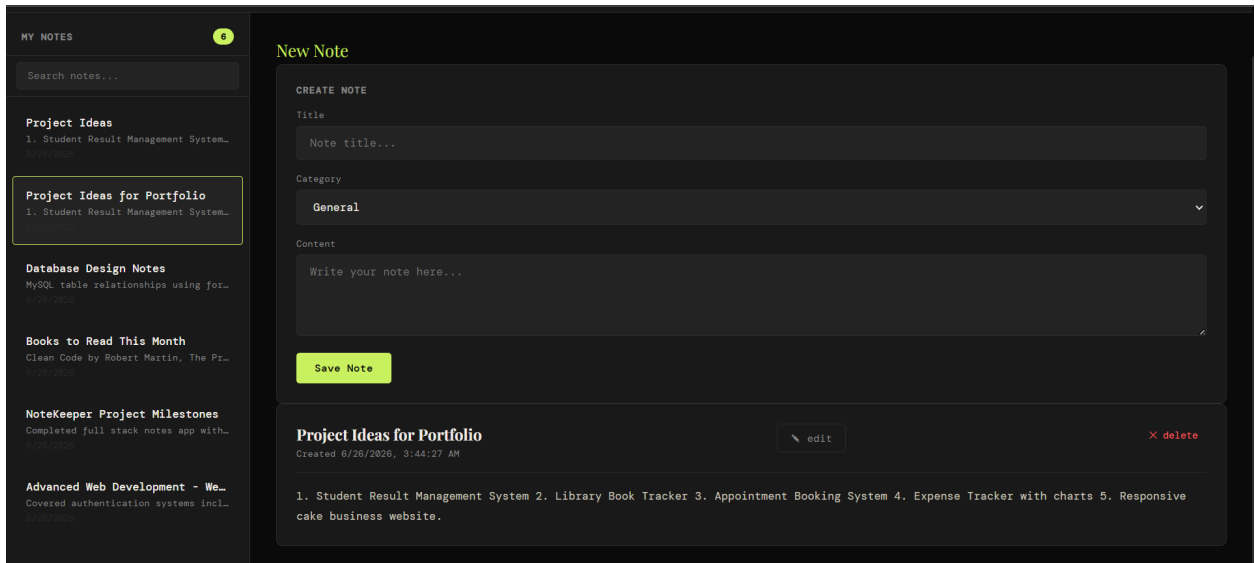


Fig 3: Browser showing the edit form pre-filled with existing note data and the Update Note button demonstrating the Update Student Details equivalent



The screenshot shows a web application interface with a dark theme. At the top left, there is a header 'New Note' in a light blue font. Below it, a form titled 'CREATE NOTE' is displayed. The form has two main sections: 'Title' and 'Content'. The 'Title' section contains a text input field with the value 'Pen Tester'. The 'Content' section contains a text area with the value 'Watch youtube tutorials'. At the bottom of the form, there is a blue button labeled 'Update Note'.

Fig 4: Browser showing the delete confirmation dialog appearing before a note is removed demonstrating the Delete with Confirmation requirement



The screenshot shows a web application interface with a dark theme. On the left, there is a sidebar with the title 'MY NOTES' and a count of 6. Below the title, there is a search bar and a list of notes. The first note is 'Project Ideas' with a sub-item '1. Student Result Management System...'. The second note is 'Project Ideas for Portfolio' with a sub-item '1. Student Result Management System...'. The third note is 'Database Design Notes' with a sub-item 'MySQL table relationships using for...'. The fourth note is 'Books to Read This Month' with a sub-item 'Clean Code by Robert Martin, The Pr...'. The fifth note is 'NoteKeeper Project Milestones' with a sub-item 'Completed full stack notes app with...'. The sixth note is 'Advanced Web Development - We...' with a sub-item 'Covered authentication systems incl...'. On the right, there is a 'New Note' form. The form has a title 'CREATE NOTE' and a text input field for 'Title' with the value 'Note title...'. Below the title, there is a dropdown menu for 'Category' with the value 'General'. Below the category, there is a text area for 'Content' with the value 'Write your note here...'. At the bottom of the form, there is a blue button labeled 'Save Note'. Below the form, there is a confirmation dialog for deleting a note. The dialog has a title 'Project Ideas for Portfolio' and a subtitle 'Created 6/26/2026, 3:44:27 AM'. It contains a list of notes: '1. Student Result Management System 2. Library Book Tracker 3. Appointment Booking System 4. Expense Tracker with charts 5. Responsive cake business website.' At the bottom of the dialog, there are two buttons: 'edit' and 'delete'.

Fig 5: Browser showing the search bar with a query typed and filtered notes appearing in real time demonstrating the Search by Name equivalent

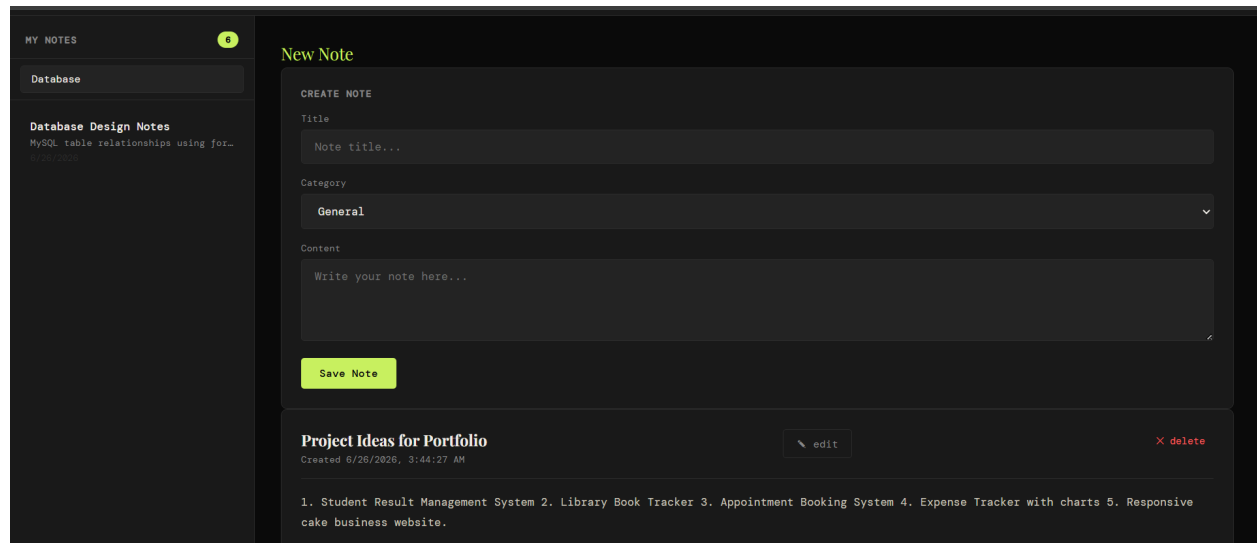
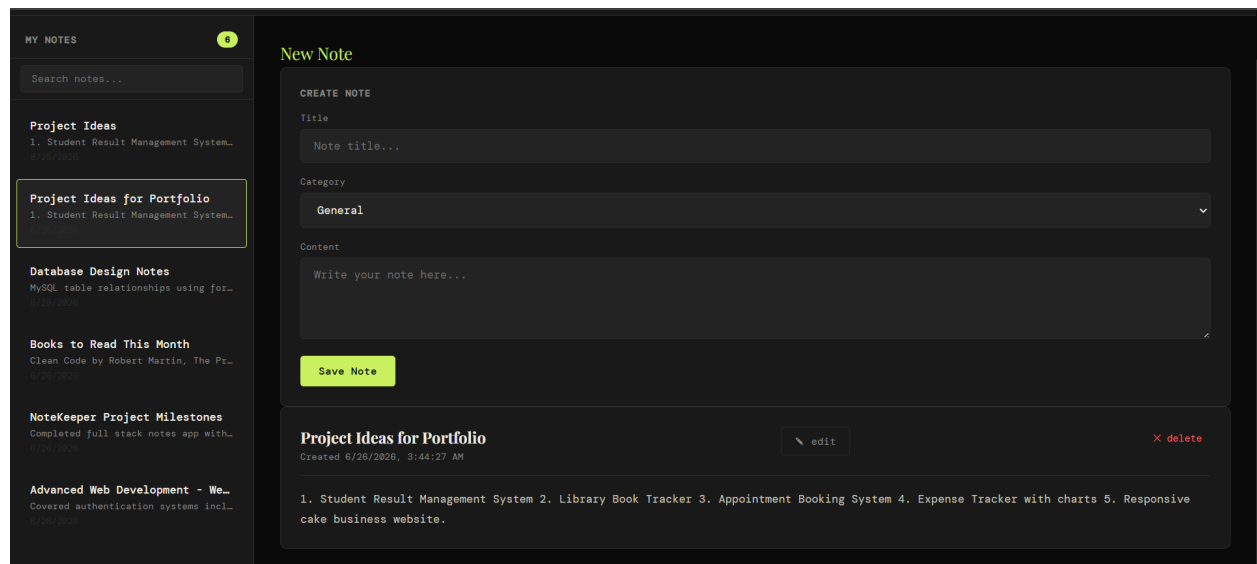


Fig 6: Browser showing the green success message after saving a note and the red error message after failed validation demonstrating success and error message display



Creativity Section

Fig 1: Browser showing the category dropdown with General, Work, Personal and Study options demonstrating course-based organization equivalent to searching students by course

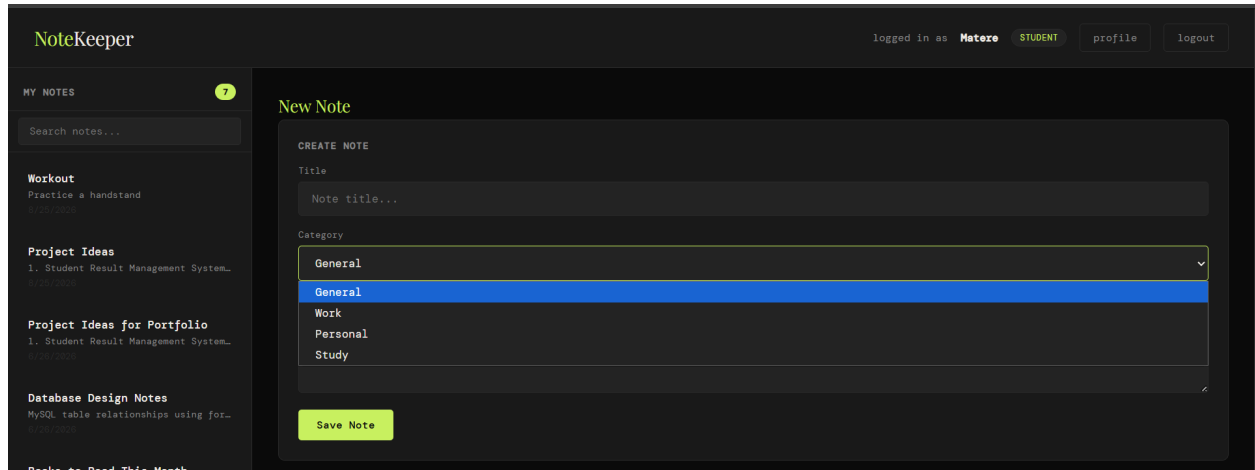


Fig 2: phpMyAdmin Browse view of the notes table showing the category column confirming category data is stored and retrievable from the database

Showing rows 0 - 12 (13 total, Query took 0.0004 seconds.)

SELECT * FROM `notes`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	user_id	title	content	category	created_at
☐ Edit ☐ Copy ☐ Delete	01cd24b3-2660-4176-8699-8dce0279f0c6	2ebe46de-9770-43d0-b472-2abe53ae4fd2	Platform Maintenance Log	MySQL database running on XAMPP locally. Sessions ...	General	2026-06-26 05:26:35
☐ Edit ☐ Copy ☐ Delete	39c3b994-22ea-4fad-b0b6-0a949de5c261	e584e197-60cf-466e-b593-850786fbfdb	Advanced Web Development - Week 7 Summary	Covered authentication systems including JWT token...	Study	2026-06-26 03:41:41
☐ Edit ☐ Copy ☐ Delete	68fc6176-e476-43f1-a448-dbb662c091f3	e584e197-60cf-466e-b593-850786fbfdb	Project Ideas	1. Student Result Management System 2. Library Boo...	General	2026-08-25 11:15:50
☐ Edit ☐ Copy ☐ Delete	752dc655-abdd-497c-aa81-e425de2347f6	e584e197-60cf-466e-b593-850786fbfdb	Books to Read This Month	Clean Code by Robert Martin, The Pragmatic Program...	Personal	2026-06-26 03:43:07
☐ Edit ☐ Copy ☐ Delete	7df2311e-06b8-4a82-8c9a-923ca094dadb	2ebe46de-9770-43d0-b472-2abe53ae4fd2	Security Configuration	Email verification enabled via Nodemailer. Passwor...	Work	2026-06-26 05:26:07
☐ Edit ☐ Copy ☐ Delete	80e4a794-ceda-48d3-bb02-18f741e12f47	0ddc47e4-8688-40c5-9ceb-ecd472e0817a	BIT3208 Week 7 Observations	Students have demonstrated good understanding of a...	Work	2026-06-26 05:28:01
☐ Edit ☐ Copy ☐ Delete	b053441c-290c-4110-9cf3-3e2e7b196db5	e584e197-60cf-466e-b593-850786fbfdb	NoteKeeper Project Milestones	Completed full stack notes app with Node.js, Expre...	Work	2026-06-26 03:42:34
☐ Edit ☐ Copy ☐ Delete	b16c8dcc-7b96-4485-85ec-f33a840ccb95	e584e197-60cf-466e-b593-850786fbfdb	Project Ideas for Portfolio	1. Student Result Management System 2. Library Boo...	General	2026-06-26 03:44:27
☐ Edit ☐ Copy ☐ Delete	c002e469-a7ee-4fce-ae47-9bf05e3c2b34	0ddc47e4-8688-40c5-9ceb-ecd472e0817a	Assignment Marking Criteria	Full stack implementation 40%, Database design 20%...	Work	2026-06-26 05:28:24

Student Reflection

This week all four CRUD operations were successfully implemented — Create, Read, Update and Delete — allowing full management of note records stored in the MySQL database. Records are searched by capturing user input from the frontend search bar and processing it using SQL **LIKE** operators with wildcard characters to match partial strings, equivalent to searching students by name in a Java JDBC system.

Week Summary

Both Node.js and Java JDBC secure database calls against SQL injection using parameterized queries, though Java enforces strict data typing through **PreparedStatement** while Node.js uses dynamic array binding with the **mysql2** driver. Both systems use the SQL **LIKE** operator with wildcard characters for search functionality.

Week 12: Web Programming

Task 1: [ASP.NET](#) Student Management Module

Fig 1: Browser showing the NoteKeeper login page demonstrating student login equivalent to ASP.NET login form connected to SQL Server

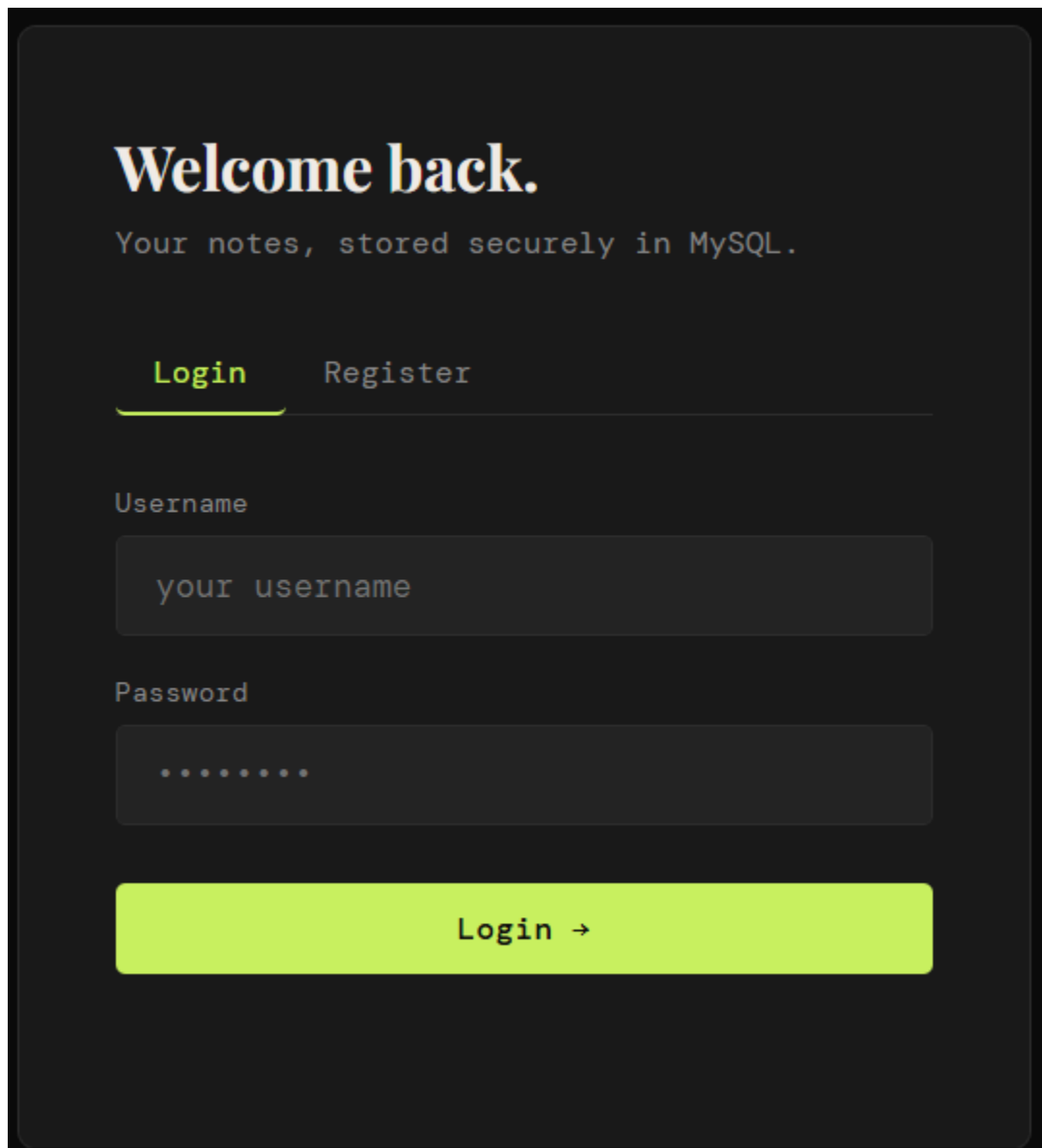
The image shows a login page for 'NoteKeeper' with a dark background. At the top, it says 'Welcome back.' in a large, bold, white font, followed by 'Your notes, stored securely in MySQL.' in a smaller, lighter font. Below this, there are two tabs: 'Login' (highlighted with a green underline) and 'Register'. The 'Login' section contains a 'Username' label above a text input field with the placeholder 'your username', and a 'Password' label above a password input field with ten dots. At the bottom, there is a large green button with the text 'Login →'.

Fig 2: Browser showing the NoteKeeper registration form with username, email, role dropdown and password fields demonstrating student registration with course selection equivalent to ASP.NET DropDownList

Welcome back.

Your notes, stored securely in MySQL.

[Login](#) [Register](#)

Username

choose a username

Email

you@example.com

Role

Student

Student

Lecturer

Administrator

Create Account →

Fig 3: Browser showing the NoteKeeper profile panel displaying username, email, role, verification status and member since date demonstrating the student dashboard with personal information

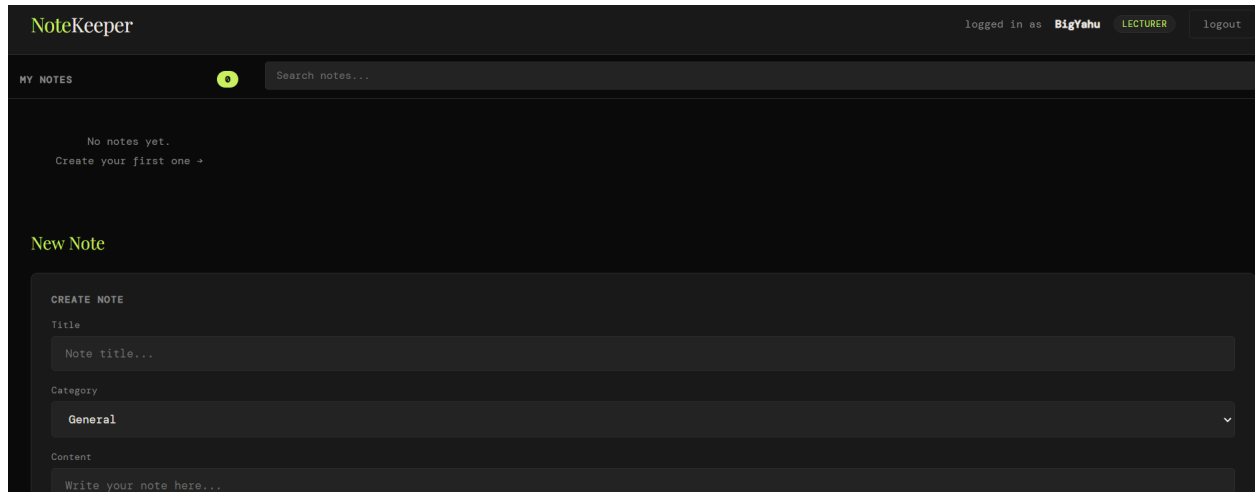


Fig 4: Browser showing the Administrator dashboard with the dynamically generated users table demonstrating the GridView equivalent showing all registered students

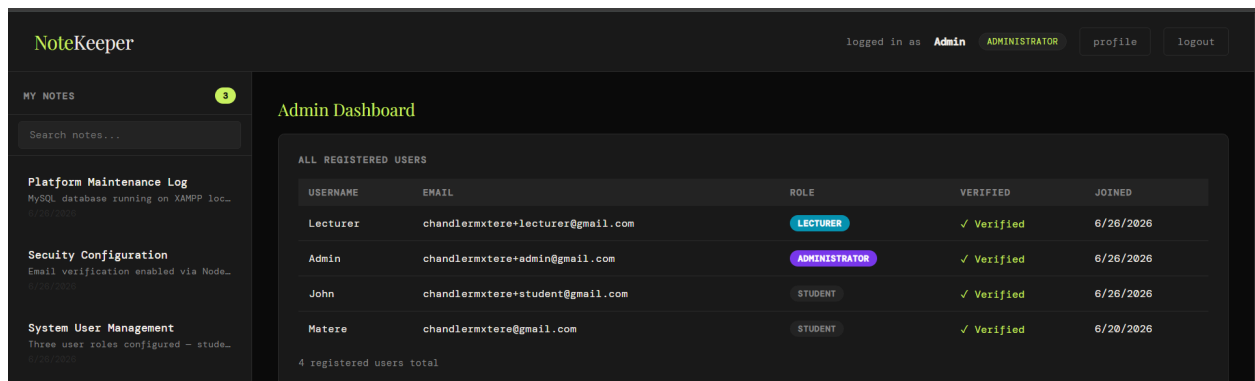


Fig 5: phpMyAdmin Browse view of the sessions table showing active session tokens demonstrating server side session management equivalent to ASP.NET Session object

	id	user_id	title	content	category	created_at
☐ Edit Copy Delete	01cd24b3-2660-4176-8699-8dce0279f0c6	2ebe46de-9770-43d0-b472-2abe53ae4fd2	Platform Maintenance Log	MySQL database running on XAMPP locally. Sessions ...	General	2026-06-26 05:26:35
☐ Edit Copy Delete	39c3b994-22ea-4fad-b0b6-0a949de5c261	e584e197-60cf-466e-b593-850786fbfedb	Advanced Web Development - Week 7 Summary	Covered authentication systems including JWT token...	Study	2026-06-26 03:41:41
☐ Edit Copy Delete	752dc655-dbdd-497c-aa81-e425de2347f6	e584e197-60cf-466e-b593-850786fbfedb	Books to Read This Month	Clean Code by Robert Martin, The Pragmatic Program...	Personal	2026-06-26 03:43:07
☐ Edit Copy Delete	7df2311e-06b8-4a82-8c9a-923ca094dbdb	2ebe46de-9770-43d0-b472-2abe53ae4fd2	Security Configuration	Email verification enabled via Nodemailer. Passwor...	Work	2026-06-26 05:26:07
☐ Edit Copy Delete	80e4a794-ceda-48d3-bb02-18f741e12f47	0ddc47e4-8688-40c5-9ceb-ecd472e0817a	BIT3208 Week 7 Observations	Students have demonstrated good understanding of a...	Work	2026-06-26 05:28:01

Student Reflection

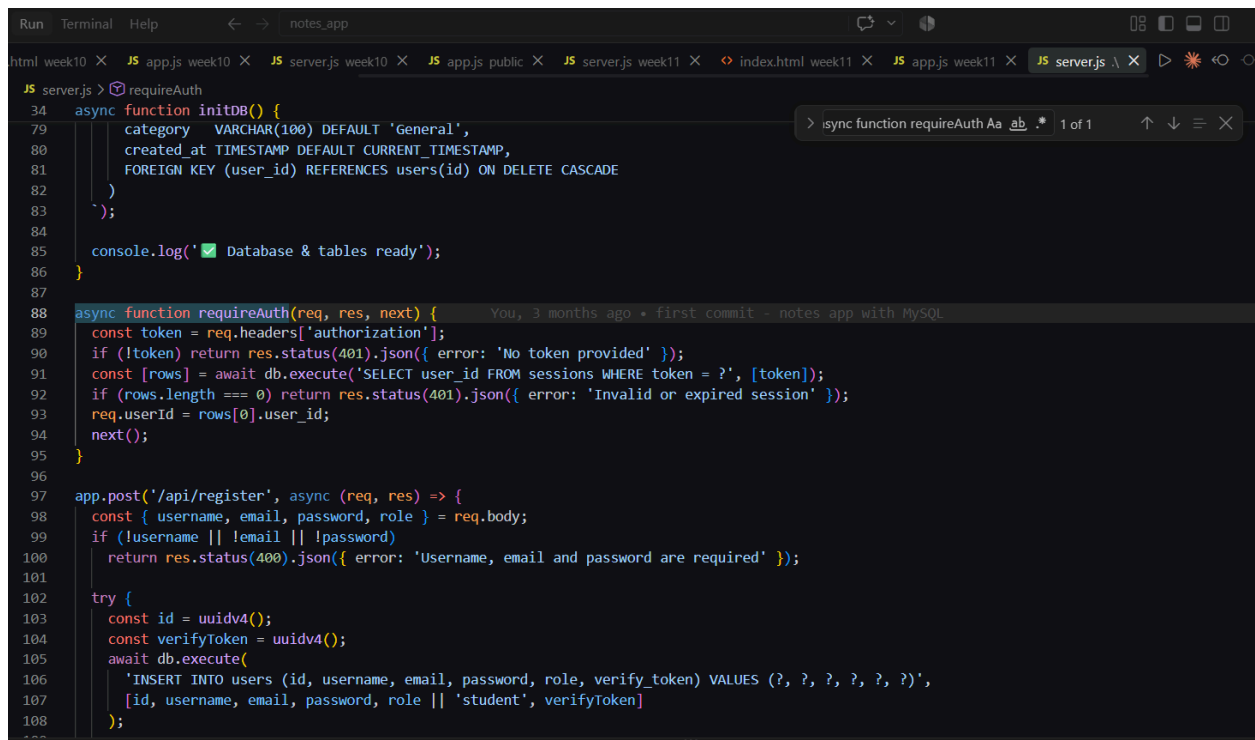
The main features of the student management module include secure user authentication, a centralized dashboard, full CRUD operations for managing note records and dynamic real-time search functionality. Session management is handled through Express middleware that validates a unique UUID token stored in the MySQL sessions table on every protected request.

Week Summary

While ASP.NET uses a built-in `Session` object and Java Servlets use `HttpSession` to natively store user state, Node.js is inherently stateless and relies on third-party middleware combined with a MySQL sessions table to replicate this behavior. The role dropdown is built using a native HTML `<select>` element rather than ASP.NET's `<asp:DropDownList>` server control, and the admin users table is constructed by manually querying MySQL.

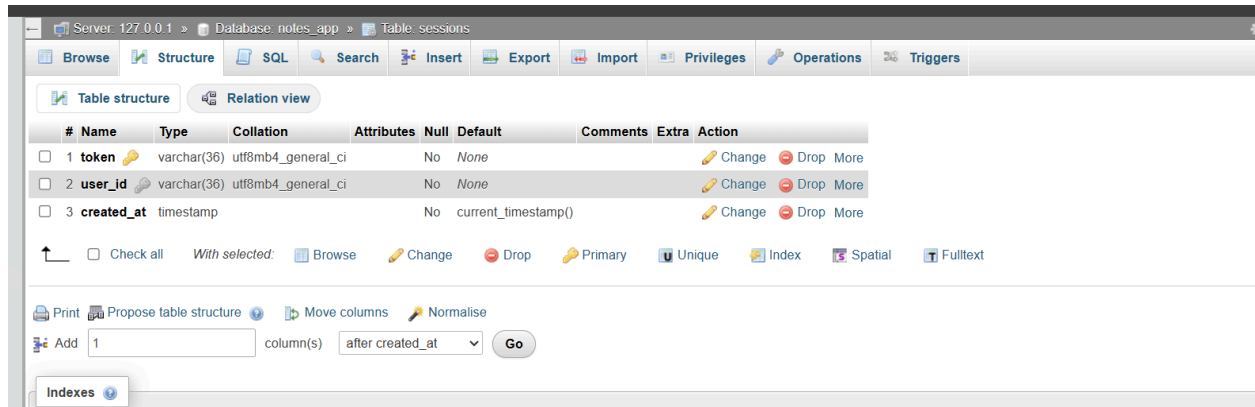
Creativity Section

Fig 1: VS Code showing the `requireAuth` middleware and session token validation in `server.js` demonstrating how Node.js session management compares to ASP.NET, Java Servlets and PHP session handling



```
Run Terminal Help  notes_app
html week10 X JS app.js week10 X JS server.js week10 X JS app.js public X JS server.js week11 X index.html week11 X JS app.js week11 X JS server.js X
JS server.js > requireAuth
34 async function initDB() {
79   category VARCHAR(100) DEFAULT 'General',
80   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
81   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
82 }
83 };
84
85 console.log('✅ Database & tables ready');
86 }
87
88 async function requireAuth(req, res, next) {
89   const token = req.headers['authorization'];
90   if (!token) return res.status(401).json({ error: 'No token provided' });
91   const [rows] = await db.execute('SELECT user_id FROM sessions WHERE token = ?', [token]);
92   if (rows.length === 0) return res.status(401).json({ error: 'Invalid or expired session' });
93   req.userId = rows[0].user_id;
94   next();
95 }
96
97 app.post('/api/register', async (req, res) => {
98   const { username, email, password, role } = req.body;
99   if (!username || !email || !password)
100     return res.status(400).json({ error: 'Username, email and password are required' });
101
102   try {
103     const id = uuidv4();
104     const verifyToken = uuidv4();
105     await db.execute(
106       'INSERT INTO users (id, username, email, password, role, verify_token) VALUES (?, ?, ?, ?, ?, ?)',
107       [id, username, email, password, role || 'student', verifyToken]
108     );
109   }
110 }
```

Fig 2: phpMyAdmin showing the sessions table structure with token, user_id and created_at columns demonstrating database-backed session storage as an alternative to server memory storage used in ASP.NET and Java



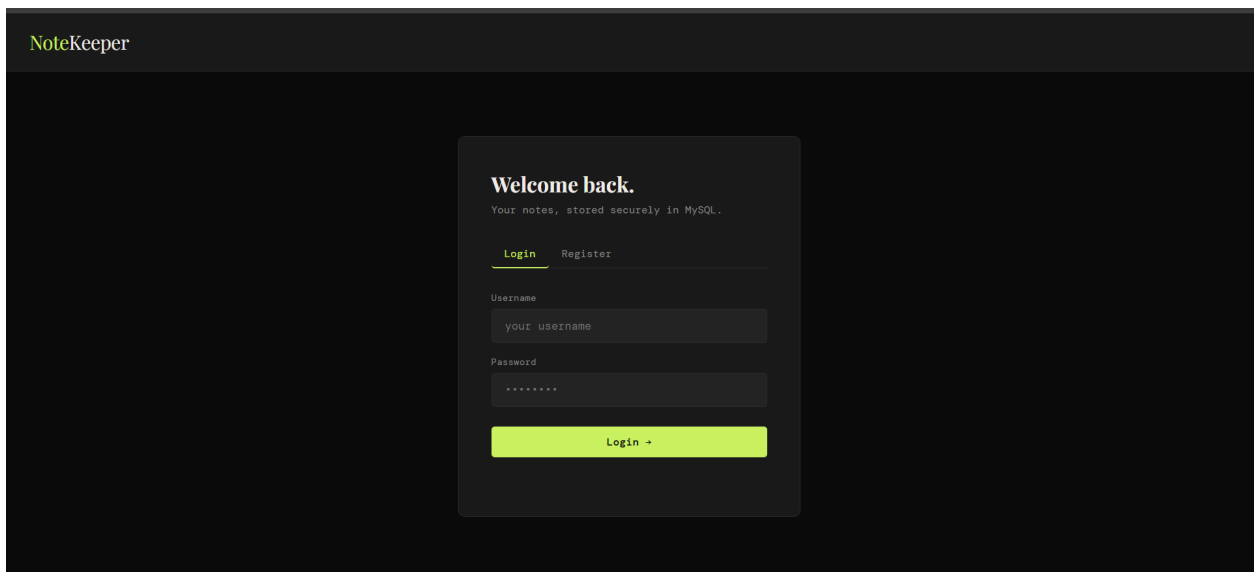
Comparison Table

Feature	ASP.NET	Java Servlets	PHP	NoteKeeper (Node.js)
Session Creation	Session["key"]	request.getSession()	session_start()	UUID token in MySQL
Session Storage	Server memory	Server memory	Server memory/files	MySQL database
Session Expiry	Configurable timeout	Configurable timeout	Configurable timeout	Manual on logout
Cookie Used	Yes (SessionID)	Yes (JSESSIONID)	Yes (PHPSESSID)	No — localStorage
Logout	Session.Abandon()	session.invalidate()	session_destroy()	DELETE from sessions table

WEEK 13: Special Topics in Web Programming: COM, CORBA, RMI, LDAP, and Search Engines

Assignment: Enterprise Level Student Management

Fig 1: Browser showing the NoteKeeper login form



The screenshot shows a web browser displaying the NoteKeeper login form. The page has a dark background. At the top left, the text "NoteKeeper" is visible in a light green font. In the center, there is a light gray rectangular box containing the login form. Inside this box, the text "Welcome back." is displayed in bold, followed by the subtitle "Your notes, stored securely in MySQL." Below this, there are two links: "Login" (underlined in green) and "Register". Under the "Login" link, there are two input fields: "Username" with the placeholder text "your username" and "Password" with placeholder dots. At the bottom of the form box is a green button with the text "Login →".

Fig 2: Browser showing the registration form with the role dropdown

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

choose a username

Email

you@example.com

Role

Student ▼

Student

Lecturer

Administrator

Create Account →

Fig 3: Browser showing the Administrator dashboard with the users table

The screenshot shows the NoteKeeper application running on localhost:3000. The user is logged in as Admin (ADMINISTRATOR). The dashboard is titled "Admin Dashboard". On the left sidebar, there are links for "MY NOTES", "Platform Maintenance Log", "Security Configuration", and "System User Management". The main content area displays a table titled "ALL REGISTERED USERS" with the following data:

USERNAME	EMAIL	ROLE	VERIFIED	JOINED
Lecturer	chandlermxtere+lecturer@gmail.com	LECTURER	✓ Verified	6/26/2026
Admin	chandlermxtere+admin@gmail.com	ADMINISTRATOR	✓ Verified	6/26/2026
John	chandlermxtere+student@gmail.com	STUDENT	✓ Verified	6/26/2026
Matere	chandlermxtere@gmail.com	STUDENT	✓ Verified	6/26/2026

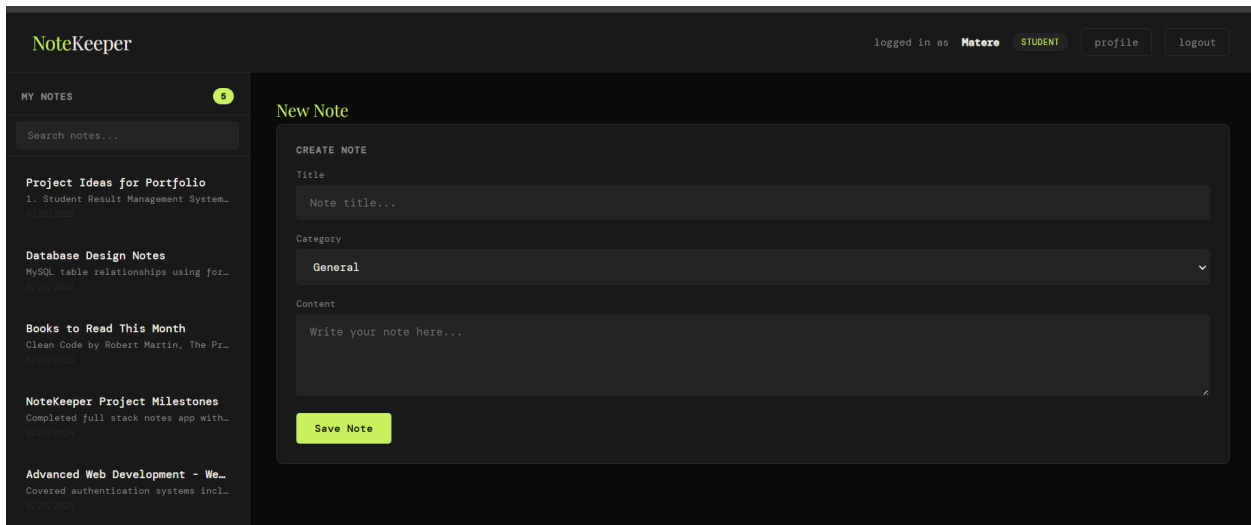
Below the table, it states "4 registered users total". There is also a "New Note" section with a "CREATE NOTE" form containing fields for Title and Category (set to General).

Fig 4: Browser showing the Lecturer dashboard with the read-only student notes panel

The screenshot shows the NoteKeeper application running on localhost:3000. The user is logged in as Lecturer (LECTURER). The dashboard is titled "Lecturer Dashboard". On the left sidebar, there are links for "MY NOTES", "Next Lecture Topics", "Assignment Marking Criteria", and "BIT3208 Week 7 Observations". The main content area displays a table titled "ALL STUDENT NOTES (READ ONLY)" with the following data:

STUDENT	TITLE	CONTENT
Lecturer	Next Lecture Topics	Cover deployment to cloud platforms, environment variables and security best practices. Include Railway and Netlify deployment
Lecturer	Assignment Marking Criteria	Full stack implementation 40%, Database design 20%, Responsive design 20%, Code quality and documentation 20%. Students must s
Lecturer	BIT3208 Week 7 Observations	Students have demonstrated good understanding of authentication systems. Key areas to review next week include role-based acco

Fig 5: Browser showing the Student dashboard with only their own notes visible



AUTHENTICATION COMPARISON TABLE

Feature	Java Servlets	PHP	ASP.NET	NoteKeeper Node.js
Session Creation	Server memory	Server/files	Server memory	MySQL database
Auth Middleware	Filter/Interceptor	Custom function	Authorization attribute	requireAuth middleware
Cookie Used	Yes (JSESSIONID)	Yes (PHPSESSID)	Yes (ASP.NET_SessionId)	No — localStorage
Role Management	Manual/Spring Security	Manual/framework	Roles/Claims	DELETE from sessions table
Password Reset	Manual implementation	Manual implementation	Identity framework	Nodemailer + UUID token
2FA Support	Manual/library	Manual/library	Identity framework	Speakeasy + Google Auth

Creativity Section: NoteKeeper as an LDAP-Equivalent Authentication System

Fig 1: Browser showing the email verification flow registration success message and Gmail inbox with verification email

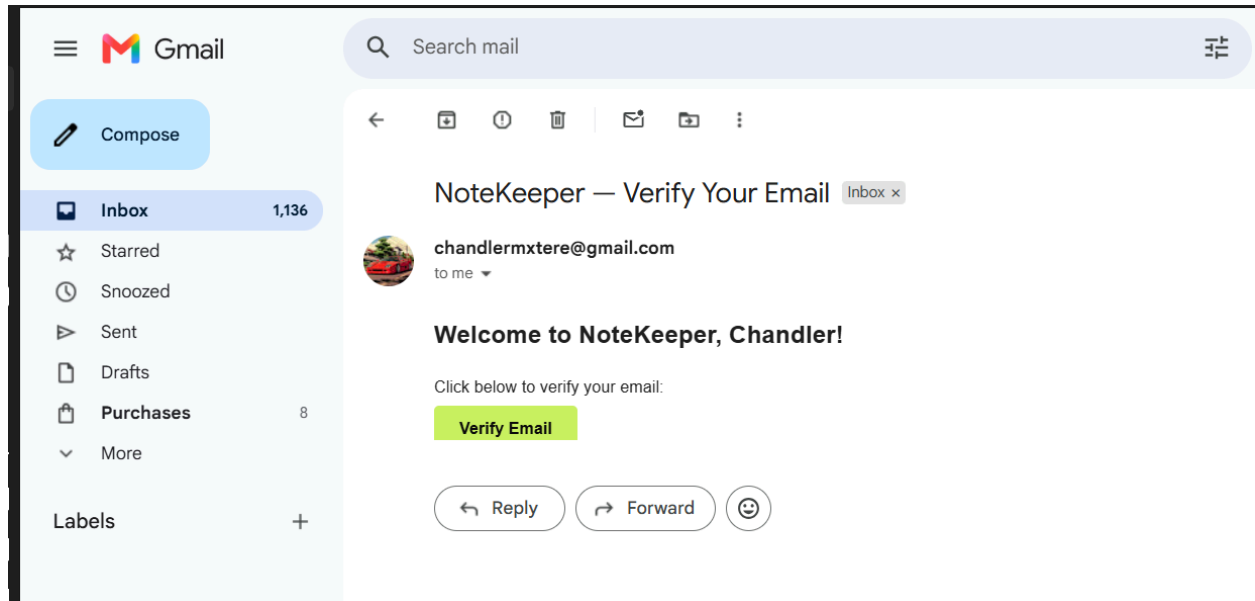
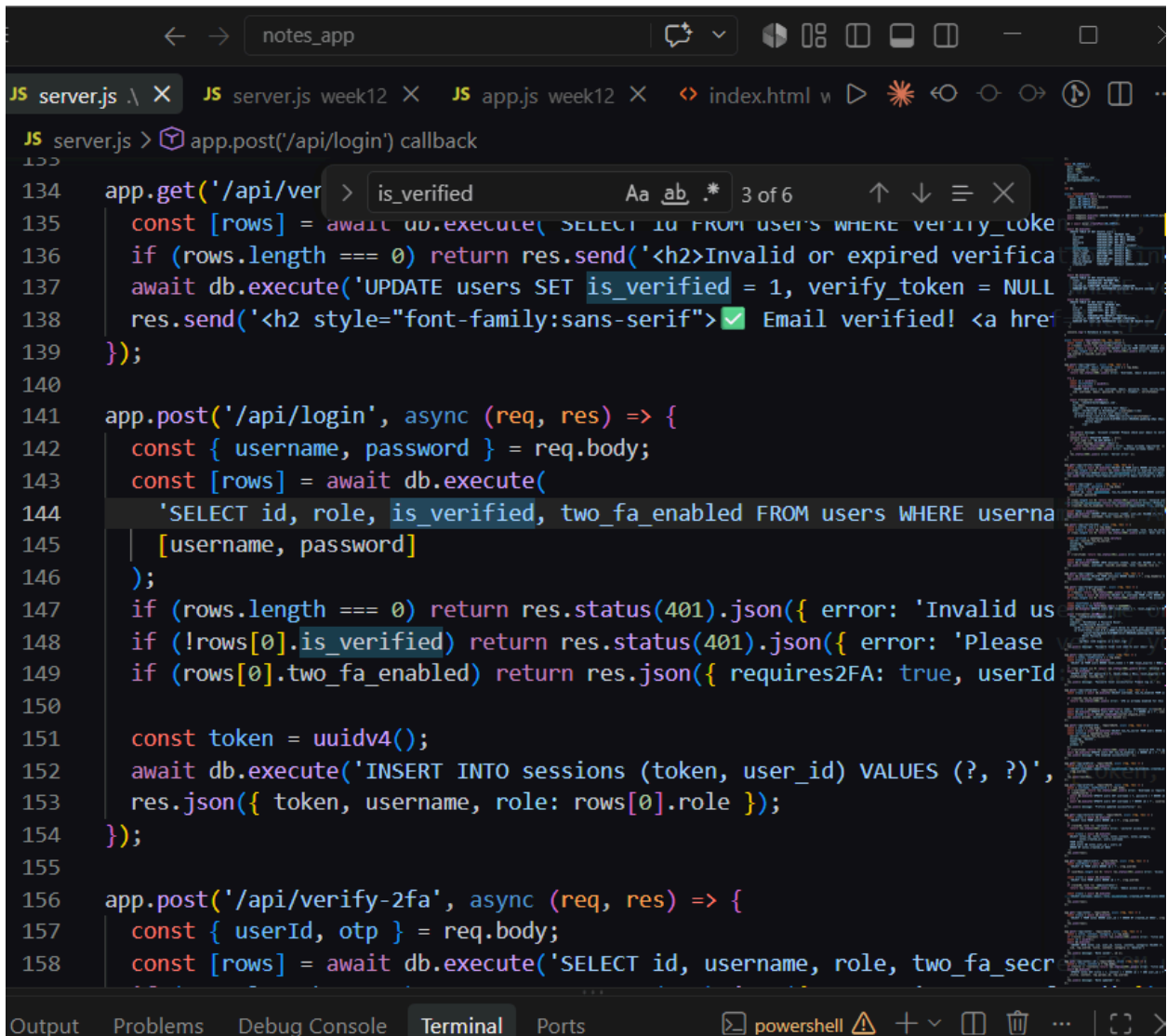


Fig 2: VS Code showing the login route in `server.js` with the `is_verified` check



The screenshot shows the VS Code editor with a file named `server.js` open. The code is written in JavaScript and uses Express.js for routing and MySQL for database operations. The `login` route is defined as an asynchronous function that receives a request and a response. It extracts the username and password from the request body and queries the database for a user with those credentials. If no user is found, it returns a 401 status with an 'Invalid user' error. If the user is found but is not verified (`is_verified` is false), it returns a 401 status with a 'Please verify your email' error. If the user is verified, it generates a token, inserts it into the sessions table, and returns the token, username, and role. The `verify-2fa` route is also partially visible, showing it receives a user ID and an OTP.

```
JS server.js > app.post('/api/login') callback
134 app.get('/api/verify', async (req, res) => {
135   const [rows] = await db.execute('SELECT id FROM users WHERE verify_token = ?');
136   if (rows.length === 0) return res.send('<h2>Invalid or expired verification token');
137   await db.execute('UPDATE users SET is_verified = 1, verify_token = NULL WHERE id = ?');
138   res.send('<h2 style="font-family:sans-serif">✔ Email verified! <a href="/api/login">Login now');
139 });
140
141 app.post('/api/login', async (req, res) => {
142   const { username, password } = req.body;
143   const [rows] = await db.execute(
144     'SELECT id, role, is_verified, two_fa_enabled FROM users WHERE username = ? AND password = ?'
145   );
146   if (rows.length === 0) return res.status(401).json({ error: 'Invalid user or password' });
147   if (!rows[0].is_verified) return res.status(401).json({ error: 'Please verify your email' });
148   if (rows[0].two_fa_enabled) return res.json({ requires2FA: true, userId: rows[0].id });
149
150   const token = uuidv4();
151   await db.execute('INSERT INTO sessions (token, user_id) VALUES (?, ?)', [token, rows[0].id]);
152   res.json({ token, username, role: rows[0].role });
153 });
154
155
156 app.post('/api/verify-2fa', async (req, res) => {
157   const { userId, otp } = req.body;
158   const [rows] = await db.execute('SELECT id, username, role, two_fa_secret FROM users WHERE id = ?');
```

Student Summary

The application implements foundational enterprise-level features including secure user authentication with email verification, role-based access control across three user types and dynamic real-time search functionality. Role-based access is enforced by storing user roles in the MySQL database and evaluating privileges on every protected request.

Week Reflection

While the application's email verification validates identity against a local database, LDAP provides a centralized directory enabling Single Sign-On across multiple enterprise systems, and the custom role-based access here manages privileges manually via database queries whereas Java and ASP.NET use robust frameworks like Servlet Filters and ASP.NET Identity to enforce access controls.